

# Part IV

## Gameplay



# Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

## Introduction to Gameplay Systems

Up until now, everything we've talked about in this book has focused on technology. We've learned that a game engine is a complex, layered software system built on top of the hardware, drivers and operating system of the target machine. We've seen how low-level engine systems provide services that are required by the rest of the engine; how human interface devices such as joypads, keyboards, mice and other devices can allow a human player to provide inputs to the engine; how the rendering engine produces 3D images on-screen; how the animation system allows characters and objects to move naturally; how the collision system detects and resolves interpenetrations between shapes; how the physics simulation causes objects to move in physically realistic ways; how the 3D audio engine renders a believable and immersive soundscape for our game world. But despite the wide range of powerful features provided by these components, if we were to put them all together, we *still* wouldn't have a game!

A game is defined not by its technology but by its *gameplay*. Gameplay can be defined as the overall experience of playing a game. The term *game mechanics* pins down this idea a bit more concretely—it is usually defined as the set of *rules* that govern the interactions between the various entities in the game. It also defines the *objectives* of the player(s), *criteria* for success and failure, the player character's *abilities*, the number and types of *non-player entities*

that exist within the game's virtual world and the *overall flow* of the gaming experience as a whole. In many games, these elements are intertwined with a compelling story and a rich cast of characters. However, story and characters are definitely not a necessary part of every video game, as evidenced by wildly successful puzzle games like *Tetris*. In their paper, "A Survey of 'Game' Portability" (<http://www.dcs.shef.ac.uk/intranet/research/resmes/CS0705.pdf>), Ahmed BinSubaih, Steve Maddock and Daniela Romano of the University of Sheffield refer to the collection of software systems used to implement gameplay as a game's *G-factor*. In the next three chapters, we'll explore the crucial tools and engine systems that define and manage the *game mechanics* (a.k.a. *gameplay*, a.k.a. *G-factor*) of a game.

## 15.1 Anatomy of a Game World

Gameplay designs vary widely from genre to genre and from game to game. That said, most 3D games, and a good number of 2D games as well, conform more or less to a few basic structural patterns. We'll discuss these patterns in the following sections, but please keep in mind that there are bound to be games out there that do not fit neatly into this mold.

### 15.1.1 World Elements

Most video games take place in a two- or three-dimensional virtual *game world*. This world is typically comprised of numerous discrete elements. Generally, these elements fall into two categories: static elements and dynamic elements. Static elements include terrain, buildings, roads, bridges and pretty much anything that doesn't move or interact with gameplay in an active way. Dynamic elements include characters, vehicles, weaponry, floating power-ups and health packs, collectible objects, particle emitters, dynamic lights, invisible regions used to detect important events in the game, splines that define the paths of objects and so on. This breakdown of the game world is illustrated in Figure 15.1.

Gameplay is generally concentrated within the dynamic elements of a game. Clearly, the layout of the static background plays a crucial role in how the game plays out. For example, a cover-based shooter wouldn't be very much fun if it were played in a big, empty, rectangular room. However, the software systems that implement gameplay are primarily concerned with updating the locations, orientations and internal states of the dynamic elements, since they are the elements that change over time. The term *game state* refers to the current state of all dynamic game world elements taken as a whole.





Figure 15.1. A game world from *Uncharted: The Lost Legacy* (© 2017™ SIE. Created and developed by Naughty Dog, PlayStation 4) showing static and dynamic elements.

The ratio of dynamic to static elements also varies from game to game. Most 3D games consist of a relatively small number of dynamic elements moving about within a relatively large static background area. Other games, like the arcade classic *Asteroids* or the Xbox 360 retro hit *Geometry Wars*, have no static elements to speak of (other than a black screen). The dynamic elements of a game are usually more expensive than the static elements in terms of CPU resources, so most 3D games are constrained to a limited number of dynamic elements. However, the higher the ratio of dynamic to static elements, the more “alive” the game world can seem to the player. As gaming hardware becomes more and more powerful, games are achieving higher and higher dynamic-to-static ratios.

It’s important to note that the distinction between the dynamic and static elements in a game world is often a bit blurry. For example, in the arcade game *Hydro Thunder*, the waterfalls were dynamic in the sense that their textures were animated, they had dynamic mist effects at their bases, and they could be placed into the game world and positioned by a game designer independently of the terrain and water surface. However, from an engineering standpoint, waterfalls were treated as static elements because they did not interact with the

boats in the race in any way (other than to obscure the player's view of hidden boost power-ups and secret passageways). Different game engines draw different lines between static and dynamic elements, and some don't draw a distinction at all (i.e., everything is potentially a dynamic element).

The distinction between static and dynamic serves primarily as an optimization tool—we can do less work when we know that the state of an object isn't going to change. For example, when we know a mesh is static and will never move, its lighting can be precomputed in the form of static vertex lighting, light maps, shadow maps, static ambient occlusion information or precomputed radiance transfer (PRT) spherical harmonics coefficients. Virtually any computation that must be done at runtime for a dynamic world element is a good candidate for precomputation or omission when applied to a static element.

Games with destructible environments are an example of how the line between the static and dynamic elements in a game world can blur. For instance, we might define three versions of every static element—an undamaged version, a damaged version, and a fully destroyed version. These background elements act like static world elements most of the time, but they can be swapped dynamically during an explosion to produce the illusion of becoming damaged. In reality, static and dynamic world elements are just two extremes along a gamut of possible optimizations. Where we draw the line between the two categories (if we draw one at all) shifts as our optimization methodologies change and adapt to the needs of the game design.

#### 15.1.1.1 Static Geometry

The geometry of a static world element is often defined in a tool like Maya. It might be one giant triangle mesh, or it might be broken up into discrete pieces. The static portions of the scene are sometimes built out of *instanced geometry*. Instancing is a memory conservation technique in which a relatively small number of unique triangle meshes are rendered multiple times throughout the game world, at different locations and orientations, in order to provide the illusion of variety. For example, a 3D modeler might create five different kinds of short wall sections and then piece them together in random combinations in order to construct miles of unique-looking walls.

Static visual elements and collision data might also be constructed from *brush geometry*. This kind of geometry originated with the Quake family of engines. A *brush* describes a shape as a collection of convex volumes, each bounded by a set of planes. Brush geometry is fast and easy to create and integrates well into a BSP-tree-based rendering engine. Brushes can be really useful for rapidly blocking out the contents of a game world. This allows

gameplay to be tested early, when it is cheap to do so. If the layout proves its worth, the art team can either texture map and fine-tune the brush geometry or replace it with more-detailed custom mesh assets. On the other hand, if the level requires redesign, the brush geometry can be easily revised without creating a lot of extra work for the art team.

### 15.1.2 World Chunks

When a game takes place in a very large virtual world, it is typically divided into discrete playable regions, which we'll call *world chunks*. Chunks are also known as *levels*, *maps*, *stages* or *areas*. The player can usually see only a handful of chunks at any given moment while playing the game, and he or she progresses from chunk to chunk as the game unfolds.

Originally, the concept of "levels" was invented as a mechanism to provide greater variety of gameplay within the memory limitations of early gaming hardware. Only one level could exist in memory at a time, but the player could progress from level to level for a much richer overall experience. Since then, game designs have branched out in many directions, and linear level-based games are much less common today. Some games are essentially still linear, but the delineations between world chunks are usually not as obvious to the player as they once were. Other games use a star topology, in which the player starts in a central hub area and can access other areas at random from the hub (perhaps only after they have been unlocked). Others use a graph-like topology, where areas are connected to one another in arbitrary ways. Still others provide the illusion of a vast, open world, and use level-of-detail (LOD) techniques to reduce memory overhead and improve performance.

Despite the richness of modern game designs, all but the smallest of game worlds are still divided into chunks of some kind. This is done for a number of reasons. First of all, memory limitations are still an important constraint (and will be until game machines with infinite memory hit the market!). World chunks are also a convenient mechanism for controlling the overall flow of the game. Chunks can serve as a division-of-labor mechanism as well; each chunk can be constructed and managed by a relatively small group of designers and artists. World chunks are illustrated in Figure 15.2.

### 15.1.3 High-Level Game Flow

A game's *high-level flow* defines a sequence, tree or graph of player *objectives*. Objectives are sometimes called *tasks*, *stages*, *levels* (a term that can also apply to world chunks) or *waves* (if the game is primarily about defeating hordes of attacking enemies). The high-level flow also provides the definition of success for each objective (e.g., clear all the enemies and get the key) and the penalty

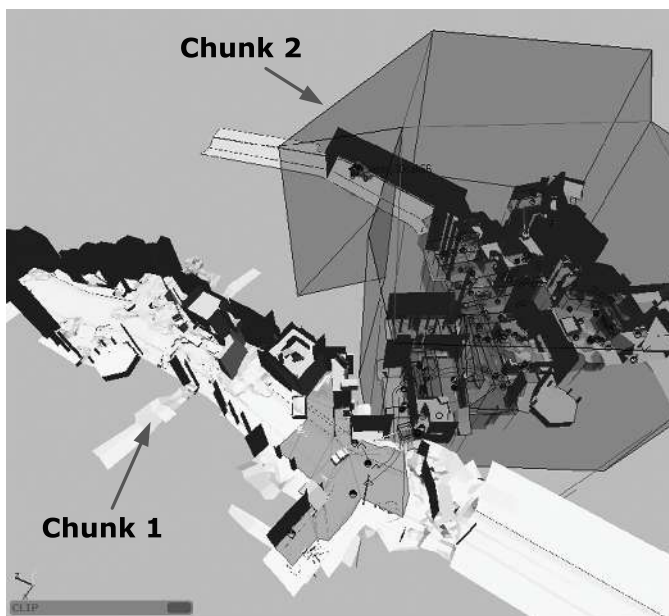


Figure 15.2. Many game worlds are divided into chunks for various reasons, including memory limitations, the need to control the flow of the game through the world, and as a division-of-labor mechanism during development.

for failure (e.g., go back to the start of the current area, possibly losing a “life” in the process). In a story-driven game, this flow might also include various in-game movies that serve to advance the player’s understanding of the story as it unfolds. These sequences are sometimes called *cut-scenes*, *in-game cinematics* (IGC) or *noninteractive sequences* (NIS). When they are rendered offline and played back as a full-screen movie, such sequences are usually called *full-motion videos* (FMV).

Early games mapped the objectives of the player one-to-one to particular world chunks (hence the dual meaning of the term “level”). For example, in *Donkey Kong*, each new level presents Mario with a new objective (namely, to reach the top of the structure and progress to the next level). However, this one-to-one mapping between world chunks and objectives is less popular in modern game design. Each objective is associated with one or more world chunks, but the coupling between chunks and objectives remains deliberately loose. This kind of design offers the flexibility to alter game objectives and world subdivision independently, which is extremely helpful from a logistic and practical standpoint when developing a game. Many games group their objectives into coarser sections of gameplay, often called *chapters* or *acts*. A typical gameplay architecture is shown in Figure 15.3.

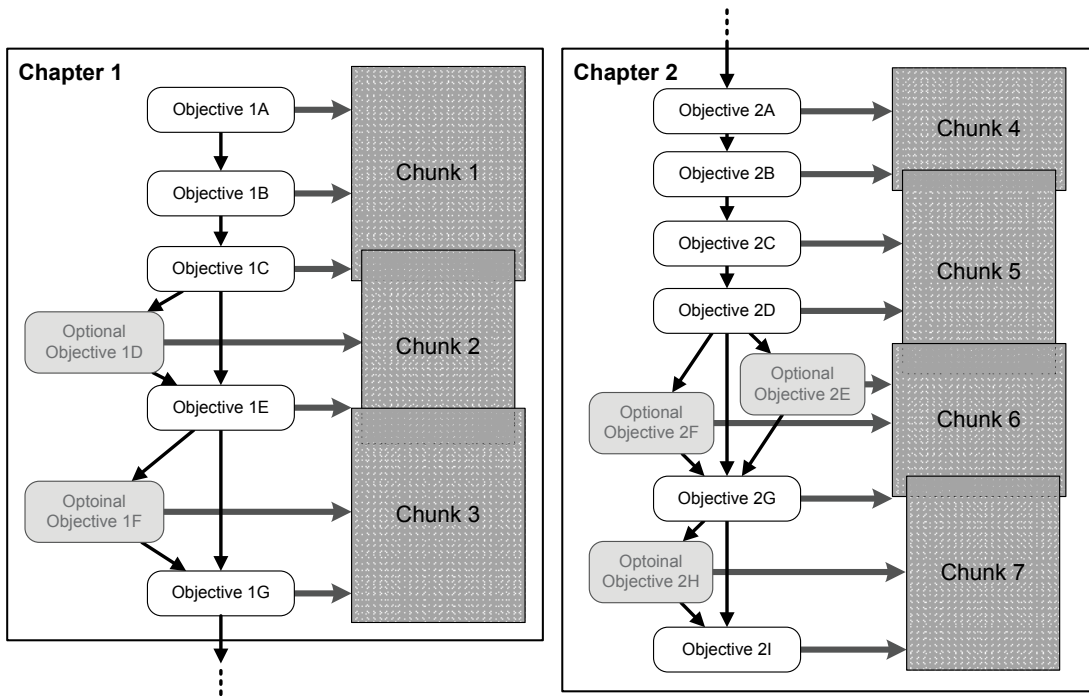


Figure I5.3. Gameplay objectives are typically arranged in a sequence, a tree, or a generalized graph, and each one maps to one or more game world chunks.

## 15.2 Implementing Dynamic Elements: Game Objects

The dynamic elements of a game are usually designed in an object-oriented fashion. This approach is intuitive and natural and maps well to the game designer's notion of how the world is constructed. He or she can visualize characters, vehicles, floating health packs, exploding barrels and myriad other dynamic objects moving about in the game. So it is only natural to want to be able to create and manipulate these elements in the game world editor. Likewise, programmers usually find it natural to implement dynamic elements as largely autonomous agents at runtime. In this book, we'll use the term *game object* (GO) to refer to virtually any dynamic element within a game world. However, this terminology is by no means standard within the industry. Game objects are commonly referred to as *entities*, *actors* or *agents*, and the list of terms goes on.

As is customary in object-oriented design, a game object is essentially a collection of *attributes* (the current state of the object) and *behaviors* (how the state changes over time and in response to events). Game objects are usually classified by *type*. Different types of objects have different attribute schemas and different behaviors. All *instances* of a particular type share the same attribute schema and the same set of behaviors, but the *values* of the attributes differ from instance to instance. (Note that if a game object's *behavior* is data-driven, say through script code or via a set of data-driven rules governing the object's responses to events, then behavior too can vary on an instance-by-instance basis.)

The distinction between a *type* and an *instance* of a type is a crucial one. For example, the game of *Pac-Man* involves four game object types: ghosts, pellets, power pills and Pac-Man. However, at any moment in time, there may be up to four instances of the type "ghost," 50–100 instances of the type "pellet," four "power pill" instances and one instance of the "Pac-Man" type.

Most object-oriented systems provide some mechanism for the *inheritance* of attributes, behavior or both. Inheritance encourages code and design reuse. The specifics of how inheritance works vary widely from game to game, but most game engines support it in some form.

### 15.2.1 Game Object Models

In computer science, the term *object model* has two related but distinct meanings. It can refer to the set of features provided by a particular programming language or formal design language. For example, we might speak of the C++ *object model* or the OMT *object model*. It can also refer to a specific object-oriented programming interface (i.e., a collection of classes, methods and interrelationships designed to solve a particular problem). One example of this latter usage is the *Microsoft Excel object model*, which allows external programs to control Excel in various ways. (See [http://en.wikipedia.org/wiki/Object\\_model](http://en.wikipedia.org/wiki/Object_model) for further discussion of the term *object model*.)

In this book, we will use the term *game object model* to describe the facilities provided by a game engine in order to permit the dynamic entities in the virtual game world to be modeled and simulated. In this sense, the term *game object model* has aspects of *both* of the definitions given above:

- A game's object model is a specific object-oriented programming interface intended to solve the particular problem of simulating the specific set of entities that make up a particular game.
- Additionally, a game's object model often extends the programming language in which the engine was written. If the game is implemented

in a non-object-oriented language like C, object-oriented facilities can be added by the programmers. And even if the game is written in an object-oriented language like C++, advanced features like reflection, persistence and network replication are often added. A game object model sometimes melds the features of multiple languages. For example, a game engine might combine a compiled programming language such as C or C++ with a scripting language like Python, Lua or Pawn and provide a unified object model that can be accessed from either language.

### 15.2.2 Tool-Side Design versus Runtime Design

The object model presented to the designers via the world editor (discussed in Section 15.4) needn't be the same object model used to implement the game at runtime.

- The tool-side game object model might be implemented at runtime using a language with no native object-oriented features at all, like C.
- A single GO type on the tool side might be implemented as a collection of classes at runtime (rather than as a single class as one might at first expect).
- Each tool-side GO might be nothing more than a unique id at runtime, with all of its state data stored in tables or collections of loosely coupled objects.

Therefore, a game really has two distinct but closely interrelated object models:

- The *tool-side object model* is defined by the set of *game object types* seen by the designers within the world editor.
- The *runtime object model* is defined by whatever set of language constructs and software systems the programmers have used to implement the tool-side object model at runtime. The runtime object model might be identical to the tool-side model or map directly to it, or it might be entirely different than the tool-side model under the hood.

In some game engines, the line between the tool-side and runtime designs is blurred or nonexistent. In others, it is very well delineated. In some engines, the implementation is actually shared between the tools and the runtime. In others, the runtime implementation looks almost totally alien relative to the tool-side view of things. Some aspects of the implementation almost always creep up into the tool-side design, and game designers must be cognizant of the performance- and memory-consumption impacts of the game worlds they

construct and the gameplay rules and object behaviors they design. That said, virtually all game engines have some form of tool-side object model and a corresponding runtime implementation of that object model.

### 15.3 Data-Driven Game Engines

In the early days of game development, games were largely hard-coded by programmers. Tools, if any, were primitive. This worked because the amount of content in a typical game was miniscule, and the bar wasn't particularly high, thanks in part to the primitive graphics and sound of which early game hardware was capable.

Today, games are orders of magnitude more complex, and the quality bar is so high that game content is often compared to the computer-generated effects in Hollywood blockbusters. Game teams have grown much larger, but the amount of game content is growing faster than team size. In the eighth generation of consoles, defined by the Xbox One and the PlayStation 4, game teams routinely speak of the need to produce ten times the content with teams that are not that much larger than in the previous generation. This trend means that a game team must be capable of producing very large amounts of content in an extremely efficient manner.

Engineering resources are often a production bottleneck because high-quality engineering talent is limited and expensive and because engineers tend to produce content much more slowly than artists and game designers (due to the complexities inherent in computer programming). Most teams now believe that it's a good idea to put at least some of the power to create content directly into the hands of the folks responsible for producing that content—namely the designers and the artists. When the behavior of a game can be controlled, in whole or in part, by *data* provided by artists and designers rather than exclusively by *software* produced by programmers, we say the engine is *data driven*.

Data-driven architectures can improve team efficiency by fully leveraging all staff members to their fullest potential and by taking some of the heat off the engineering team. It can also lead to improved *iteration times*. Whether a developer wants to make a slight tweak to the game's content or completely revise an entire level, a data-driven design allows the developer to see the effects of the changes quickly, ideally with little or no help from an engineer. This saves valuable time and can permit the team to polish their game to a very high level of quality.

That being said, it's important to realize that data-driven features often come at a heavy cost. Tools must be provided to allow game designers and



artists to define game content in a data-driven manner. The runtime code must be changed to handle the wide range of possible inputs in a robust way. Tools must also be provided in-game to allow artists and designers to preview their work and troubleshoot problems. All of this software requires significant time and effort to write, test and maintain.

Sadly, many teams make a mad rush into data-driven architectures without stopping to study the impacts of their efforts on their particular game design and the specific needs of their team members. In their haste, such teams often dramatically overshoot the mark, producing overly complex tools and engine systems that are difficult to use, bug-ridden and virtually impossible to adapt to the changing requirements of the project. Ironically, in their efforts to realize the benefits of a data-driven design, a team can easily end up with significantly lower productivity than the old-fashioned hard-coded methods.

Every game engine should have some data-driven components, but a game team must exercise extreme care when selecting which aspects of the engine to data-drive. It's crucial to weigh the costs of creating a data-driven or rapid-iteration feature against the amount of time the feature is expected to save the team over the course of the project. It's also incredibly important to keep the KISS mantra ("keep it simple, stupid") in mind when designing and implementing data-driven tools and engine systems. To paraphrase Albert Einstein, everything in a game engine should be made as simple as possible, but no simpler.

## 15.4 The Game World Editor

We've already discussed data-driven asset-creation tools, such as Maya, Photoshop, Havok content tools and so on. These tools generate individual assets for consumption by the rendering engine, animation system, audio system, physics system and so on. The analog to these tools in the gameplay space is the *game world editor*—a tool (or a suite of tools) that permits game world chunks to be defined and populated with static and dynamic elements.

All commercial game engines have some kind of world editor tool.

- A well-known tool called *Radiant* is used to create maps for the *Quake* and *Doom* family of engines. A screenshot of Radiant is shown in Figure 15.4.
- Valve's *Source* engine, the engine that drives *Half-Life 2*, *The Orange Box*, *Team Fortress 2*, the *Portal* series, the *Left 4 Dead* series and *Titanfall*, provides an editor called *Hammer* (previously distributed under the names *Worldcraft* and *The Forge*). Figure 15.5 shows a screenshot of Hammer.

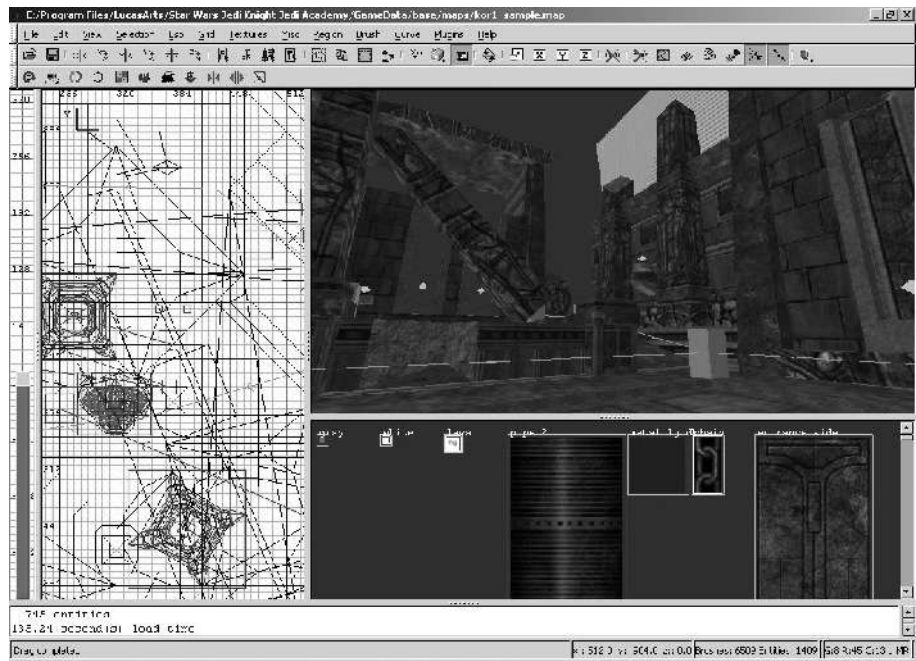
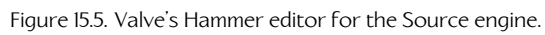


Figure 15.4. The Radiant world editor for the Quake and Doom family of engines.

- Crytek's CRYENGINE provides a powerful suite of world creation and editing tools. These tools support real-time editing of multiplatform game environments simultaneously, both in 2D and true stereoscopic 3D. Crytek's Sandbox editor is depicted in Figure 15.6.

The game world editor generally permits the initial states of game objects (i.e., the values of their attributes) to be specified. Most game world editors also give their users some sort of ability to control the *behaviors* of the dynamic objects in the game world. This control might be via data-driven configuration parameters (e.g., object A should start in an invisible state, object B should immediately attack the player when spawned, object C is flammable, etc.), or behavioral control might be via a scripting language, thereby shifting the game designers' tasks into the realm of programming. Some world editors even allow entirely new types of game objects to be defined, with little or no programmer intervention.



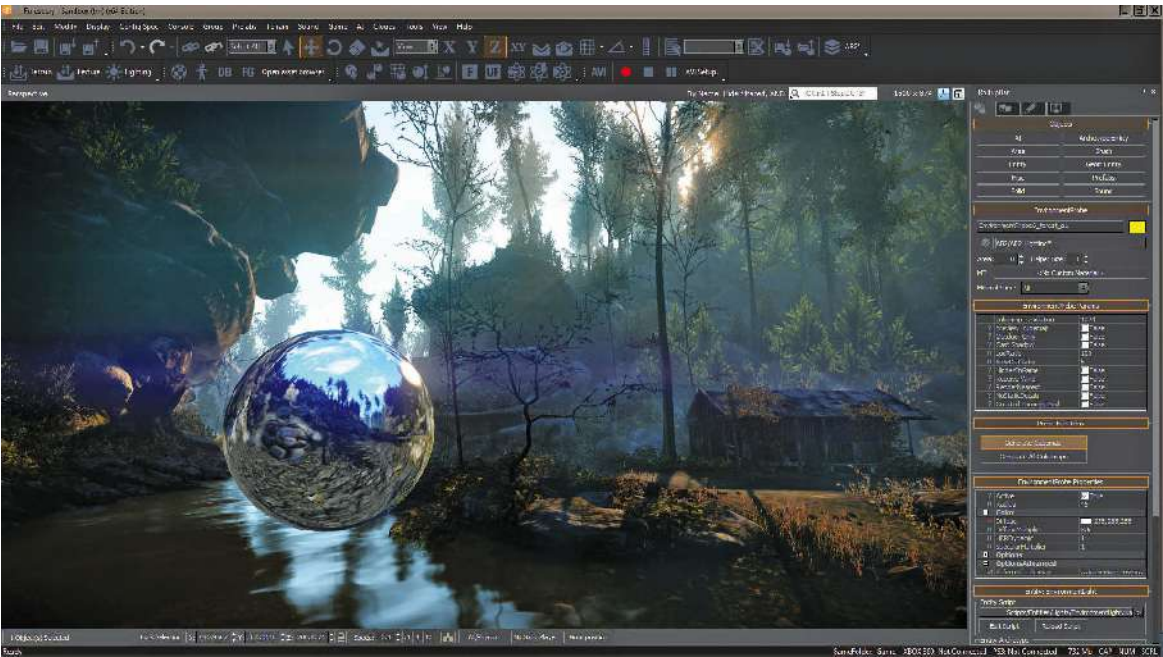


Figure 15.6. The Sandbox editor for CRYENGINE. (See Color Plate XXV.)

### 15.4.1 Typical Features of a Game World Editor

The design and layout of game world editors vary widely, but most editors provide a reasonably standard set of features. These include, but are certainly not limited to, the following.

#### 15.4.1.1 World Chunk Creation and Management

The unit of world creation is usually a chunk (also known as a level or map—see Section 15.1.2). The game world editor typically allows new chunks to be created and existing chunks to be renamed, broken up, combined or destroyed. Each chunk can be linked to one or more static meshes and/or other static data elements such as AI navigation maps, descriptions of ledges that can be grabbed by the player, cover point definitions and so on. In some engines, a chunk is defined by a single background mesh and cannot exist without one. In other engines, a chunk may have an independent existence, perhaps defined by a bounding volume (e.g., AABB, OBB or arbitrary polygonal region), and can be populated by zero or more meshes and/or brush geometry (see Section 1.7.2.1).

Some world editors provide dedicated tools for authoring terrain, water and other specialized static elements. In other engines, these elements might be authored using standard DCC applications but tagged in some way to indicate to the asset conditioning pipeline and/or the runtime engine that they are special. (For example, in the *Uncharted* and *The Last of Us* series, water was authored as a triangle mesh, but it was mapped with a special material that indicated that it was to be treated as water.) Sometimes, special world elements are created and edited in a separate, stand-alone tool. For example, the height field terrain in *Medal of Honor: Pacific Assault* was authored using a customized version of a tool obtained from another team within Electronic Arts because this was more expedient than trying to integrate a terrain editor into Radiant, the world editor being used on the project at the time.

#### 15.4.1.2 Game World Visualization

It's important for the user of a game world editor to be able to visualize the contents of the game world. As such, virtually all game world editors provide a three-dimensional perspective view of the world and/or a two-dimensional orthographic projection. It's common to see the view pane divided into four sections, three for top, side and front orthographic elevations and one for the 3D perspective view.

Some editors provide these world views via a custom rendering engine integrated directly into the tool. Other editors are themselves integrated into

a 3D geometry editor like Maya or 3ds Max, so they can simply leverage the tool's viewports. Still other editors are designed to communicate with the actual game engine and use it to render the 3D perspective view. Some editors are even integrated into the engine itself.

#### 15.4.1.3 Navigation

Clearly, a world editor wouldn't be of much use if the user weren't able to move around within the game world. In an orthographic view, it's important to be able to scroll and zoom in and out. In a 3D view, various camera control schemes are used. It may be possible to focus on an individual object and rotate around it. It may also be possible to switch into a "fly through" mode where the camera rotates about its own focal point and can be moved forward, backward, up and down and panned left and right.

Some editors provide a host of convenience features for navigation. These include the ability to select an object and focus in on it with a single key press, the ability to save various relevant camera locations and then jump between them, various camera movement speed modes for coarse navigation and fine camera control, a Web-browser-like navigation history that can be used to jump around the game world and so on.

#### 15.4.1.4 Selection

A game world editor is primarily designed to allow the user to populate a game world with static and dynamic elements. As such, it's important for the user to be able to select individual elements for editing. Some editors only allow a single object to be selected at a time, while more-advanced editors permit multiobject selections. Objects might be selected via a rubber-band box in the orthographic view or by ray cast style picking in the 3D view. Many editors also display a list of all world elements in a scrolling list or tree view so that objects can be found and selected by name. Some world editors also allow selections to be named and saved for later retrieval.

Game worlds are often quite densely populated. As such, it can sometimes be difficult to select a desired object because other objects are in the way. This problem can be overcome in a number of ways. When using a ray cast to select objects in 3D, the editor might allow the user to cycle through all of the objects that the ray is currently intersecting rather than always selecting the nearest one. Many editors allow the currently selected object(s) to be temporarily hidden from view. That way, if you don't get the object you want the first time, you can always hide it and try again. As we'll see in the next section, layers can also be an effective way to reduce clutter and improve the user's ability to select objects successfully.

#### 15.4.1.5 Layers

Some editors also allow objects to be grouped into predefined or user-defined *layers*. This can be an incredibly useful feature, allowing the contents of the game world to be organized sensibly. Entire layers can be hidden or shown to reduce clutter on-screen. Layers might be color-coded for easy identification. Layers can be an important part of a division-of-labor strategy, as well. For example, when the lighting team is working on a world chunk, they can hide all of the elements in the scene that are not relevant to lighting.

What's more, if the game world editor is capable of loading and saving layers individually, conflicts can be avoided when multiple people are working on a single world chunk at the same time. For example, all of the lights might be stored in one layer, all of the background geometry in another and all AI characters in a third. Since each layer is totally independent, the lighting, background and NPC teams can all work simultaneously on the same world chunk.

#### 15.4.1.6 Property Grid

The static and dynamic elements that populate a game world chunk typically have various properties (also known as attributes) that can be edited by the user. Properties might be simple key-value pairs and be limited to simple atomic data types like Booleans, integers, floating-point numbers and strings. In some editors, more complex properties are supported, including arrays of data and nested compound data structures. More complex data types may be supported too, such as vectors, RGB colors and references to external assets (audio files, meshes, animations, etc.)

Most world editors display the attributes of the currently selected object(s) in a scrollable property grid view. An example of a property grid is shown in Figure 15.7. The grid allows the user to see the current values of each attribute and edit the values by typing, using check boxes or drop-down combo boxes, dragging spinner controls up and down and so on.

#### *Editing Multiobject Selections*

In editors that support multiobject selection, the property grid may support multiobject editing as well. This advanced feature displays an amalgam of the attributes of all objects in the selection. If a particular attribute has the same value across all objects in the selection, the value is shown as-is, and editing the value in the grid causes the property value to be updated in all selected objects. If the attribute's value differs from object to object within the selection, the property grid typically shows no value at all. In this case, if a new value

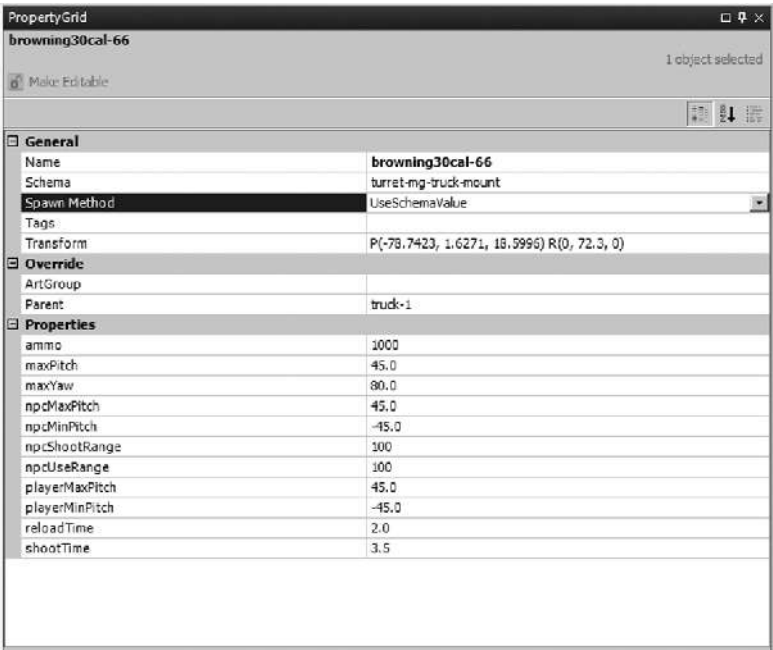


Figure 15.7. A typical property grid.

is typed into the field in the grid, it will overwrite the values in all selected objects, bringing them all into agreement.

Another problem arises when the selection contains a heterogeneous collection of objects (i.e., objects whose types differ). Each type of object can potentially have a different set of attributes, so the property grid must display only those attributes that are common to all object types in the selection. This can still be useful, however, because game object types often inherit from a common base type. For example, most objects have a position and orientation. In a heterogeneous selection, the user can still edit these shared attributes even though more-specific attributes are temporarily hidden from view.

*Free-Form Properties*

Normally, the set of properties associated with an object, and the data types of those properties, are defined on a per-object-type basis. For example, a renderable object has a position, orientation, scale and mesh, while a light has position, orientation, color, intensity and light type. Some editors also allow additional “free-form” properties to be defined by the user on a per-instance basis.



These properties are usually implemented as a flat list of key-value pairs. The user is free to choose the name (key) of each free-form property, along with its data type and its value. This can be incredibly useful for prototyping new gameplay features or implementing one-off scenarios.

#### 15.4.1.7 Object Placement and Alignment Aids

Some object properties are treated in a special way by the world editor. Typically the position, orientation and scale of an object can be controlled via special handles in the orthographic and perspective viewports, just like in Maya or Max. In addition, asset linkages often need to be handled in a special way. For example, if we change the mesh associated with an object in the world, the editor should display this mesh in the orthographic and 3D perspective viewports. As such, the game world editor must have special knowledge of these properties—it cannot treat them generically, as it can most object properties.

Many world editors provide a host of object placement and alignment aids in addition to the basic translation, rotation and scale tools. Many of these features borrow heavily from the feature sets of commercial graphics and 3D modeling tools like Photoshop, Maya, Visio and others. Examples include snap to grid, snap to terrain, align to object and many more.

#### 15.4.1.8 Special Object Types

Just as some object properties must be handled in a special way by the world editor, certain types of objects also require special handling. Examples include:

- *Lights.* The world editor usually uses special icons to represent lights, since they have no mesh. The editor may attempt to display the light's approximate effect on the geometry in the scene as well, so that designers can move lights around in real time and get a reasonably good feel for how the scene will ultimately look.
- *Particle emitters.* Visualization of particle effects can also be problematic in editors that are built on a stand-alone rendering engine. In this case, particle emitters might be displayed using icons only, or some attempt might be made to emulate the particle effect in the editor. Of course, this is not a problem if the editor is in-game or can communicate with the running game for live tweaking.
- *Sound sources.* As we discussed in Chapter 14, a 3D rendering engine models sound sources as 3D points or volumes. It may be convenient to provide specialized editing tools for these in the world editor. For example, sound designers will find it helpful if they can visualize the

maximum radius of an omnidirectional sound emitter, or the direction vector and cone of a directional emitter.

- *Regions.* A region is a volume of space that is used by the game to detect relevant events such as objects entering or leaving the volume or to demark areas for various purposes. Some game engines restrict regions to being modeled as spheres or oriented boxes, while others may permit arbitrary convex polygonal shapes when viewed from above, with strictly horizontal sides. Still others might allow regions to be constructed out of more complex geometry, such as  $k$ -DOPs (see Section 13.3.4.5). If regions are always spherical then the designers might be able to make do with a “Radius” property in the property grid, but to define or modify the extents of an arbitrarily shaped region, a special-case editing tool is almost certainly required.
- *Splines.* A spline is a three-dimensional curve defined by a set of control points and possibly tangent vectors at the points, depending on the type of mathematical curve used. Catmull-Rom splines are commonly used because they are fully defined by a set of control points (without tangents), and the curve always passes through all of the control points. But no matter what type of splines are supported, the world editor typically needs to provide the ability to display the splines in its viewports, and the user must be able to select and manipulate individual control points. Some world editors actually support two selection modes—a “coarse” mode for selecting objects in the scene and a “fine” mode for selecting the individual components of a selected object, such as the control points of a spline or the vertices of a region.
- *Nav meshes for AI.* In many games, NPCs navigate by running path-finding algorithms within the navigable regions of the game world. These navigable regions must be defined, and the world editor usually plays a central role in allowing AI designers to create, visualize and edit these regions. For example, a *nav mesh* is a 2D triangle mesh that provides a simple description of the boundaries of the navigable region, as well as providing connectivity information to the path finder.
- *Other custom data.* Of course, every game has its own specific data requirements. The world editor may be called upon to provide custom visualization and editing facilities for these pieces of data. Examples include a description of the “affordances” (windows, doorways, possible points of attack or defense) within a play space for use by the AI system, or geometric features that describe things like cover points or grabbable ledges for use by the player character and/or NPCs.

#### 15.4.1.9 Saving and Loading World Chunks

Of course, no world editor would be complete if it were unable to load and save world chunks. The granularity with which world chunks can be loaded and saved differs widely from engine to engine. Some engines store each world chunk in a single file, while others allow individual layers to be loaded and saved independently. Data formats also vary across engines. Some use custom binary formats, others text formats like XML or JSON. Each design has its pros and cons, but every editor provides the ability to load and save world chunks in some form—and every game engine is capable of loading world chunks so that they can be played at runtime.

#### 15.4.1.10 Rapid Iteration

A good game world editor usually supports some degree of dynamic tweaking for rapid iteration. Some editors run within the game itself, allowing the user to see the effects of his or her changes immediately. Others provide a live connection from the editor to the running game. Still other world editors operate entirely offline, either as a stand-alone tool or as a plug-in to a DCC application like Lightwave or Maya. These tools sometimes permit modified data to be reloaded dynamically into the running game. The specific mechanism isn't important—all that matters is that users have a reasonably short *round-trip iteration time* (i.e., the time between making a change to the game world and seeing the effects of that change in-game). It's important to realize that iterations don't have to be instantaneous. Iteration times should be commensurate with the scope and frequency of the changes being made. For example, we might expect tweaking a character's maximum health to be a very fast operation, but when making major changes to the lighting environment for an entire world chunk, a much longer iteration time might be acceptable.

### 15.4.2 Integrated Asset Management Tools

In some engines, the game world editor is integrated with other aspects of game asset database management, such as defining mesh and material properties, defining animations, blend trees, animation state machines, setting up collision and physical properties of objects, managing texture resources and so on. (See Section 7.2.1.2 for a discussion of the game asset database.)

Perhaps the best-known example of this design in action is *UnrealEd*, the editor used to create content for games built on the Unreal Engine. UnrealEd is integrated directly into the game engine, so any changes made in the editor are made directly to the dynamic elements in the running game. This makes rapid iteration very easy to achieve. But UnrealEd is much more than a game

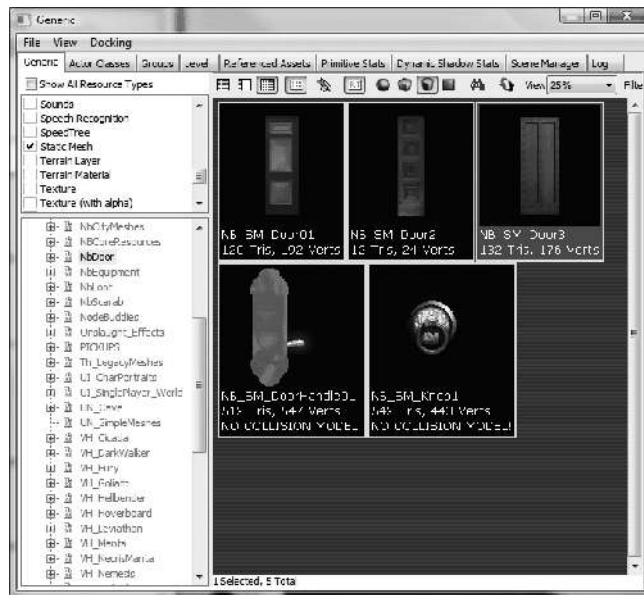


Figure 15.8. UnrealEd's Generic Browser provides access to the entire game asset database.

world editor—it is actually a complete content-creation package. It manages the entire database of game assets, from animations to audio clips to triangle meshes to textures to materials and shaders and much more. UnrealEd provides its user with a unified, real-time, WYSIWYG view into the entire asset database, making it a powerful enabler of any rapid, efficient game development process. A few screenshots from UnrealEd are shown in Figures 15.8 and 15.9.

#### 15.4.2.1 Data Processing Costs

In Section 7.2.1, we learned that the asset conditioning pipeline (ACP) converts game assets from their various source formats into the formats required by the game engine. This is typically a two-step process. First, the asset is exported from the DCC application to a platform-independent intermediate format that only contains the data that is relevant to the game. Second, the asset is processed into a format that is optimized for a specific platform. On a project targeting multiple gaming platforms, a single platform-independent asset gives rise to multiple platform-specific assets during this second phase.

One of the key differences between tools pipelines is the point at which this second platform-specific optimization step is performed. UnrealEd performs it when assets are first imported into the editor. This approach pays off in

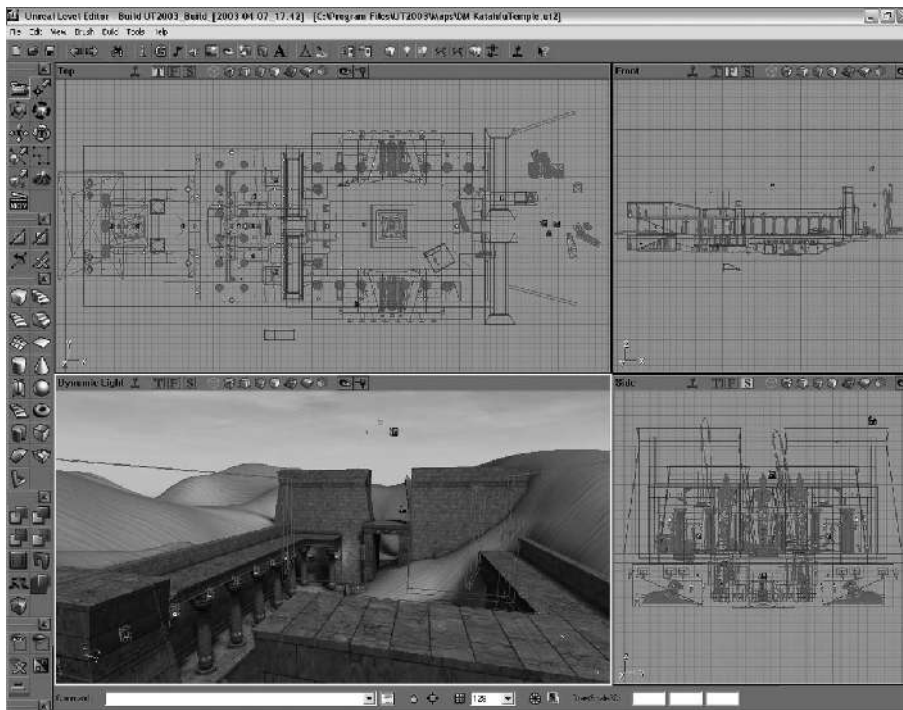


Figure 15.9. UnrealEd also provides a world editor.

rapid iteration time when iterating on level design. However, it can make the cost of changing source assets like meshes, animations, audio assets and so on more painful. Other engines like the Source engine and the *Quake* engine pay the asset optimization cost when baking out the level prior to running the game. *Halo* gives the user the option to change raw assets at any time; they are converted into an optimized form when they are first loaded into the engine, and the results are cached to prevent the optimization step from being performed needlessly every time the game is run.



# Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

# 16

## Runtime Gameplay Foundation Systems

### 16.1 Components of the Gameplay Foundation System

Most game engines provide a suite of runtime software components that together provide a framework upon which a game's unique rules, objectives and dynamic world elements can be constructed. There is no standard name for these components within the game industry, but we will refer to them collectively as the engine's *gameplay foundation system*. If a line between engine and game can reasonably be drawn between the game engine and the game itself, then these systems lie just beneath this line. In theory, one can construct gameplay foundation systems that are for the most part game-agnostic. However, in practice, these systems almost always contain genre- or game-specific details. In fact, the line between the engine and the game can probably be best visualized as one big blur—a gradient that arcs across these components as it links the engine to the game. In some game engines, one might even go so far as to consider the gameplay foundation systems as lying entirely *above* the engine-game line. The differences between game engines are most acute when it comes to the design and implementation of their gameplay components. That said, there are a surprising number of common patterns across engines, and those commonalities will be the topic of our discussions here.

Every game engine approaches the problem of gameplay software design a bit differently. However, most engines provide the following major subsystems in some form:

- *Runtime game object model.* This is an implementation of the abstract game object model advertised to the game designers via the world editor.
- *Level management and streaming.* This system loads and unloads the contents of the virtual worlds in which gameplay takes place. In many engines, level data is streamed into memory during gameplay, thus providing the illusion of a large seamless world (when in fact it is broken into discrete chunks).
- *Real-time object model updating.* In order to permit the game objects in the world to behave autonomously, each object must be updated periodically. This is where all of the disparate systems in a game engine truly come together into a cohesive whole.
- *Messaging and event handling.* Most game objects need to communicate with one another. This is usually done via an abstract messaging system. Inter-object messages often signal changes in the state of the game world called *events*. So the messaging system is referred to as the *event system* in many studios.
- *Scripting.* Programming high-level game logic in a language like C or C++ can be cumbersome. To improve productivity, allow rapid iteration, and put more power into the hands of the non-programmers on the team, a scripting language is often integrated into the game engine. This language might be text-based, like Python or Lua, or it might be a graphical language, like Unreal's Blueprints.
- *Objectives and game flow management.* This subsystem manages the player's objectives and the overall flow of the game. This is usually described by a sequence, a tree, or a generalized graph of player objectives. Objectives are often grouped into chapters, especially if the game is highly story-driven as many modern games are. The game flow management system manages the overall flow of the game, tracks the player's accomplishment of objectives and gates the player from one area of the game world to the next as the objectives are accomplished. Some designers refer to this as the "spine" of the game.

Of these major systems, the runtime object model is probably the most complex. It typically provides most, if not all, of the following features:

- *Spawning and destroying game objects dynamically.* The dynamic elements in a game world often need to come and go during gameplay. Health



packs disappear once they have been picked up, explosions appear and then dissipate and enemy reinforcements mysteriously come from around a corner just when you think you've cleared the level. Many game engines provide a system for managing the memory and other resources associated with dynamically spawned game objects. Other engines simply disallow dynamic creation or destruction of game objects altogether.

- *Linkage to low-level engine systems.* Every game object has some kind of linkage to one or more underlying engine systems. Most game objects are visually represented by renderable triangle meshes. Some have particle effects. Many generate sounds. Some animate. Many have collision, and some are dynamically simulated by the physics engine. One of the primary responsibilities of the gameplay foundation system is to ensure that every game object has access to the services of the engine systems upon which it depends.
- *Real-time simulation of object behaviors.* At its core, a game engine is a real-time dynamic computer simulation of an agent-based model. This is just a fancy way of saying that the game engine needs to update the states of all the game objects dynamically over time. The objects may need to be updated in a very particular order, dictated in part by dependencies between the objects, in part by their dependencies on various engine subsystems, and in part because of the interdependencies between those engine subsystems themselves.
- *Ability to define new game object types.* Every game's requirements change and evolve as the game is developed. It's important that the game object model be flexible enough to permit new object types to be added easily and exposed to the world editor. In an ideal world, it should be possible to define a new type of object in an entirely data-driven manner. However, in many engines, the services of a programmer are required in order to add new game object types.
- *Unique object ids.* Typical game worlds contain hundreds or even thousands of individual game objects of various types. At runtime, it's important to be able to identify or search for a particular object. This means each object needs some kind of unique identifier. A human-readable name is the most convenient kind of id, but we must be wary of the performance costs of using strings at runtime. Integer ids are the most efficient choice, but they are very difficult for human game developers to work with. Arguably the best solution is to use hashed string ids (see Section 6.4.3.1) as our object identifiers, as they are as efficient as integers

but can be converted back into string form for ease of reading.

- *Game object queries.* The gameplay foundation system must provide some means of finding objects within the game world. We might want to find a specific object by its unique id, or all the objects of a particular type, or we might want to perform advanced queries based on arbitrary criteria (e.g., find all enemies within a 20 m radius of the player character).
- *Game object references.* Once we've found the objects, we need some mechanism for holding *references* to them, either briefly within a single function or for much longer periods of time. An object reference might be as simple as a pointer to a C++ class instance, or it might be something more sophisticated, like a handle or a reference-counted smart pointer.
- *Finite state machine support.* Many types of game objects are best modeled as finite state machines (FSM). Some game engines provide the ability for a game object to exist in one of many possible states, each with its own attributes and behavioral characteristics.
- *Network replication.* In a networked multiplayer game, multiple game machines are connected together via a LAN or the Internet. The state of a particular game object is usually owned and managed by one machine. However, that object's state must also be *replicated* (communicated) to the other machines involved in the multiplayer game so that all players have a consistent view of the object.
- *Saving and loading / object persistence.* Many game engines allow the current states of the game objects in the world to be saved to disk and later reloaded. This might be done to support a "save anywhere" save-game system or as a way of implementing network replication, or it might simply be the primary means of loading game world chunks that were authored in the world editor tool. Object persistence usually requires certain language features, such as *runtime type identification* (RTTI), *reflection* and *abstract construction*. RTTI and reflection provide software with a means of determining an object's *type*, and what *attributes* and *methods* its class provides, dynamically at runtime. Abstract construction allows instances of a class to be created without having to hard-code the name of the class—a very useful feature when serializing an object instance into memory from disk. If RTTI, reflection and abstract construction are not natively supported in your language of choice, these features can be added manually.

We'll spend the remainder of this chapter delving into each of these sub-systems in depth.

## 16.2 Runtime Object Model Architectures

In the world editor, the game designer is presented with an abstract game object model, which defines the various types of dynamic elements that can exist in the game, how they behave and what kinds of attributes they have. At runtime, the gameplay foundation system must provide a concrete implementation of this object model. This is by far the largest component of any gameplay foundation system.

The runtime object model implementation may or may not bear any resemblance to the abstract tool-side object model. For example, it might not be implemented in an object-oriented programming language at all, or it might use a collection of interconnected class instances to represent a single abstract game object. Whatever its design, the runtime object model must provide a faithful reproduction of the object types, attributes and behaviors advertised by the world editor.

The runtime object model is the in-game manifestation of the abstract tool-side object model presented to the designers in the world editor. Designs vary widely, but most game engines follow one of two basic architectural styles:

- *Object-centric.* In this style, each tool-side game object is represented at runtime by a single class instance or a small collection of interconnected instances. Each object has a set of *attributes* and *behaviors* that are encapsulated within the class (or classes) of which the object is an instance. The game world is just a collection of game objects.
- *Property-centric.* In this style, each tool-side game object is represented only by a unique id (implemented as an integer, hashed string id or string). The *properties* of each game object are distributed across many data tables, one per property type, and keyed by object id (rather than being centralized within a single class instance or collection of interconnected instances). The properties themselves are often implemented as instances of hard-coded classes. The *behavior* of a game object is implicitly defined by the collection of properties of which it is composed. For example, if an object has the “Health” property, then it can be damaged, lose health and eventually die. If an object has the “MeshInstance” property, then it can be rendered in 3D as an instance of a triangle mesh.

There are distinct advantages and disadvantages to each of these architectural styles. We’ll investigate each one in some detail and note where one style has significant potential benefits over the other as they arise.

### 16.2.1 Object-Centric Architectures

In an *object-centric* game world object architecture, each logical game object is implemented as an instance of a class, or possibly a collection of interconnected class instances. Under this broad umbrella, many different designs are possible. We'll investigate a few of the most common designs in the following sections.

#### 16.2.1.1 A Simple Object-Based Model in C: *Hydro Thunder*

Game object models needn't be implemented in an object-oriented language like C++ at all. For example, the arcade hit *Hydro Thunder*, by Midway Home Entertainment in San Diego, was written entirely in C. *Hydro* employed a very simple game object model consisting of only a few object types:

- boats (player- and AI-controlled),
- floating blue and red boost icons,
- ambient animated objects (animals on the side of the track, etc.),
- the water surface,
- ramps,
- waterfalls,
- particle effects,
- race track sectors (two-dimensional polygonal regions connected to one another that together define the watery region in which boats could race),
- static geometry (terrain, foliage, buildings along the sides of the track, etc.), and
- two-dimensional heads-up display (HUD) elements.

A few screenshots of *Hydro Thunder* are shown in Figure 16.1. Notice the hovering boost icons in both screenshots and the shark swimming by in the left image (an example of an ambient animated object).

*Hydro* had a C struct named `World_t` that stored and managed the contents of a game world (i.e., a single race track). The world contained pointers to arrays of various kinds of game objects. The static geometry was a single mesh instance. The water surface, waterfalls and particle effects were each represented by custom data structures. The boats, boost icons and other dynamic objects in the game were represented by instances of a general-purpose struct called `WorldOb_t` (i.e., a world object). This was *Hydro's* equivalent of a *game object* as we've defined it in this chapter.



Figure 16.1. Screenshots from the arcade game *Hydro Thunder*, developed by Midway Home Entertainment in San Diego.

The `WorldOb_t` data structure contained data members encoding the position and orientation of the object, the 3D mesh used to render it, a set of collision spheres, simple animation state information (*Hydro* only supported rigid hierarchical animation), physical properties like velocity, mass and buoyancy, and other data common to all of the dynamic objects in the game. In addition, each `WorldOb_t` contained three pointers: a `void*` “user data” pointer, a pointer to a custom “update” function and a pointer to a custom “draw” function. So while *Hydro Thunder* was not object-oriented in the strictest sense, the *Hydro* engine did extend its non-object-oriented language (C) to support rudimentary implementations of two important OOP features: *inheritance* and *polymorphism*. The user data pointer permitted each type of game object to maintain custom state information specific to its type while inheriting the features common to all world objects. For example, the *Banshee* boat had a different booster mechanism than the *Rad Hazard*, and each booster mechanism required different state information to manage its deployment and stowing animations. The two function pointers acted like *virtual functions*, allowing world objects to have polymorphic behaviors (via their “update” functions) and polymorphic visual appearances (via their “draw” functions).

```
struct WorldOb_s
{
    Orient_t m_transform;    /* position/rotation */
    Mesh3d*  m_pMesh;       /* 3D mesh */
    /* ... */
    void*    m_pUserData;   /* custom state */

    void      (*m_pUpdate)(); /* polymorphic update */
    void      (*m_pDraw)();  /* polymorphic draw */
};
typedef struct WorldOb_s WorldOb_t;
```

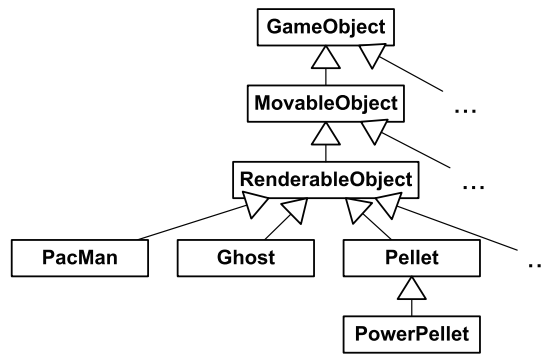


Figure 16.2. A hypothetical class hierarchy for the game *Pac-Man*.

### 16.2.1.2 Monolithic Class Hierarchies

It's natural to want to classify game object types taxonomically. This tends to lead game programmers toward an object-oriented language that supports inheritance. A class hierarchy is the most intuitive and straightforward way to represent a collection of interrelated game object types. So it is not surprising that the majority of commercial game engines employ a class hierarchy based technique.

Figure 16.2 shows a simple class hierarchy that could be used to implement the game *Pac-Man*. This hierarchy is rooted (as many are) at a common class called `GameObject`, which might provide some facilities needed by all object types, such as RTTI or serialization. The `MovableObject` class represents any object that has a position and orientation. `RenderableObject` gives the object an ability to be rendered (in the case of traditional *Pac-Man*, via a sprite, or in the case of a modern 3D *Pac-Man* game, perhaps via a triangle mesh). From `RenderableObject` are derived classes for the ghosts, Pac-Man, pellets and power pills that make up the game. This is just a hypothetical example, but it illustrates the basic ideas that underlie most game object class hierarchies—namely that common, generic functionality tends to exist at the root of the hierarchy, while classes toward the leaves of the hierarchy tend to add increasingly specific functionality.

A game object class hierarchy usually begins small and simple, and in that form, it can be a powerful and intuitive way to describe a collection of game object types. However, as class hierarchies grow, they have a tendency to deepen and widen simultaneously, leading to what I call a *monolithic class hierarchy*. This kind of hierarchy arises when virtually all classes in the game object model inherit from a single, common base class. The Unreal Engine's game object model is a classic example, as Figure 16.3 illustrates.

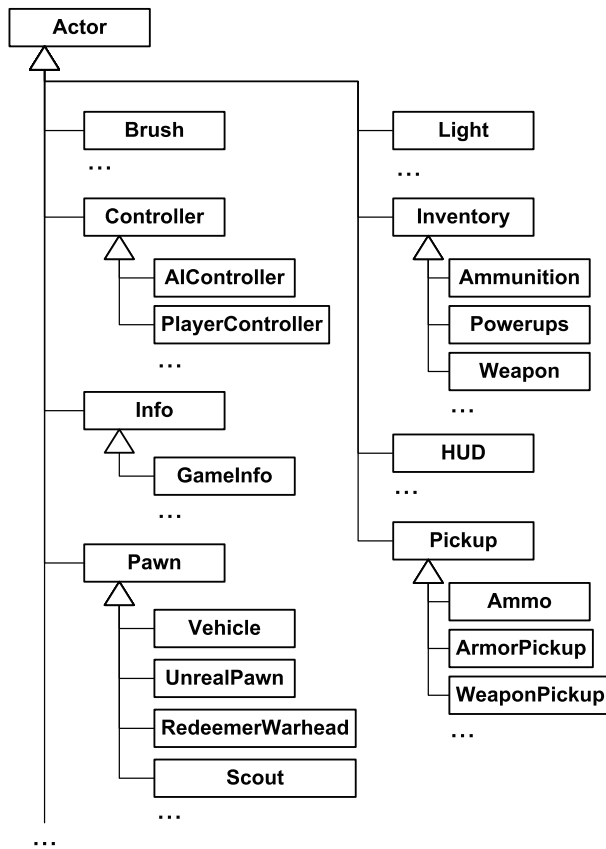


Figure 16.3. An excerpt from the game object class hierarchy in the *Unreal* engine.

### 16.2.1.3 Problems with Deep, Wide Hierarchies

Monolithic class hierarchies tend to cause problems for the game development team for a wide range of reasons. The deeper and wider a class hierarchy grows, the more extreme these problems can become. In the following sections, we'll explore some of the most common problems caused by wide, deep class hierarchies.

#### *Understanding, Maintaining and Modifying Classes*

The deeper a class lies within a class hierarchy, the harder it is to understand, maintain and modify. This is because to understand a class, you really need to understand all of its parent classes as well. For example, modifying the

behavior of an innocuous-looking virtual function in a derived class could violate the assumptions made by any one of the many base classes, leading to subtle, difficult-to-find bugs.

### *Inability to Describe Multidimensional Taxonomies*

A hierarchy inherently classifies objects according to a particular system of criteria known as a *taxonomy*. For example, *biological taxonomy* (also known as *alpha taxonomy*) classifies all living things according to genetic similarities, using a tree with eight levels: domain, kingdom, phylum, class, order, family, genus and species. At each level of the tree, a different criterion is used to divide the myriad life forms on our planet into more and more refined groups.

One of the biggest problems with any hierarchy is that it can only classify objects along a single “axis”—according to one particular set of criteria—at each level of the tree. Once the criteria have been chosen for a particular hierarchy, it becomes difficult or impossible to classify along an entirely different set of “axes.” For example, biological taxonomy classifies objects according to genetic traits, but it says nothing about the colors of the organisms. In order to classify organisms by color, we’d need an entirely different tree structure.

In object-oriented programming, this limitation of hierarchical classification often manifests itself in the form of wide, deep and confusing class hierarchies. When one analyzes a real game’s class hierarchy, one often finds that its structure attempts to meld a number of different classification criteria into a single class tree. In other cases, concessions are made in the class hierarchy to accommodate a new type of object whose characteristics were not anticipated when the hierarchy was first designed. For example, imagine the seemingly logical class hierarchy describing different types of vehicles, depicted in Figure 16.4.

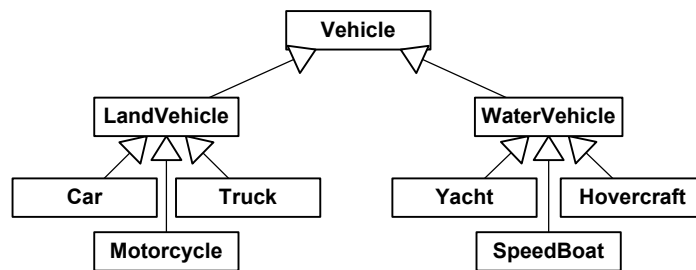


Figure 16.4. A seemingly logical class hierarchy describing various kinds of vehicles.



What happens when the game designers announce to the programmers that they now want the game to include an *amphibious* vehicle? Such a vehicle does not fit into the existing taxonomic system. This may cause the programmers to panic or, more likely, to “hack” their class hierarchy in various ugly and error-prone ways.

### *Multiple Inheritance: The Deadly Diamond*

One solution to the amphibious vehicle problem is to utilize C++’s multiple inheritance (MI) features, as shown in Figure 16.5. At first glance, this seems like a good solution. However, multiple inheritance in C++ poses a number of practical problems. For example, multiple inheritance can lead to an object that contains multiple copies of its base class’ members—a condition known as the “deadly diamond” or “diamond of death.” (See Section 3.1.1.3 for more details.)

The difficulties in building an MI class hierarchy that works and that is understandable and maintainable usually outweigh the benefits. As a result, most game studios prohibit or severely limit the use of multiple inheritance in their class hierarchies.

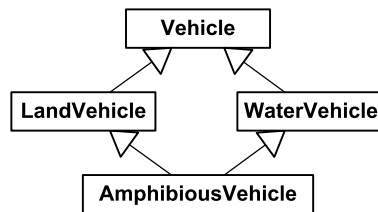


Figure 16.5. A diamond-shaped class hierarchy for amphibious vehicles.

### *Mix-In Classes*

Some teams do permit a limited form of MI, in which a class may have any number of parent classes but only *one* grandparent. In other words, a class may inherit from one and only one class in the main inheritance hierarchy, but it may also inherit from any number of *mix-in classes* (stand-alone classes with no base class). This permits common functionality to be factored out into a mix-in class and then spot-patched into the main hierarchy wherever it is needed. This is shown in Figure 16.6. However, as we’ll see below, it’s usually better to *compose* or *aggregate* such classes than to *inherit* from them.

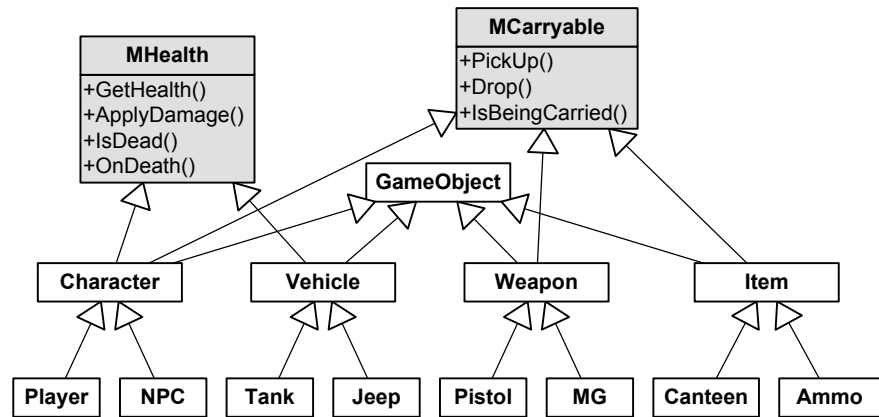


Figure 16.6. A class hierarchy with mix-in classes. The MHealth mix-in class adds the notion of health and the ability to be killed to any class that inherits it. The MCarryable mix-in class allows an object that inherits it to be carried by a Character.

### *The Bubble-Up Effect*

When a monolithic class hierarchy is first designed, the root class or classes are usually very simple, each one exposing only a minimal feature set. However, as more and more functionality is added to the game, the desire to *share code* between two or more *unrelated* classes begins to cause features to “bubble up” the hierarchy.

For example, we might start out with a design in which only wooden crates can float in water. However, once our game designers see those cool floating crates, they begin to ask for other kinds of floating objects, like characters, bits of paper, vehicles and so on. Because “floating versus non-floating” was not one of the original classification criteria when the hierarchy was designed, the programmers quickly discover the need to add flotation to classes that are totally unrelated within the class hierarchy. Multiple inheritance is frowned upon, so the programmers decide to move the flotation code up the hierarchy, into a base class that is common to all objects that need to float. The fact that some of the classes that derive from this common base class *cannot float* is seen as less of a problem than duplicating the flotation code across multiple classes. (A Boolean member variable called something like `m_bCanFloat` might even be added to make the distinction clear.) The ultimate result is that flotation eventually becomes a feature of the root object in the class hierarchy (along with pretty much every other feature in the game).

The Actor class in Unreal is a classic example of this “bubble-up effect.” It contains data members and code for managing rendering, animation, physics, world interaction, audio effects, network replication for multiplayer games,

object creation and destruction, actor iteration (i.e., the ability to iterate over all actors meeting a certain criteria and perform some operation on them), and message broadcasting. Encapsulating the functionality of various engine sub-systems is difficult when features are permitted to “bubble up” to the root-most classes in a monolithic class hierarchy.

#### 16.2.1.4 Using Composition to Simplify the Hierarchy

Perhaps the most prevalent cause of monolithic class hierarchies is over-use of the “is-a” relationship in object-oriented design. For example, in a game’s GUI, a programmer might decide to derive the class `Window` from a class called `Rectangle`, using the logic that GUI windows are always rectangular. However, a window *is not a* rectangle—it *has a* rectangle, which defines its boundary. So a more workable solution to this particular design problem is to embed an instance of the `Rectangle` class inside the `Window` class, or to give the `Window` a pointer or reference to a `Rectangle`.

In object-oriented design, the “has-a” relationship is known as *composition*. In composition, a class A either contains an *instance* of class B directly, or contains a *pointer* or *reference* to an instance of B. Strictly speaking, in order for the term “composition” to be applicable, class A must *own* class B. This means that when an instance of class A is created, it automatically creates an instance of class B as well; when that instance of A is destroyed, its instance of B is destroyed, too. We can also link classes to one another via a pointer or reference *without* having one of the classes manage the other’s lifetime. In that case, the technique is usually called *aggregation*.

#### *Converting Is-A to Has-A*

Converting “is-a” relationships into “has-a” relationships can be a useful technique for reducing the width, depth and complexity of a game’s class hierarchy. To illustrate, let’s take a look at the hypothetical monolithic hierarchy shown in Figure 16.7. The root `GameObject` class provides some basic functionality required by all game objects (e.g., RTTI, reflection, persistence via serialization, network replication, etc.). The `MovableObject` class represents any game object that has a *transform* (i.e., a position, orientation and optional scale). `RenderableObject` adds the ability to be rendered on-screen. (Not all game objects need to be rendered—for example, an invisible `TriggerRegion` class could be derived directly from `MovableObject`.) The `CollidableObject` class provides collision information to its instances. The `AnimatingObject` class grants to its instances the ability to be animated via a skeletal joint hierarchy. Finally, the `PhysicalObject` gives its instances the

ability to be physically simulated (e.g., a rigid body falling under the influence of gravity and bouncing around in the game world).

One big problem with this class hierarchy is that it limits our design choices when creating new types of game objects. If we want to define an object type that is physically simulated, we are forced to derive its class from `PhysicalObject` even though it may not require skeletal animation. If we want a game object class with collision, it must inherit from `CollidableObject` even though it may be invisible and hence not require the services of `RenderableObject`.

A second problem with the hierarchy shown in Figure 16.7 is that it is difficult to extend the functionality of the existing classes. For example, let's imagine we want to support morph target animation, so we derive two new classes from `AnimatingObject` called `SkeletalObject` and `MorphTargetObject`. If we wanted both of these new classes to have the ability to be physically simulated, we'd be forced to refactor `PhysicalObject` into two nearly identical classes, one derived from `SkeletalObject` and one from `MorphTargetObject`, or turn to multiple inheritance.

One solution to these problems is to isolate the various features of a `GameObject` into independent classes, each of which provides a single, well-defined service. Such classes are sometimes called *components* or *service objects*. A componentized design allows us to select only those features we need for each type of game object we create. In addition, it permits each feature to be maintained, extended or refactored without affecting the others. The individual components are also easier to understand, and easier to test, because they are decoupled from one another. Some component classes correspond directly to a single engine subsystem, such as rendering, animation, collision, physics, audio, etc. This allows these subsystems to remain distinct and well-encapsulated when they are integrated together for use by a particular game object.

Figure 16.8 shows how our class hierarchy might look after refactoring it into components. In this revised design, the `GameObject` class acts like a hub, containing pointers to each of the optional components we've defined. The `MeshInstance` component is our replacement for the `RenderableObject` class—it represents an instance of a triangle mesh and encapsulates the knowledge of how to render it. Likewise, the `AnimationController` component replaces `AnimatingObject`, exposing skeletal animation services to the `GameObject`. Class `Transform` replaces `MovableObject` by maintaining the position, orientation and scale of the object. The `RigidBody` class represents the collision geometry of a game object and provides its `GameObject` with an interface into the low-level collision and physics systems, replacing

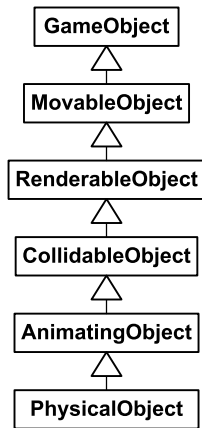


Figure 16.7. A hypothetical game object class hierarchy using only inheritance to associate the classes.

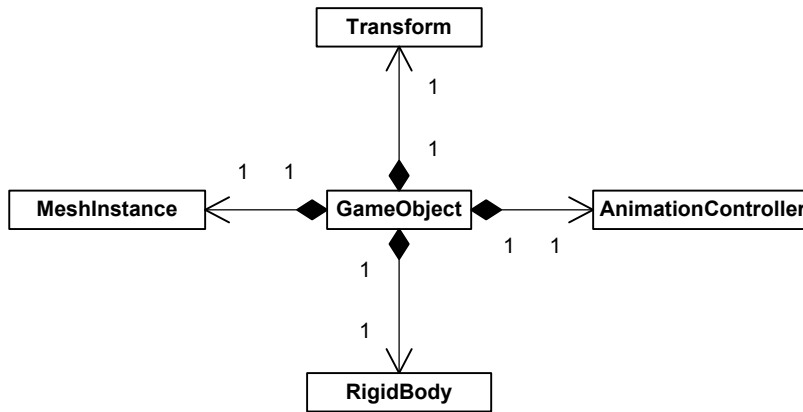


Figure 16.8. Our hypothetical game object class hierarchy, refactored to favor class composition over inheritance.

CollidableObject and PhysicalObject.

### Component Creation and Ownership

In this kind of design, it is typical for the “hub” class to *own* its components, meaning that it manages their *lifetimes*. But how should a `GameObject` “know” which components to create? There are numerous ways to solve this problem, but one of the simplest is to provide the root `GameObject` class with pointers to all possible components. Each unique type of game object is defined as a derived class of `GameObject`. In the `GameObject` constructor, all of the component pointers are initially set to `nullptr`. Each derived class’s constructor is then free to create whatever components it may need. For convenience, the default `GameObject` destructor can clean up all of the components automatically. In this design, the hierarchy of classes derived from `GameObject` serves as the primary taxonomy for the kinds of objects we want in our game, and the component classes serve as optional add-on features.

One possible implementation of the component creation and destruction logic for this kind of hierarchy is shown below. However, it’s important to realize that this code is just an example—implementation details vary widely, even between engines that employ essentially the same kind of class hierarchy design.

```

class GameObject
{
protected:

```

```

        // My transform (position, rotation, scale).
        Transform          m_transform;

        // Standard components:
        MeshInstance*      m_pMeshInst;
        AnimationController* m_pAnimController;
        RigidBody*          m_pRigidBody;

public:
    GameObject()
    {
        // Assume no components by default.
        // Derived classes will override.
        m_pMeshInst = nullptr;
        m_pAnimController = nullptr;
        m_pRigidBody = nullptr;
    }

    ~GameObject()
    {
        // Automatically delete any components created by
        // derived classes. (Deleting null pointers OK.)
        delete m_pMeshInst;
        delete m_pAnimController;
        delete m_pRigidBody;
    }

    // ...
};

class Vehicle : public GameObject
{
protected:
    // Add some more components specific to Vehicles...
    Chassis* m_pChassis;
    Engine*  m_pEngine;
    // ...

public:
    Vehicle()
    {
        // Construct standard GameObject components.
        m_pMeshInst = new MeshInstance;
        m_pRigidBody = new RigidBody;

        // NOTE: We'll assume the animation controller
        // must be provided with a reference to the mesh

```

```

        // instance so that it can provide it with a
        // matrix palette.
        m_pAnimController
            = new AnimationController(*m_pMeshInst);

        // Construct vehicle-specific components.
        m_pChassis = new Chassis(*this,
                                *m_pAnimController);

        m_pEngine = new Engine(*this);
    }

    ~Vehicle()
    {
        // Only need to destroy vehicle-specific
        // components, as GameObject cleans up the
        // standard components for us.
        delete m_pChassis;
        delete m_pEngine;
    }
};

```

#### 16.2.1.5 Generic Components

Another more flexible (but also trickier to implement) alternative is to provide the root game object class with a generic linked list of components. The components in such a design usually all derive from a common base class—this allows us to iterate over the linked list and perform polymorphic operations, such as asking each component what type it is or passing an event to each component in turn for possible handling. This design allows the root game object class to be largely oblivious to the component types that are available and thereby permits new types of components to be created without modifying the game object class in many cases. It also allows a particular game object to contain an arbitrary number of instances of each type of component. (The hard-coded design permits only a fixed number, determined by how many pointers to each component exist within the game object class.)

This kind of design is illustrated in Figure 16.9. It is trickier to implement than a hard-coded component model because the game object code must be written in a totally generic way. The component classes can likewise make no assumptions about what other components might or might not exist within the context of a particular game object. The choice between hard-coding the component pointers or using a generic linked list of components is not an easy one to make. Neither design is clearly superior—they each have their pros and cons, and different game teams take different approaches.

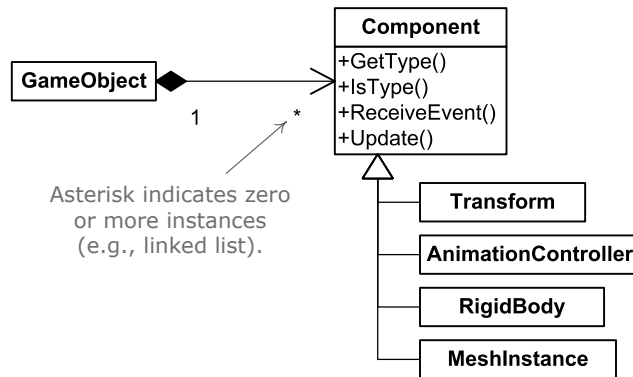


Figure 16.9. A linked list of components can provide flexibility by allowing the hub game object to be unaware of the details of any particular component.

#### 16.2.1.6 Pure Component Models

What would happen if we were to take the componentization concept to its extreme? We would move literally *all* of the functionality out of our root `GameObject` class into various component classes. At this point, the game object class would quite literally be a behavior-less container, with a unique id and a bunch of pointers to its components, but otherwise containing no logic of its own. So why not eliminate the class entirely? One way to do this is to give each component a copy of the game object's unique id. The components are now linked together into a logical grouping by id. Given a way to quickly look up any component by id, we would no longer need the `GameObject` "hub" class at all. I will use the term *pure component model* to describe this kind of architecture. It is illustrated in Figure 16.10.

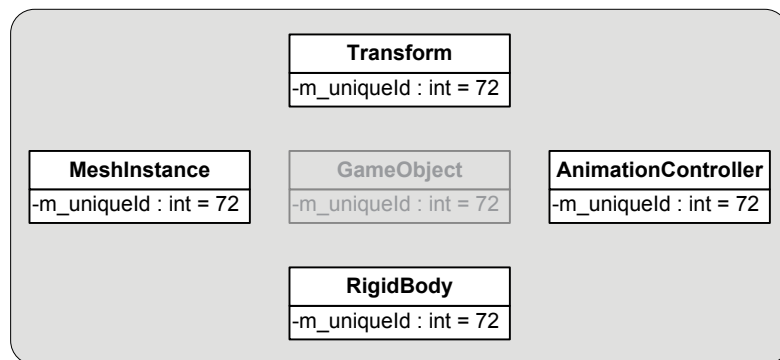


Figure 16.10. In a pure component model, a logical game object is comprised of many components, but the components are linked together only indirectly, by sharing a unique id.



A pure component model is not quite as simple as it first sounds, and it is not without its share of problems. For one thing, we still need some way of defining the various concrete types of game objects our game needs and then arranging for the correct component classes to be instantiated whenever an instance of the type is created. Our `GameObject` hierarchy used to handle construction of components for us. Instead, we might use a factory pattern, in which we define factory classes, one per game object type, with a virtual construction function that is overridden to create the proper components for each game object type. Or we might turn to a data-driven model, where the game object types are defined in a text file that can be parsed by the engine and consulted whenever a type is instantiated.

Another issue with a components-only design is inter-component communication. Our central `GameObject` acted as a “hub,” marshalling communications between the various components. In pure component architectures, we need an efficient way for the components making up a single game object to talk to one another. This could be done by having each component look up the other components using the game object’s unique id. However, we probably want a much more efficient mechanism—for example the components could be prewired into a circular linked list.

In the same sense, sending messages from one game object to another is difficult in a pure componentized model. We can no longer communicate with the `GameObject` instance, so we either need to know a priori with which component we wish to communicate, or we must multicast to all components that make up the game object in question. Neither option is ideal.

Pure component models can and have been made to work on real game projects. These kinds of models have their pros and cons, but again, they are not clearly better than any of the alternative designs. Unless you’re part of a research and development effort, you should probably choose the architecture with which you are most comfortable and confident, and which best fits the needs of the particular game you are building.

### 16.2.2 Property-Centric Architectures

Programmers who work frequently in an object-oriented programming language tend to think naturally in terms of objects that contain attributes (data members) and behaviors (methods, member functions). This is the *object-centric view*:

- `Object1`
  - `Position = (0, 3, 15)`
  - `Orientation = (0, 43, 0)`

- Object2
  - Position =  $(-12, 0, 8)$
  - Health = 15
- Object3
  - Orientation =  $(0, -87, 10)$

However, it is possible to think primarily in terms of the attributes, rather than the objects. We define the set of all properties that a game object might have. Then for each property, we build a table containing the values of that property corresponding to each game object that has it. The property values are keyed by the objects' unique ids. This is what we will call the *property-centric view*:

- Position
  - Object1 =  $(0, 3, 15)$
  - Object2 =  $(-12, 0, 8)$
- Orientation
  - Object1 =  $(0, 43, 0)$
  - Object3 =  $(0, -87, 10)$
- Health
  - Object2 = 15

Property-centric object models have been used very successfully on many commercial games, including *Deus Ex 2* and the *Thief* series of games. See Section 16.2.2.5 for more details on exactly how these projects designed their object systems.

A property-centric design is more akin to a relational database than an object model. Each attribute acts like a table in a relational database, with the game objects' unique id as the *primary key*. Of course, in object-oriented design, an object is defined not only by its *attributes*, but also by its *behavior*. If all we have are tables of properties, then where do we implement the behavior? The answer to this question varies somewhat from engine to engine, but most often the behaviors are implemented in one or both of the following places:

- in the properties themselves, and/or
- via script code.

Let's explore each of these ideas further.

### 16.2.2.1 Implementing Behavior via Property Classes

Each type of property can be implemented as a *property class*. Properties can be as simple as a single Boolean or floating-point value or as complex as a renderable triangle mesh or an AI “brain.” Each property class can provide behaviors via its hard-coded methods (member functions). The overall behavior of a particular game object is determined by the aggregation of the behaviors of all its properties.

For example, if a game object contains an instance of the `Health` property, it can be damaged and eventually destroyed or killed. The `Health` object can respond to any attacks made on the game object by decrementing the object’s health level appropriately. A property object can also communicate with other property objects within the same game object to produce cooperative behaviors. For example, when the `Health` property detects and responds to an attack, it could possibly send a message to the `AnimatedSkeleton` property, thereby allowing the game object to play a suitable hit reaction animation. Similarly, when the `Health` property detects that the game object is about to die or be destroyed, it can talk to the `RigidBodyDynamics` property to activate a physics-driven explosion or a “rag doll” dead body simulation.

### 16.2.2.2 Implementing Behavior via Script

Another option is to store the property values as raw data in one or more database-like tables and use script code to implement a game object’s behaviors. Every game object could have a special property called something like `scriptId`, which, if present, specifies the block of script code (script function, or script object if the scripting language is itself object-oriented) that will manage the object’s behavior. Script code could also be used to allow a game object to respond to events that occur within the game world. See Section 16.8 for more details on event systems and Section 16.9 for a discussion of game scripting languages.

In some property-centric engines, a core set of hard-coded property classes is provided by the engineers, but a facility is provided allowing game designers and programmers to implement new property types entirely in script. This approach was used successfully on the *Dungeon Siege* project, for example.

### 16.2.2.3 Properties versus Components

It’s important to note that many of the authors cited in Section 16.2.2.5 use the term “component” to refer to what I call a “property object” here. In Section 16.2.1.4, I used the term “component” to refer to a subobject in an object-centric design, which isn’t quite the same as a property object.

However, property objects are very closely related to components in many ways. In both designs, a single logical game object is made up of multiple subobjects. The main distinction lies in the roles of the subobjects. In a property-centric design, each subobject defines a particular attribute of the game object itself (e.g., health, visual representation, inventory, a particular magic power, etc.), whereas in a component-based (object-centric) design, the subobjects often represent linkages to particular low-level engine subsystems (renderer, animation, collision and dynamics, etc.) This distinction is so subtle as to be virtually irrelevant in many cases. You can call your design a *pure component model* (Section 16.2.1.6) or a *property-centric design* as you see fit, but at the end of the day, you'll have essentially the same result—a logical game object that is comprised of, and derives its behavior from, a collection of subobjects.

#### 16.2.2.4 Pros and Cons of Property-Centric Designs

There are a number of potential benefits to an attribute-centric approach. It tends to be more memory-efficient, because we need only store attribute data that is actually in use (i.e., there are never game objects with unused data members). It is also easier to construct such a model in a data-driven manner—designers can define new attributes easily, without recompiling the game, because there are no game object class definitions to be changed. Programmers need only get involved when entirely new types of properties need to be added (presuming the property cannot be added via script).

A property-centric design can also be more cache-friendly than an object-centric model, because data of the same type is stored *contiguously* in memory. This is a commonplace optimization technique on modern gaming hardware, where the cost of accessing memory is far higher than the cost of executing instructions and performing calculations. (For example, on the PlayStation 3, the cost of a single cache miss is equivalent to the cost of executing literally thousands of CPU instructions.) By storing data contiguously in RAM, we can reduce or eliminate cache misses, because when we access one element of a data array, a large number of its neighboring elements are loaded into the same cache line. This approach to data design is sometimes called the *struct of arrays* technique, in contrast to the more-traditional *array of structs* approach. The differences between these two memory layouts are illustrated by the code snippet below. (Note that we wouldn't really implement a game object model in exactly this way—this example is meant only to illustrate the way in which a property-centric design tends to produce many contiguous arrays of like-typed data, rather than a single array of complex objects.)

```
static const U32 MAX_GAME_OBJECTS = 1024;
```

```

// Traditional array-of-structs approach.

struct GameObject
{
    U32          m_uniqueId;
    Vector       m_pos;
    Quaternion   m_rot;
    float        m_health;

    // ...
};

GameObject g_aAllGameObjects[MAX_GAME_OBJECTS];

// Cache-friendlier struct-of-arrays approach.

struct AllGameObjects
{
    U32          m_aUniqueId[MAX_GAME_OBJECTS];
    Vector       m_aPos[MAX_GAME_OBJECTS];
    Quaternion   m_aRot[MAX_GAME_OBJECTS];
    float        m_aHealth[MAX_GAME_OBJECTS];

    // ...
};

AllGameObjects g_allGameObjects;

```

Attribute-centric models have their share of problems as well. For example, when a game object is just a grab bag of properties, it becomes much more difficult to enforce relationships between those properties. It can be hard to implement a desired large-scale behavior merely by cobbling together the fine-grained behaviors of a group of property objects. It's also much trickier to debug such systems, as the programmer cannot slap a game object into the watch window in the debugger in order to inspect all of its properties at once.

#### 16.2.2.5 Further Reading

A number of interesting PowerPoint presentations on the topic of property-centric architectures have been given by prominent engineers in the game industry at various game development conferences.

- Rob Fermier, "Creating a Data Driven Engine," Game Developer's Conference, 2002.
- Scott Bilas, "A Data-Driven Game Object System," Game Developer's Conference, 2002.

- Alex Duran, “Building Object Systems: Features, Tradeoffs, and Pitfalls,” Game Developer’s Conference, 2003.
- Jeremy Chatelaine, “Enabling Data Driven Tuning via Existing Tools,” Game Developer’s Conference, 2003.
- Doug Church, “Object Systems,” presented at a game development conference in Seoul, Korea, 2003; conference organized by Chris Hecker, Casey Muratori, Jon Blow and Doug Church. <http://chrishecker.com/images/6/6f/ObjSys.ppt>.

### 16.3 World Chunk Data Formats

As we’ve seen, a world chunk generally contains both *static* and *dynamic* world elements. The static geometry might be represented by one big triangle mesh, or it might be comprised of many smaller meshes. Each mesh might be *instanced* multiple times—for example, a single door mesh might be reused for all of the doorways in the chunk. The static data usually includes collision information stored as a triangle soup, a collection of convex shapes and/or other simpler geometric shapes like planes, boxes, capsules or spheres. Other static elements include volumetric *regions* that can be used to detect events or delineate areas within the game world, an AI *navigation mesh*, a set of line segments delineating *edges* within the background geometry that can be grabbed by the player character and so on. We won’t get into the details of these data formats here, because we’ve already discussed most of them in previous sections.

The dynamic portion of the world chunk contains some kind of representation of the game objects within that chunk. A game object is defined by its *attributes* and its *behaviors*, and an object’s behaviors are determined either directly or indirectly by its *type*. In an object-centric design, the object’s type directly determines which class(es) to instantiate in order to represent the object at runtime. In a property-centric design, a game object’s behavior is determined by the amalgamation of the behaviors of its properties, but the type still determines which properties the object should have (or one might say that an object’s properties define its type). So, for each game object, a world chunk data file generally contains:

- *The initial values of the objects’ attributes.* The world chunk defines the state of each game object as it should exist when first spawned into the game world. An object’s attribute data can be stored in a number of different formats. We’ll explore a few popular formats below.

- *Some kind of specification of the object's type.* In an object-centric engine, this might be a string, a hashed string id or some other unique type id. In a property-centric design, the type might be stored explicitly, or it might be defined implicitly by the collection of properties/attributes of which the object is comprised.

### 16.3.1 Binary Object Images

One way to store a collection of game objects into a disk file is to write a binary image of each object into the file, exactly as it looks in memory at runtime. This makes spawning game objects trivial. Once the game world chunk has been loaded into memory, we have ready-made images of all our objects, so we simply let them fly.

Well, not quite. Storing binary images of “live” C++ class instances is problematic for a number of reasons, including the need to handle *pointers* and *virtual tables* in a special way, and the possibility of having to *endian-swap* the data within each class instance. (These techniques are described in detail in Section 7.2.2.9.) Moreover, binary object images are inflexible and not robust to making changes. Gameplay is one of the most dynamic and unstable aspects of any game project, so it is wise to select a data format that supports rapid development and is robust to frequent changes. As such, the binary object image format is not usually a good choice for storing game object data (although this format can be suitable for more stable data structures, like mesh data or collision geometry).

### 16.3.2 Serialized Game Object Descriptions

*Serialization* is another means of storing a representation of a game object's internal state to a disk file. This approach tends to be more portable and simpler to implement than the binary object image technique. To serialize an object out to disk, the object is asked to produce a stream of data that contains enough detail to permit the original object to be reconstructed later. When an object is serialized back into memory from disk, an instance of the appropriate class is created, and then the stream of attribute data is read in order to initialize the new object's internal state. If the original serialized data stream was complete, the new object should be identical to the original for all intents and purposes.

Serialization is supported natively by some programming languages. For example, C# and Java both provide standardized mechanisms for serializing object instances to and from an XML text format. The C++ language unfortunately does not provide a standardized serialization facility. However, many C++ serialization systems have been successfully built, both inside and out-

side the game industry. We won't get into all the details of how to write a C++ object serialization system here, but we'll describe the data format and a few of the main systems that need to be written in order to get serialization to work in C++.

Serialization data isn't a binary image of the object. Instead, it is usually stored in a more-convenient and more-portable format. XML is a popular format for object serialization because it is well-supported and standardized, it is somewhat human-readable and it has excellent support for hierarchical data structures, which arise frequently when serializing collections of interrelated game objects. Unfortunately, XML is notoriously slow to parse, which can increase world chunk load times. For this reason, some game engines use a proprietary binary format that is faster to parse and more compact than XML text.

Many game engines (and non-game object serialization systems) have turned to the text-based JSON data format (<http://www.json.org>) as an alternative to XML. JSON is also used ubiquitously for data communication over the World Wide Web. For example, the Facebook API communicates exclusively using JSON.

The mechanics of serializing an object to and from disk are usually implemented in one of two basic ways:

- We can introduce a pair of virtual functions called something like `SerializeOut()` and `SerializeIn()` in our base class and arrange for each derived class to provide custom implementations of them that "know" how to serialize the attributes of that particular class.
- We can implement a *reflection* system for our C++ classes. We can then write a generic system that can automatically serialize any C++ object for which reflection information is available.

*Reflection* is a term used by the C# language, among others. In a nutshell, reflection data is a runtime description of the contents of a class. It stores information about the name of the class, what data members it contains, the types of each data member and the offset of each member within the object's memory image, and it also contains information about all of the class's member functions. Given reflection information for an arbitrary C++ class, we could quite easily write a general-purpose object serialization system.

The tricky part of a C++ reflection system is generating the reflection data for all of the relevant classes. This can be done by encapsulating a class's data members in `#define` macros that extract relevant reflection information by providing a virtual function that can be overridden by each derived class in



order to return appropriate reflection data for that class, by hand-coding a reflection data structure for each class, or via some other inventive approach.

In addition to attribute information, the serialization data stream invariably includes the name or unique id of each object's *class* or *type*. The class id is used to instantiate the appropriate class when the object is serialized into memory from disk. A class id can be stored as a string, a hashed string id, or some other kind of unique id.

Unfortunately, C++ provides no way to instantiate a class given only its name as a string or id. The class name must be known at compile time, and so it must be hard-coded by a programmer (e.g., `new ConcreteClass`). To work around this limitation of the language, C++ object serialization systems invariably include a *class factory* of some kind. A factory can be implemented in any number of ways, but the simplest approach is to create a data table that maps each class name/id to some kind of function or functor object that has been hard-coded to instantiate that particular class. Given a class name or id, we simply look up the corresponding function or functor in the table and call it to instantiate the class.

### 16.3.3 Spawners and Type Schemas

Both binary object images and serialization formats have an Achilles heel. They are both defined by the runtime implementation of the game object types they store, and hence they both require the world editor to contain intimate knowledge of the game engine's runtime implementation. For example, in order for the world editor to write out a binary image of a heterogeneous collection of game objects, it must either link directly with the runtime game engine code, or it must be painstakingly hand-coded to produce blocks of bytes that exactly match the data layout of the game objects at runtime. Serialization data is less-tightly coupled to the game object's implementation, but again, the world editor either needs to link with runtime game object code in order to gain access to the classes' `SerializeIn()` and `SerializeOut()` functions, or it needs access to the classes' reflection information in some way.

The coupling between the game world editor and the runtime engine code can be broken by abstracting the descriptions of our game objects in an implementation-independent way. For each game object in a world chunk data file, we store a little block of data, often called a *spawner*. A spawner is a lightweight, data-only representation of a game object that can be used to instantiate and initialize that game object at runtime. It contains the id of the game object's tool-side *type*. It also contains a table of simple key-value pairs that describe the initial attributes of the game object. These attributes

often include a model-to-world transform, since most game objects have a distinct position, orientation and scale in world space. When the game object is spawned, the appropriate class or classes are instantiated, as determined by the spawner's type. These runtime objects can then consult the dictionary of key-value pairs in order to initialize their data members appropriately.

A spawner can be configured to spawn its game object immediately upon being loaded, or it can lie dormant until asked to spawn at some later time during the game. Spawners can be implemented as first-class objects, so they can have a convenient functional interface and can store useful metadata in addition to object attributes. A spawner can even be used for purposes other than spawning game objects. For example, in the Naughty Dog engine, designers used spawners to define important points or coordinate axes in the game world. These were called *position spawners* or *locator spawners*. Locators have many uses in a game, such as:

- defining points of interest for an AI character,
- defining a set of coordinate axes relative to which a set of animations can be played in perfect synchronization,
- defining the location at which a particle effect or audio effect should originate,
- defining waypoints along a race track,

and the list goes on.

#### 16.3.3.1 Object Type Schemas

A game object's attributes and behaviors are defined by its type. In a game world editor that employs a spawner-based design, a game object type can be represented by a data-driven *schema* that defines the collection of attributes that should be visible to the user when creating or editing an object of that type. At runtime, the tool-side object type can be mapped in either a hard-coded or data-driven way to a class or collection of classes that must be instantiated in order to spawn a game object of the given type.

Type schemas can be stored in a simple text file for consumption by the world editor and for inspection and editing by its users. For example, a schema file might look something like this:

```
enum LightType
{
    Ambient, Directional, Point, Spot
}
```

```

type Light
{
    String      UniqueId;
    LightType   Type;
    Vector      Pos;
    Quaternion   Rot;
    Float        Intensity : min(0.0), max(1.0);
    ColorARGB    DiffuseColor;
    ColorARGB    SpecularColor;
    // ...
}

type Vehicle
{
    String      UniqueId;
    Vector      Pos;
    Quaternion   Rot;
    MeshReference Mesh;
    Int          NumWheels : min(2), max(4);
    Float        TurnRadius;
    Float        TopSpeed : min(0.0);
    // ...
}

//...

```

The above example brings a few important details to light. You'll notice that the *data types* of each attribute are defined, in addition to their names. These can be simple types like strings, integers and floating-point values, or they can be specialized types like vectors, quaternions, ARGB colors, or references to special asset types like meshes, collision data and so on. In this example, we've even provided a mechanism for defining enumerated types, like `LightType`. Another subtle point is that the object type schema provides additional information to the world editor, such as what type of GUI element to use when editing the attribute. Sometimes an attribute's GUI requirements are implied by its data type—strings are generally edited with a text field, Booleans via a check box, vectors via three text fields for the *x*-, *y*- and *z*-coordinates or perhaps via a specialized GUI element designed for manipulating vectors in 3D. The schema can also specify meta-information for use by the GUI, such as minimum and maximum allowable values for integer and floating-point attributes, lists of available choices for drop-down combo boxes and so on.

Some game engines permit object type schemas to be inherited, much like

classes. For example, every game object needs to know its *type* and must have a *unique id* so that it can be distinguished from all the other game objects at runtime. These attributes could be specified in a top-level schema, from which all other schemas are derived.

### 16.3.3.2 Default Attribute Values

As you can well imagine, the number of attributes in a typical game object schema can grow quite large. This translates into a lot of data that must be specified by the game designer for each instance of each game object type he or she places into the game world. It can be extremely helpful to define *default values* in the schema for many of the attributes. This permits game designers to place “vanilla” instances of a game object type with little effort but still permits him or her to fine-tune the attribute values on specific instances as needed.

One inherent problem with default values arises when the default value of a particular attribute changes. For example, our game designers might have originally wanted Orcs to have 20 hit points. After many months of production, the designers might decide that Orcs should be more powerful and have 30 hit points by default. Any new Orcs placed into a game world will now have 30 hit points unless otherwise specified. But what about all the Orcs that were placed into game world chunks prior to the change? Do we need to find all of these previously created Orcs and manually change their hit points to 30?

Ideally, we’d like to design our spawner system so that changes in default values automatically propagate to all preexisting instances that have not had their default values overridden explicitly. One easy way to implement this feature is to simply omit key-value pairs for attributes whose value does not differ from the default value. Whenever an attribute is missing from the spawner, the appropriate default can be used. (This presumes that the game engine has access to the object type schema file, so that it can read in the attributes’ default values. Either that or the tool can do it—in which case, propagating new default values requires a simple rebuild of all world chunk(s) affected by the change.) In our example, most of the preexisting Orc spawners would have had no `HitPoints` key-value pair at all (unless of course one of the spawner’s hit points had been changed from the default value manually). So when the default value changes from 20 to 30, these Orcs will automatically use the new value.

Some engines allow default values to be overridden in derived object types. For example, the schema for a type called `Vehicle` might define a default `TopSpeed` of 80 miles per hour. A derived `Motorcycle` type schema could override this `TopSpeed` to be 100 miles per hour.

### 16.3.3.3 Some Benefits of Spawners and Type Schemas

The key benefits of separating the spawner from the implementation of the game object are *simplicity*, *flexibility* and *robustness*. From a data management point of view, it is much simpler to deal with a table of key-value pairs than it is to manage a binary object image with pointer fix-ups or a custom serialized object format. The key-value pairs approach also makes the data format extremely flexible and robust to changes. If a game object encounters key-value pairs that it is not expecting to see, it can simply ignore them. Likewise, if the game object is unable to find a key-value pair that it needs, it has the option of using a default value instead. This makes a key-value pair data format extremely robust to changes made by both the designers and the programmers.

Spawners also simplify the design and implementation of the game world editor, because it only needs to know how to manage lists of key-value pairs and object type schemas. It doesn't need to share code with the runtime game engine in any way, and it is only very loosely coupled to the engine's implementation details.

Spawners and archetypes give game designers and programmers a great deal of flexibility and power. Designers can define new game object type schemas within the world editor with little or no programmer intervention. The programmer can implement the runtime implementation of these new object types whenever his or her schedule allows it. The programmer does not need to immediately provide an implementation of each new object type as it is added in order to avoid breaking the game. New object data can exist safely in the world chunk files with or without a runtime implementation, and runtime implementations can exist with or without corresponding data in the world chunk file.

## 16.4 Loading and Streaming Game Worlds

To bridge the gap between the offline world editor and our runtime game object model, we need a way to load world chunks into memory and unload them when they are no longer needed. The game world loading system has two main responsibilities: to manage the file I/O necessary to load game world chunks and other needed assets from disk into memory and to manage the allocation and deallocation of memory for these resources. The engine also needs to manage the *spawning* and *destruction* of game objects as they come and go in the game, both in terms of allocating and deallocating memory for the objects and ensuring that the proper classes are instantiated for each game object. In the following sections, we'll investigate how game worlds

are loaded and also have a look at how object spawning systems typically work.

### 16.4.1 Simple Level Loading

The most straightforward game world loading approach, and the one used by all of the earliest games, is to allow one and only one game world chunk (a.k.a. level) to be loaded at a time. When the game is first started, and between pairs of levels, the player sees a static or simply animated two-dimensional loading screen while he or she waits for the level to load.

Memory management in this kind of design is quite straightforward. As we mentioned in Section 7.2.2.7, a stack-based allocator is very well-suited to a one-level-at-a-time world loading design. When the game first runs, any resource data that is required across all game levels is loaded at the bottom of the stack. We'll call these *load and stay resident* assets (LSR) for the purposes of this discussion. The location of the stack pointer is recorded after the LSR assets have been fully loaded. Each game world chunk, along with its associated mesh, texture, audio, animation and other resource data, is loaded on top of the LSR assets on the stack. When the level has been completed by the player, all of its memory can be freed by simply resetting the stack pointer to the top of the LSR asset block. At this point, a new level can be loaded in its place. This is illustrated in Figure 16.11.

While this design is very simple, it has a number of drawbacks. For one thing, the player only sees the game world in discrete chunks—there is no way to implement a vast, contiguous, seamless world using this technique. Another problem is that during the time the level's resource data is being loaded, there is no game world in memory. So, the player is forced to watch a two-dimensional loading screen of some sort.

### 16.4.2 Toward Seamless Loading: Air Locks

The best way to avoid boring level-loading screens is to permit the player to continue playing the game while the next world chunk and its associated resource data are being loaded. One simple approach would be to divide the memory that we've set aside for game world assets into two equally sized blocks. We could load level A into one memory block, allow the player to start playing level A and then load level B into the other block using a streaming file I/O library (i.e., the loading code would run in a separate thread). The big problem with this technique is that it cuts the size of each level in half relative to what would be possible with a one-level-at-a-time approach.

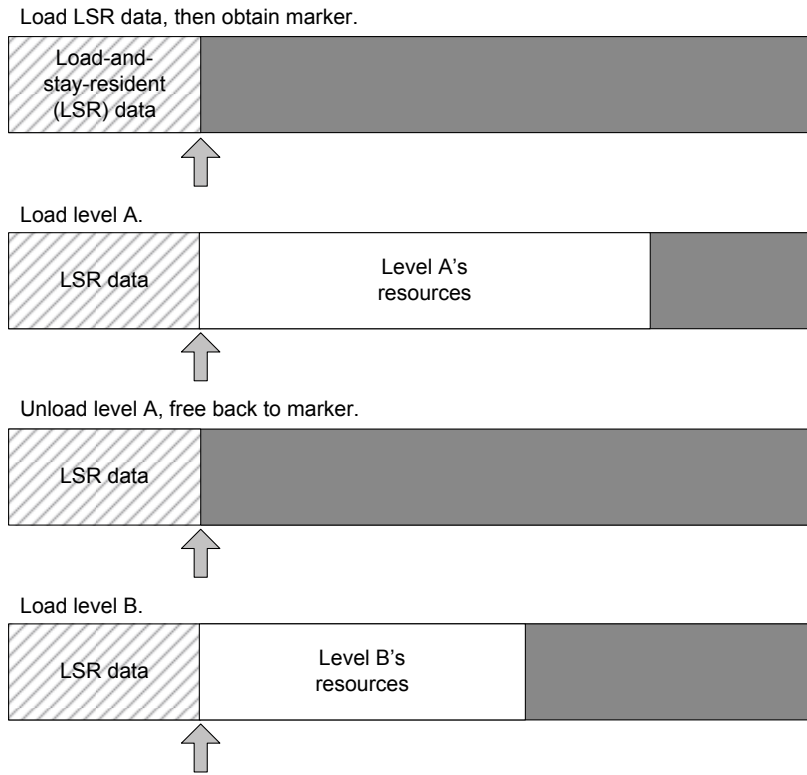


Figure 16.11. A stack-based memory allocator is extremely well-suited to a one-level-at-a-time world loading system.

We can achieve a similar effect by dividing the game world memory into two unequally sized blocks—a large block that can contain a “full” game world chunk and a small block that is only large enough to contain a tiny world chunk. The small chunk is sometimes known as an “air lock.”

When the game starts, a “full” chunk and an “air lock” chunk are loaded. The player progresses through the full chunk and into the air lock, at which point some kind of gate or other impediment ensures that the player can neither see the previous full world area nor return to it. The full chunk can then be un-loaded, and a new full-sized world chunk can be loaded. During the load, the player is kept busy doing some task within the air lock. The task might be as simple as walking from one end of a hallway to the other, or it could be something more engaging, like solving a puzzle or fighting some enemies.

Asynchronous file I/O is what enables the full world chunk to be loaded while the player is simultaneously playing in the air lock region. See Section 7.1.3 for more details. It's important to note that an air lock system does *not* free us from displaying a loading screen whenever a new game is started, because during the initial load there is no game world in memory in which to play. However, once the player is in the game world, he or she needn't see a loading screen ever again, thanks to air locks and asynchronous data loading.

*Halo* for the Xbox used a technique similar to this. The large world areas were invariably connected by smaller, more confined areas. As you play *Halo*, watch for confined areas that prevent you from back-tracking—you'll find one roughly every 5–10 minutes of gameplay. *Jak 2* for the PlayStation 2 used the air lock technique as well. The game world was structured as a hub area (the main city) with a number of offshoot areas, each of which was connected to the hub via a small, confined air lock region.

### 16.4.3 Game World Streaming

Many game designs call for the player to feel like he or she is playing in a huge, contiguous, seamless world. Ideally, the player should not be confined to small air lock regions periodically—it would be best if the world simply unfolded in front of the player as naturally and believably as possible.

Modern game engines support this kind of seamless world by using a technique known as *streaming*. World streaming can be accomplished in various ways. The main goals are always (a) to load data while the player is engaged in regular gameplay tasks and (b) to manage the memory in such a way as to eliminate *fragmentation* while permitting data to be loaded and unloaded as needed as the player progresses through the game world.

Recent consoles and PCs have a lot more memory than their predecessors, so it is now possible to keep multiple world chunks in memory simultaneously. We could imagine dividing our memory space into, say, three equally sized buffers. At first, we load world chunks A, B and C into these three buffers and allow the player to start playing through chunk A. When he or she enters chunk B and is far enough along that chunk A can no longer be seen, we can unload chunk A and start loading a new chunk D into the first buffer. When B can no longer be seen, it can be dumped and chunk E loaded. This recycling of buffers can continue until the player has reached the end of the contiguous game world.

The problem with a coarse-grained approach to world streaming is that it places onerous restrictions on the size of a world chunk. All chunks in the entire game must be roughly the same size—large enough to fill up the majority of one of our three memory buffers but never any larger.



One way around this problem is to employ a much finer-grained subdivision of memory. Rather than streaming relatively large chunks of the world, we can divide every game asset, from game world chunks to foreground meshes to textures to animation banks, into equally sized blocks of data. We can then use a chunky, pool-based memory allocation system like the one described in Section 7.2.2.7 to load and unload resource data as needed without having to worry about memory fragmentation. This is essentially the technique employed by Naughty Dog's engine. (Although Naughty Dog's implementation also employs some sophisticated techniques for making use of what would otherwise be unused space at the ends of under-full chunks.)

#### 16.4.3.1 Determining Which Resources to Load

One question that arises when using a fine-grained chunky memory allocator for world streaming is how the engine will know what resources to load at any given moment during gameplay. In the Naughty Dog engine, we use a relatively simple system of *level load regions* to control the loading and unloading of assets.

All of the *Uncharted* and *The Last of Us* games are set in multiple, geographically distinct, contiguous game worlds. For example, *Uncharted: Drake's Fortune* takes place in a jungle and on an island. Each of these worlds exists in a single, consistent world space, but they are divided up into numerous geographically adjacent chunks. A simple convex volume known as a *region* encompasses each of the chunks; the regions overlap each other somewhat. Each region contains a list of the world chunks that should be in memory when the player is in that region.

At any given moment, the player is within one or more of these regions. To determine the set of world chunks that should be in memory, we simply take the *union* of the chunk lists from each of the regions enclosing the Nathan Drake character. The level loading system periodically checks this master chunk list and compares it against the set of world chunks that are currently in memory. If a chunk disappears from the master list, it is unloaded, thereby freeing up all of the allocation blocks it occupied. If a new chunk appears in the list, it is loaded into any free allocation blocks that can be found. The level load regions and world chunks are designed in such a way as to ensure that the player never sees a chunk disappear when it is unloaded and that there's enough time between the moment at which a chunk starts loading and the moment its contents are first seen by the player to permit the chunk to be fully streamed into memory. This technique is illustrated in Figure 16.12.

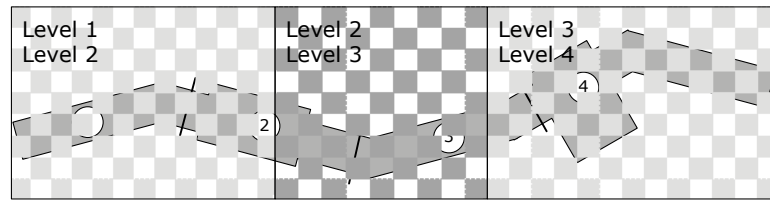


Figure 16.12. A game world divided into chunks. Level load regions, each with a requested chunk list, are arranged in such a way as to guarantee that the player never sees a chunk pop in or out of view.

#### 16.4.3.2 PlayGo on the PlayStation 4

The PlayStation 4 includes a new feature called PlayGo that makes the process of downloading a game (as opposed to buying it on Blu-ray) a lot less painful than it has traditionally been. PlayGo works by downloading only the minimum subset of data required in order to play the first section of the game. The PS4 downloads the rest of the game's content in the background, while the player continues to experience the game without interruption. In order for this to work well, the game must of course support seamless level streaming, as we've described above.

#### 16.4.4 Memory Management for Object Spawning

Once a game world has been loaded into memory, we need to manage the process of *spawning* the dynamic game objects in the world. Most game engines have some kind of game object spawning system that manages the instantiation of the class or classes that make up each game object and handles destruction of game objects when they are no longer needed. One of the central jobs of any object spawning system is to manage the dynamic allocation of memory for newly spawned game objects. Dynamic allocation can be slow, so steps must be taken to ensure allocations are as efficient as possible. And because game objects come in a wide variety of sizes, dynamically allocating them can cause memory to become fragmented, leading to premature out-of-memory conditions. There are a number of different approaches to game object memory management. We'll explore a few common ones in the following sections.

##### 16.4.4.1 OffLine Memory Allocation for Object Spawning

Some game engines solve the problems of allocation speed and memory fragmentation in a rather draconian way, by simply disallowing dynamic memory allocation during gameplay altogether. Such engines permit game world

chunks to be loaded and unloaded dynamically, but they spawn in all dynamic game objects immediately upon loading a chunk. Thereafter, no game objects can be created or destroyed. You can think of this technique as obeying a “law of conservation of game objects.” No game objects are created or destroyed once a world chunk has been loaded.

This technique avoids memory fragmentation because the memory requirements of all the game objects in a world chunk are (a) known a priori and (b) bounded. This means that the memory for the game objects can be allocated offline by the world editor and included as part of the world chunk data itself. All game objects are therefore allocated out of the same memory used to load the game world and its resources, and they are no more prone to fragmentation than any other loaded resource data. This approach also has the benefit of making the game’s memory usage patterns highly predictable. There’s no chance that a large group of game objects is going to spawn into the world unexpectedly, and cause the game to run out of memory.

On the downside, this approach can be quite limiting for game designers. Dynamic object spawning can be simulated by allocating a game object in the world editor but instructing it to be invisible and dormant when the world is first loaded. Later, the object can “spawn” by simply activating itself and making itself visible. But the game designers have to predict the total number of game objects of each type that they’ll need when the game world is first created in the world editor. If they want to provide the player with an infinite supply of health packs, weapons, enemies or some other kind of game object, they either need to work out a way to recycle their game objects, or they’re out of luck.

#### 16.4.4.2 Dynamic Memory Management for Object Spawning

Game designers would probably prefer to work with a game engine that supports true dynamic object spawning. Although this is more difficult to implement than a static game object spawning approach, it can be implemented in a number of different ways.

Again, the primary problem is memory fragmentation. Because different types of game objects (and sometimes even different instances of the same type of object) occupy different amounts of memory, we cannot use our favorite fragmentation-free allocator—the pool allocator. And because game objects are generally destroyed in a different order than that in which they were spawned, we cannot use a stack-based allocator either. Our only choice appears to be a fragmentation-prone heap allocator. Thankfully, there are many ways to deal with the fragmentation problem. We’ll investigate a few common ones in the following sections.

### *One Memory Pool per Object Type*

If the individual instances of each game object type are all guaranteed to occupy the same amount of memory, we could consider using a separate memory pool for each object type. Actually, we only need one pool per *unique game object size*, so object types of the same size can share a single pool.

Doing this allows us to completely avoid memory fragmentation, but one limitation of this approach is that we need to maintain lots of separate pools. We also need to make educated guesses about how many of each type of object we'll need. If a pool has too many elements, we end up wasting memory; if it has too few, we won't be able to satisfy all of the spawn requests at runtime, and game objects will fail to spawn.

### *Small Memory Allocators*

We can transform the idea of one pool per game object type into something more workable by allowing a game object to be allocated out of a pool whose elements are larger than the object itself. This can reduce the number of unique memory pools we need significantly, at the cost of some potentially wasted memory in each pool.

For example, we might create a set of pool allocators, each one with elements that are twice as large as those of its predecessor—perhaps 8, 16, 32, 64, 128, 256 and 512 bytes. We can also use a sequence of element sizes that conforms to some other suitable pattern or base the list of sizes on allocation statistics collected from the running game.

Whenever we try to allocate a game object, we search for the smallest pool whose elements are larger than or equal to the size of the object we're allocating. We accept that for some objects, we'll be wasting space. In return, we alleviate all of our memory fragmentation problems—a reasonably fair trade. If we ever encounter a memory allocation request that is larger than our largest pool, we can always turn it over to the general-purpose heap allocator, knowing that fragmentation of large memory blocks is not nearly as problematic as fragmentation involving tiny blocks.

This type of allocator is sometimes called a *small memory allocator*. It can eliminate fragmentation (for allocations that fit into one of the pools). It can also speed up memory allocations significantly for small chunks of data, because a pool allocation involves two pointer manipulations to remove the element from the linked list of free elements—a much less-expensive operation than a general-purpose heap allocation.

### *Memory Relocation*

Another way to eliminate fragmentation is to attack the problem directly. This approach is known as *memory relocation*. It involves shifting allocated memory blocks down into adjacent free holes to remove fragmentation. Moving the memory is easy, but because we are moving “live” allocated objects, we need to be very careful about fixing up any pointers into the memory blocks we move. See Section 6.2.2.2 for more details.

### 16.4.5 Saved Games

Many games allow the player to save his or her progress, quit the game and then load up the game at a later time in exactly the state he or she left it. A saved game system is similar to the world chunk loading system in that it is capable of loading the state of the game world from a disk file or memory card. But the requirements of this system differ somewhat from those of a world loading system, so the two are usually distinct (or overlap only partially).

To understand the differences between the requirements of these two systems, let’s briefly compare world chunks to saved game files. World chunks specify the initial conditions of all dynamic objects in the world, but they also contain a full description of all static world elements. Much of the static information, such as background meshes and collision data, tends to take up a lot of disk space. As such, world chunks are sometimes comprised of multiple disk files, and the total amount of data associated with a world chunk is usually large.

A saved game file must also store the current state information of the game objects in the world. However, it does not need to store a duplicate copy of any information that can be determined by reading the world chunk data. For example, there’s no need to save out the static geometry in a saved game file. A saved game need not store every detail of every object’s state either. Some objects that have no impact on gameplay can be omitted altogether. For the other game objects, we may only need to store partial state information. As long as the player can’t tell the difference between the state of the game world before and after it has been saved and reloaded (or if the differences are irrelevant to the player), then we have a successful saved game system. As such, saved game files tend to be much smaller than world chunk files and may place more of an emphasis on data compression and omission. Small file sizes are especially important when numerous saved game files must fit onto the tiny memory cards that were used on older consoles. But even today, with consoles that are equipped with large hard drives and linked to a cloud save system, it’s still a good idea to keep the size of a saved game file as small as

possible.

#### 16.4.5.1 Checkpoints

One approach to save games is to limit saves to specific points in the game, known as *checkpoints*. The benefit of this approach is that most of the knowledge about the state of the game is saved in the current world chunk(s) in the vicinity of each checkpoint. This data is always exactly the same, no matter which player is playing the game, so it needn't be stored in the saved game. As a result, saved game files based on checkpoints can be extremely small. We might need to store only the name of the last checkpoint reached, plus perhaps some information about the current state of the player character, such as the player's health, number of lives remaining, what items he has in his inventory, which weapon(s) he has and how much ammo each one contains. Some games based on checkpoints don't even store this information—they start the player off in a known state at each checkpoint. Of course, the downside of a game based on checkpoints is the possibility of user frustration, especially if checkpoints are few and far between.

#### 16.4.5.2 Save Anywhere

Some games support a feature known as *save anywhere*. As the name implies, such games permit the state of the game to be saved at literally any point during play. To implement this feature, the size of the saved game data file must increase significantly. The current locations and internal states of every game object whose state is relevant to gameplay must be saved and then restored when the game is loaded again later.

In a save-anywhere design, a saved game data file contains basically the same information as a world chunk, minus the world's static components. It is possible to utilize the same data format for both systems, although there may be factors that make this infeasible. For example, the world chunk data format might be designed for flexibility, but the saved game format might be compressed to minimize the size of each saved game.

As we've mentioned, one way to reduce the amount of data that needs to be stored in a saved game file is to omit certain irrelevant game objects and to omit some irrelevant details of others. For example, we needn't remember the exact time index within every animation that is currently playing or the exact momentums and velocities of every physically simulated rigid body. We can rely on the imperfect memories of human gamers and save only a rough approximation to the game's state.

## 16.5 Object References and World Queries

Every game object generally requires some kind of unique id so that it can be distinguished from the other objects in the game, found at runtime, serve as a target of inter-object communication and so on. Unique object ids are equally helpful on the tool side, as they can be used to identify and find game objects within the world editor.

At runtime, we invariably need various ways to find game objects. We might want to find an object by its unique id, by its type, or by a set of arbitrary criteria. We often need to perform proximity-based queries, for example finding all enemy aliens within a 10 m radius of the player character.

Once a game object has been found via a query, we need some way to refer to it. In a language like C or C++, object references might be implemented as pointers, or we might use something more sophisticated, like handles or smart pointers. The lifetime of an object reference can vary widely, from the scope of a single function call to a period of many minutes. In the following sections, we'll first investigate various ways to implement object references. Then we'll explore the kinds of queries we often require when implementing gameplay and how those queries might be implemented.

### 16.5.1 Pointers

In C or C++, the most straightforward way to implement an object reference is via a pointer (or a reference in C++). Pointers are powerful and are just about as simple and intuitive as you can get. However, pointers suffer from a number of problems:

- *Orphaned objects.* Ideally, every object should have an *owner*—another object that is responsible for managing its lifetime—creating it and then deleting it when it is no longer needed. But pointers don't give the programmer any help in enforcing this rule. The result can be an *orphaned* object—an object that still occupies memory but is no longer needed or referenced by any other object in the system.
- *Stale pointers.* If an object is deleted, ideally we should null-out any and all pointers to that object. If we forget to do so, however, we end up with a stale pointer—a pointer to a block of memory that used to be occupied by a valid object but is now free memory. If anyone tries to read or write data through a stale pointer, the result can be a crash or incorrect program behavior. Stale pointers can be difficult to track down because they may continue to work for some time after the object has deleted. Only much later, when a new object is allocated on top of the stale memory block, does the data actually change and cause a crash.

- *Invalid pointers.* A programmer is free to store any address in a pointer, including a totally invalid address. A common problem is dereferencing a null pointer. These problems can be guarded against by using assertion macros to check that pointers are never null prior to dereferencing them. Even worse, if a piece of data is misinterpreted as a pointer, dereferencing it can cause the program to read or write an essentially random memory address. This usually results in a crash or other major problem that can be very tough to debug.

Many game engines make heavy use of pointers, because they are by far the fastest, most efficient and easiest-to-work-with way to implement object references. However, experienced programmers are always wary of pointers, and some game teams turn to more sophisticated kinds of object references, either out of a desire to use safer programming practices or out of necessity. For example, if a game engine relocates allocated data blocks at runtime to eliminate memory fragmentation (see Section 6.2.2.2), simple pointers cannot be used. We either need to use a type of object reference that is robust to memory relocation, or we need to manually fix up any pointers into every relocated memory block at the time it is moved.

### 16.5.2 Smart Pointers

A *smart pointer* is a small object that acts like a pointer for most intents and purposes but avoids most of the problems inherent with native C/C++ pointers. At its simplest, a smart pointer contains a native pointer as a data member and provides a set of overloaded operators that make it act like a pointer in most ways. Pointers can be dereferenced, so the `*` and `->` operators are overloaded to return a reference and a pointer to the referenced object, respectively, as you'd expect. Pointers can undergo arithmetic operations, so the `+`, `-`, `++` and `--` operators are also overloaded appropriately.

Because a smart pointer is an object, it can contain additional metadata and/or take additional steps not possible with a regular pointer. For example, a smart pointer might contain information that allows it to recognize when the object to which it points has been deleted and start returning a null address if so.

Smart pointers can also help with object lifetime management by cooperating with one another to determine the number of references to a particular object. This is called *reference counting*. When the number of smart pointers that reference a particular object drops to zero, we know that the object is no longer needed, so it can be automatically deleted. This can free the programmer from having to worry about object ownership and orphaned objects.



Reference counting usually also lies at the core of the “garbage collection” systems found in modern programming languages like Java and Python.

Smart pointers have their share of problems. For one thing, they are relatively easy to implement, but they are tough to get right. There are a great many cases to handle, and the `std::auto_ptr` class provided by the original C++ standard library is widely recognized to be inadequate in many situations. Thankfully most of these issues were resolved in C++11 with the introduction of three smart pointer classes: `std::shared_ptr`, `std::weak_ptr` and `std::unique_ptr`.

The C++11 smart pointer classes were modeled after the rich smart pointer facilities provided by the Boost C++ template library. It defines six different varieties of smart pointers:

- `boost::scoped_ptr`. A pointer to a single object with one owner.
- `boost::scoped_array`. A pointer to an array of objects with a single owner.
- `boost::shared_ptr`. A pointer to an object whose lifetime is shared by multiple owners.
- `boost::shared_array`. A pointer to an array of objects whose lifetimes are shared by multiple owners.
- `boost::weak_ptr`. A pointer that does not own or automatically destroy the object it references (whose lifetime is assumed to be managed by a `shared_ptr`).
- `boost::intrusive_ptr`. A pointer that implements reference counting by assuming that the pointed-to object will maintain the reference count itself. Intrusive pointers are useful because they are the same size as a native C++ pointer (because no reference-counting apparatus is required) and because they can be constructed directly from native pointers.

Properly implementing a smart pointer class can be a daunting task. Have a glance at the Boost smart pointer documentation ([http://www.boost.org/doc/libs/1\\_36\\_0/libs/smart\\_ptr/smart\\_ptr.htm](http://www.boost.org/doc/libs/1_36_0/libs/smart_ptr/smart_ptr.htm)) to see what I mean. All sorts of issues come up, including:

- type safety of smart pointers,
- the ability for a smart pointer to be used with an incomplete type,
- correct smart pointer behavior when an exception occurs, and
- runtime costs, which can be high.

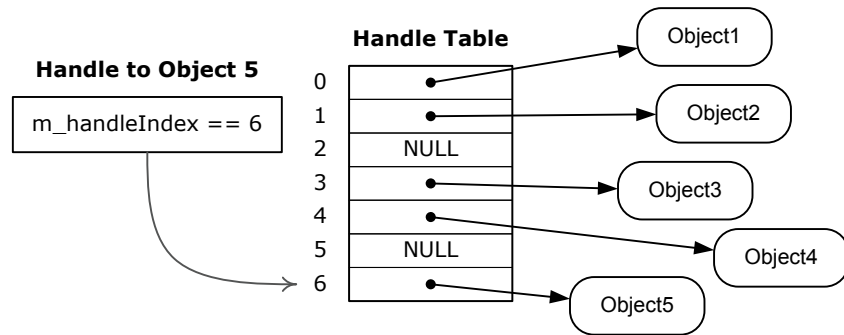


Figure 16.13. A handle table contains raw object pointers. A handle is simply an index into this table.

I worked on a project that attempted to implement its own smart pointers, and we were fixing all sorts of nasty bugs with them up until the very end of the project. My personal recommendation is to stay away from smart pointers; if you must use them, use a mature implementation such as the C++11 standard library or Boost, rather than rolling your own.

### 16.5.3 Handles

A *handle* acts like a smart pointer in many ways, but it is simpler to implement and tends to be less prone to problems. A handle is basically an integer index into a global *handle table*. The handle table, in turn, contains pointers to the objects to which the handles refer. To create a handle, we simply search the handle table for the address of the object in question and store its index in the handle. To dereference a handle, the calling code simply indexes the appropriate slot in the handle table and dereferences the pointer it finds there. This is illustrated in Figure 16.13.

Because of the simple level of indirection afforded by the handle table, handles are much safer and more flexible than raw pointers. If an object is deleted, it can simply null out its entry in the handle table. This causes all existing handles to the object to be immediately and automatically converted to null references. Handles also support memory relocation. When an object is relocated in memory, its address can be found in the handle table and updated appropriately. Again, all existing handles to the object are automatically updated as a result.

A handle can be implemented as a raw integer. However, the handle table index is usually wrapped in a simple class so that a convenient interface for creating and dereferencing the handle can be provided.

Handles are prone to the possibility of referencing a stale object. For example, let's say we create a handle to object A, which occupies slot 17 in the handle table. Later, object A is deleted, and slot 17 is nulled out. Later still, a new object B is created, and it just happens to occupy slot 17 in the handle table. If there are still any handles to object A lying around when object B is created, they will suddenly start referring to object B (instead of null). This is certainly not desirable behavior.

One simple solution to the stale handle problem is to include a unique object id in each handle. That way, when a handle to object A is created, it contains not only slot index 17, but the object id "A." When object B takes A's place in the handle table, any leftover handles to A will agree on the handle index but disagree on the object id. This allows stale object A handles to continue to return null when dereferenced rather than returning a pointer to object B unexpectedly.

The following code snippet shows how a simple handle class might be implemented. Notice that we've also included the handle index in the `GameObject` class itself—this allows us to create new handles to a `GameObject` very quickly without having to search the handle table for its address to determine its handle index.

```
// Within the GameObject class, we store a unique id,
// and also the object's handle index, for efficient
// creation of new handles.

class GameObject
{
private:
    // ...

    GameObjectId m_uniqueId;           // object's unique id
    U32          m_handleIndex;        // speedier handle
                                           // creation

    friend class GameObjectHandle;      // access to id and
                                           // index

    // ...

public:
    GameObject() // constructor
    {
        // The unique id might come from the world editor,
        // or it might be assigned dynamically at runtime.
        m_uniqueId = AssignUniqueObjectId();
```

```

        // The handle index is assigned by finding the
        // first free slot in the handle table.
        m_handleIndex = FindFreeSlotInHandleTable();

        // ...
    }
    // ...

};

// This constant defines the size of the handle table,
// and hence the maximum number of game objects that can
// exist at any one time.
static const U32 MAX_GAME_OBJECTS = 2048;

// This is the global handle table -- a simple array of
// pointers to GameObjects.
static GameObject* g_apGameObject[MAX_GAME_OBJECTS];

// This is our simple game object handle class.
class GameObjectHandle
{
private:
    U32 m_handleIndex;           // index into the handle
                                // table
    GameObjectId m_uniqueId;     // unique id avoids stale
                                // handles

public:
    explicit GameObjectHandle(GameObject& object) :
        m_handleIndex(object.m_handleIndex),
        m_uniqueId(object.m_uniqueId)
    {
    }

    // This function dereferences the handle.
    GameObject* ToObject() const
    {
        GameObject* pObject
            = g_apGameObject[m_handleIndex];

        if (pObject != nullptr
            && pObject->m_uniqueId == m_uniqueId)
        {
            return pObject;
        }
    }

```

```
        return nullptr;
    }
};
```

This example is functional but incomplete. We might want to implement copy semantics, provide additional constructor variants and so on. The entries in the global handle table might contain additional information, not just a raw pointer to each game object. And of course, a fixed size handle table implementation like this one isn't the only possible design; handle systems vary somewhat from engine to engine.

We should note that one fortunate side benefit of a global handle table is that it gives us a ready-made list of all active game objects in the system. The global handle table can be used to quickly and efficiently iterate over all game objects in the world, for example. It can also make implementing other kinds of queries easier in some cases.

#### 16.5.4 Game Object Queries

Every game engine provides at least a few ways to find game objects at runtime. We'll call these searches *game object queries*. The simplest type of query is to find a particular game object by its unique id. However, a real game engine makes many other types of game object queries. Here are just a few examples of the kinds of queries a game developer might want to make:

- Find all enemy characters with line of sight to the player.
- Iterate over all game objects of a certain type.
- Find all destructible game objects with more than 80% health.
- Transmit damage to all game objects within the blast radius of an explosion.
- Iterate over all objects in the path of a bullet or other projectile, in nearest-to-farthest order.

This list could go on for many pages, and of course its contents are highly dependent upon the design of the particular game being made.

For maximum flexibility in performing game object queries, we could imagine a general-purpose game object database, complete with the ability to formulate arbitrary queries using arbitrary search criteria. Ideally, our game object database would perform all of these queries extremely efficiently and rapidly, making maximum use of whatever hardware and software resources are available.

In reality, such an ideal combination of flexibility and blinding speed is generally not possible. Instead, game teams usually determine which types of queries are most likely to be needed during development of the game, and specialized data structures are implemented to accelerate those particular types of queries. As new queries become necessary, the engineers either leverage preexisting data structures to implement them, or they invent new ones if sufficient speed cannot be obtained. Here are a few examples of specialized data structures that can accelerate specific types of game object queries:

- *Finding game objects by unique id.* Pointers or handles to the game objects could be stored in a hash table or binary search tree keyed by unique id.
- *Iterating over all objects that meet a particular criterion.* The game objects could be presorted into linked lists based on various criteria (presuming the criteria are known a priori). For example, we might construct a list of all game objects of a particular type, maintain a list of all objects within a particular radius of the player, etc.
- *Finding all objects in the path of a projectile or with line of sight to some target point.* The collision system is usually leveraged to perform these kinds of game object queries. Most collision systems provide fast ray casts, and some also provide the ability to cast other shapes such as spheres or arbitrary convex volumes into the world to determine what they hit. (See Section 13.3.7.)
- *Finding all objects within a given region or radius.* We might consider storing our game objects in some kind of spatial hash data structure. This could be as simple as a horizontal grid placed over the entire game world or something more sophisticated, such as a quadtree, octree, kd-tree or other data structure that encodes spatial proximity.

## 16.6 Updating Game Objects in Real Time

Every game engine, from the simplest to the most complex, requires some means of updating the internal state of every game object over time. The *state* of a game object can be defined as the values of all its *attributes* (sometimes called its *properties*, and called *data members* in the C++ language). For example, the state of the ball in *Pong* is described by its  $(x, y)$  position on the screen and its velocity (speed and direction of travel). Because games are dynamic, time-based simulations, a game object's state describes its configuration at *one specific instant in time*. In other words, a game object's notion of time is *discrete* rather than *continuous*. (However, as we'll see, it's helpful to think of the ob-

jects' states as changing continuously and then being sampled discretely by the engine, because it helps you to avoid some common pitfalls.)

In the following discussions, we'll use the symbol  $\mathbf{S}_i(t)$  to denote the state of object  $i$  at an arbitrary time  $t$ . The use of vector notation here is not strictly mathematically correct, but it reminds us that a game object's state acts like a heterogeneous  $n$ -dimensional vector, containing all sorts of information of various data types. We should note that this usage of the term "state" is not the same as the states in a *finite state machine*. A game object may very well be implemented in terms of one—or many—finite state machines, but in that case, a specification of the current state of each FSM would merely be a part of the game object's overall state vector  $\mathbf{S}(t)$ .

Most low-level engine subsystems (rendering, animation, collision, physics, audio and so on) require periodic updating, and the game object system is no exception. As we saw in Chapter 8, updating is usually done via a single master loop called the *game loop*. Virtually all game engines update game object states as part of their main game loop—in other words, they treat the game object model as just another engine subsystem that requires periodic servicing.

Game object updating can therefore be thought of as the process of determining the state of each object at the current time  $\mathbf{S}_i(t)$  given its state at a previous time  $\mathbf{S}_i(t - \Delta t)$ . Once all object states have been updated, the current time  $t$  becomes the new previous time  $(t - \Delta t)$ , and this process repeats for as long as the game is running. Usually, one or more *clocks* are maintained by the engine—one that tracks real time exactly and possibly others that may or may not correspond to real time. These clocks provide the engine with the absolute time  $t$  and/or with the change in time  $\Delta t$  from iteration to iteration of the game loop. The clock that drives the updating of game object states is usually permitted to diverge from real time. This allows the behaviors of the game objects to be paused, slowed down, sped up or even run in reverse—whatever is required in order to suit the needs of the game design. These features are also invaluable for debugging and development of the game.

As we learned in Chapter 1, a game object updating system is an example of what is known as a *dynamic, real-time, agent-based computer simulation* in computer science. Game object updating systems also exhibit some aspects of *discrete event simulations* (see Section 16.8 for more details on events). These are well-researched areas of computer science, and they have many applications outside the field of interactive entertainment. Games are one of the more complex kinds of agent-based simulation—as we'll see, updating game object states over time in a dynamic, interactive virtual environment can be surprisingly difficult to get right. Game programmers can learn a lot about game

object updating by studying the wider field of agent-based and discrete event simulations. And researchers in those fields can probably learn a thing or two from game engine design as well!

As with all high-level game engine systems, every engine takes a slightly (or sometimes radically) different approach. However, as before, most game teams encounter a common set of problems, and certain design patterns tend to crop up again and again in virtually every engine. In this section, we'll investigate these common problems and some common solutions to them. Please bear in mind that game engines may exist that employ very different solutions to the ones described here, and some game designs face unique problems that we can't possibly cover here.

### 16.6.1 A Simple Approach (That Doesn't Work)

The simplest way to update the states of a collection of game objects is to iterate over the collection and call a virtual function, named something like `Update()`, on each object in turn. This is typically done once during each iteration of the main game loop (i.e., once per *frame*). Game object classes can provide custom implementations of the `Update()` function in order to perform whatever tasks are required to advance the state of that type of object to the next discrete time index. The time delta from the previous frame can be passed to the update function so that objects can take proper account of the passage of time. At its simplest, then, our `Update()` function's signature might look something like this:

```
virtual void Update(float dt);
```

For the purposes of the following discussions, we'll assume that our engine employs a monolithic object hierarchy, in which each game object is represented by a single instance of a single class. However, we can easily extend the ideas here to virtually any object-centric design. For example, to update a component-based object model, we could call `Update()` on every component that makes up each game object, or we could call `Update()` on the "hub" object and let it update its associated components as it sees fit. We can also extend these ideas to property-centric designs, by calling some sort of `Update()` function on each property instance every frame.

They say that the devil is in the details, so let's investigate two important details here. First, how should we maintain the collection of all game objects? And second, what kinds of things should the `Update()` function be responsible for doing?



### 16.6.1.1 Maintaining a Collection of Active Game Objects

The collection of active game objects is often maintained by a singleton manager class, perhaps named something like `GameWorld` or `GameObjectManager`. The collection of game objects generally needs to be dynamic, because game objects are spawned and destroyed as the game is played. Hence a *linked list* of pointers, smart pointers or handles to game objects is one simple and effective approach. (Some game engines disallow dynamic spawning and destroying of game objects; such engines can use a statically sized *array* of game object pointers, smart pointers or handles rather than a linked list.) As we'll see below, most engines use more complex data structures to keep track of their game objects rather than just a simple, flat linked list. But for the time being, we can visualize the data structure as a linked list for simplicity.

### 16.6.1.2 Responsibilities of the `Update()` Function

A game object's `Update()` function is primarily responsible for determining the state of that game object at the current discrete time index  $S_i(t)$  given its previous state  $S_i(t - \Delta t)$ . Doing this may involve applying a rigid body dynamics simulation to the object, sampling a preauthored animation, reacting to events that have occurred during the current time step and so on.

Most game objects interact with one or more engine subsystems. They may need to animate, be rendered, emit particle effects, play audio, collide with other objects and static geometry and so on. Each of these systems has an internal state that must also be updated over time, usually once or a few times per frame. It might seem reasonable and intuitive to simply update all of these subsystems directly from within the game object's `Update()` function. For example, consider the following hypothetical update function for a Tank object:

```
virtual void Tank::Update(float dt)
{
    // Update the state of the tank itself.
    MoveTank(dt);
    DeflectTurret(dt);
    FireIfNecessary();

    // Now update low-level engine subsystems on behalf
    // of this tank. (NOT a good idea... see below!)
    m_pAnimationComponent->Update(dt);
    m_pCollisionComponent->Update(dt);
    m_pPhysicsComponent->Update(dt);
    m_pAudioComponent->Update(dt);
    m_pRenderingComponent->draw();
}
```

```
}
```

Given that our `Update()` functions are structured like this, the game loop could be driven almost entirely by the updating of the game objects, like this:

```
while (true)
{
    PollJoypad();

    float dt = g_gameClock.CalculateDeltaTime();

    for (each gameObject)
    {
        // This hypothetical Update() function updates
        // all engine subsystems!
        gameObject.Update(dt);
    }

    g_renderingEngine.SwapBuffers();
}
```

However attractive the simple approach to object updating shown above may seem, it is usually not viable in a commercial-grade game engine. In the following sections, we'll explore some of the problems with this simplistic approach and investigate common ways in which each problem can be solved.

### 16.6.2 Performance Constraints and Batched Updates

Most low-level engine systems have extremely stringent performance constraints. They operate on a large quantity of data, and they must do a large number of calculations every frame as quickly as possible. As a result, most engine systems benefit from *batched updating*. For example, it is usually far more efficient to update a large number of animations in one batch than it is to update each object's animation interleaved with other unrelated operations, such as collision detection, physical simulation and rendering.

In most commercial game engines, each engine subsystem is updated directly or indirectly by the main game loop rather than being updated on a per-game object basis from within each object's `Update()` function. If a game object requires the services of a particular engine subsystem, it asks that subsystem to allocate some subsystem-specific state information on its behalf. For example, a game object that wishes to be rendered via a triangle mesh might request the rendering subsystem to allocate a *mesh instance* for its use. (As described in Section 11.1.1.5, a mesh instance represents a single instance of a

triangle mesh—it keeps track of the position, orientation and scale of the instance in world space whether or not it is visible, per-instance material data and any other per-instance information that may be relevant.) The rendering engine maintains a collection of mesh instances internally. It can manage the mesh instances however it sees fit in order to maximize its own runtime performance. The game object controls *how* it is rendered by manipulating the properties of the mesh instance object, but the game object does not control the rendering of the mesh instance directly. Instead, after all game objects have had a chance to update themselves, the rendering engine draws all visible mesh instances in one efficient batch update.

With batched updating, a particular game object's `Update()` function, such as that of our hypothetical tank object, might look more like this:

```
virtual void Tank::Update(float dt)
{
    // Update the state of the tank itself.
    MoveTank(dt);
    DeflectTurret(dt);
    FireIfNecessary();

    // Control the properties of my various engine
    // subsystem components, but do NOT update
    // them here...

    if (justExploded)
    {
        m_pAnimationComponent->PlayAnimation("explode");
    }

    if (isVisible)
    {
        m_pCollisionComponent->Activate();
        m_pRenderingComponent->Show();
    }
    else
    {
        m_pCollisionComponent->Deactivate();
        m_pRenderingComponent->Hide();
    }

    // etc.
}
```

The game loop then ends up looking more like this:

```
while (true)
{
```

```
PollJoypad();

float dt = g_gameClock.CalculateDeltaTime();

for (each gameObject)
{
    gameObject.Update(dt);
}

g_animationEngine.Update(dt);
g_physicsEngine.Simulate(dt);
g_collisionEngine.DetectAndResolveCollisions(dt);
g_audioEngine.Update(dt);
g_renderingEngine.RenderFrameAndSwapBuffers();
}
```

Batched updating provides many performance benefits, including but not limited to:

- *Maximal cache coherency.* Batched updating allows an engine subsystem to achieve maximum cache coherency because its per-object data is maintained internally and can be arranged in a single, contiguous region of RAM.
- *Minimal duplication of computations.* Global calculations can be done once and reused for many game objects rather than being redone for each object.
- *Reduced reallocation of resources.* Engine subsystems often need to allocate and manage memory and/or other resources during their updates. If the update of a particular subsystem is interleaved with those of other engine subsystems, these resources must be freed and reallocated for each game object that is processed. But if the updates are batched, the resources can be allocated once per frame and reused for all objects in the batch.
- *Efficient pipelining.* Many engine subsystems perform a virtually identical set of calculations on each and every object in the game world. When updates are batched, a scatter/gather approach can be employed to divide large workloads across multiple CPU cores. This kind of parallelism cannot be achieved when processing each object in isolation.

Performance benefits aren't the only reason to favor a batch updating approach. Some engine subsystems simply don't work at all when updated on a per-object basis. For example, if we are trying to resolve collisions within a system of multiple dynamic rigid bodies, a satisfactory solution cannot be

found in general by considering each object in isolation. The interpenetrations between these objects must be resolved as a group, either via an iterative approach or by solving a linear system.

### 16.6.3 Object and Subsystem Interdependencies

Even if we didn't care about performance, a simplistic per-object updating approach breaks down when game objects *depend* on one another. For example, a human character might be holding a cat in her arms. In order to calculate the world-space pose of the cat's skeleton, we first need to calculate the world-space pose of the human. This implies that the *order* in which objects are updated is important to the proper functioning of the game.

Another related problem arises when engine *subsystems* depend on one another. For example, a rag doll physics simulation must be updated in concert with the animation engine. Typically, the animation system produces an intermediate, local-space skeletal pose. These joint transforms are converted to world space and applied to a system of connected rigid bodies that approximate the skeleton within the physics system. The rigid bodies are simulated forward in time by the physics system, and then the final resting places of the joints are applied back to their corresponding joints in the skeleton. Finally, the animation system calculates the world-space pose and skinning matrix palette. So once again, the updating of the animation and physics systems must occur in a particular order in order to produce correct results. These kinds of inter-subsystem dependencies are commonplace in game engine design.

#### 16.6.3.1 Phased Updates

To account for inter-subsystem dependencies, we can explicitly code our engine subsystem updates in the proper order within the main game loop. For example, to handle the interplay between the animation system and rag doll physics, we might write something like this:

```
while (true) // main game loop
{
    // ...

    g_animationEngine.CalculateIntermediatePoses(dt);
    g_ragdollSystem.ApplySkeletonsToRagDolls();
    g_physicsEngine.Simulate(dt); // runs ragdolls too
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons();
    g_animationEngine.FinalizePoseAndMatrixPalette();
```

```

        // ...
    }

```

We must be careful to update the states of our game objects at the right time during the game loop. This is often not as simple as calling a single `Update()` function per game object per frame. Game objects may depend upon the intermediate results of calculations performed by various engine subsystems. For example, a game object might request that animations be played prior to the animation system running its update. However, that same object may also want to procedurally adjust the intermediate pose generated by the animation system prior to that pose being used by the rag doll physics system and/or the final pose and matrix palette being generated. This implies that the object must be updated twice, once before the animation calculates its intermediate poses and once afterward.

Many game engines allow their game objects to run update logic at multiple points during the frame. For example, the Naughty Dog engine (the engine that drives the *Uncharted* and *The Last of Us* game series) updates game objects three times—once before animation blending, once after animation blending but prior to final pose generation and once after final pose generation. This can be accomplished by providing each game object class with three virtual functions that act as “hooks.” In such a system, the game loop ends up looking something like this:

```

while (true) // main game loop
{
    // ...

    for (each gameObject)
    {
        gameObject.PreAnimUpdate(dt);
    }

    g_animationEngine.CalculateIntermediatePoses(dt);

    for (each gameObject)
    {
        gameObject.PostAnimUpdate(dt);
    }

    g_ragdollSystem.ApplySkeletonsToRagDolls();
    g_physicsEngine.Simulate(dt); // runs ragdolls too
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons();
}

```

```
g_animationEngine.FinalizePoseAndMatrixPalette();

for (each gameObject)
{
    gameObject.FinalUpdate(dt);
}

// ...
}
```

We can provide our game objects with as many update phases as we see fit. But we must be careful, because iterating over all game objects and calling a virtual function on each one can be expensive. Also, not all game objects require all update phases—iterating over objects that don’t require a particular phase is a pure waste of CPU bandwidth.

Actually, the above example isn’t completely realistic. Iterating directly over all game objects to call their `PreAnimUpdate()`, `PostAnimUpdate()` and `FinalUpdate()` hook functions would be highly inefficient, because only a small percentage of the objects might actually need to perform any logic in each hook. It’s also an inflexible design, because only game objects are supported—if we wanted to update a particle system during the post-animation phase, we’d be out of luck. Finally, such a design would lead to unnecessary *coupling* between the low-level engine systems and the game object system.

A generic callback mechanism would be a much better design choice. In such a design, the animation system would provide a facility by which any client code (game objects or any other engine system) could *register a callback function* for each of the three update phases (pre-animation, post-animation and final). The animation system would iterate through all registered callbacks and call them, without any “knowledge” of game objects *per se*. This design maximizes performance, because only those clients that actually *need* updates register callbacks and are called each frame. It also maximizes flexibility and eliminates unnecessary coupling between the game object system and other engine subsystems, because any client is allowed to register a callback, not just game objects.

#### 16.6.3.2 Bucketed Updates

In the presence of *inter-object* dependencies, the phased updates technique described above must be adjusted a little. This is because inter-object dependencies can lead to conflicting rules governing the order of updating. For example, let’s imagine that object B is being held by object A. Further, let’s assume that we can only update object B after A has been *fully* updated, including the

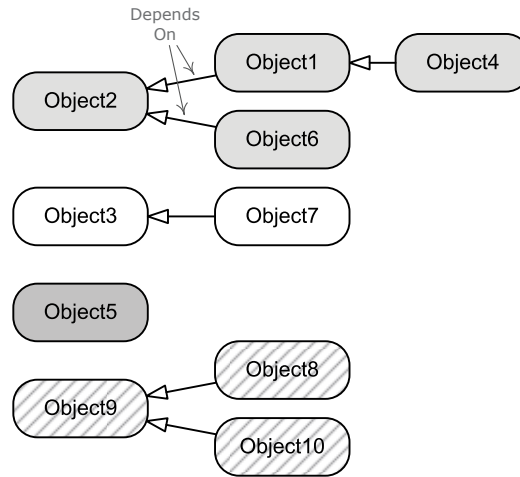


Figure 16.14. Inter-object update order dependencies can be viewed as a forest of dependency trees.

calculation of its final world-space pose and matrix palette. This conflicts with the need to batch animation updates of all game objects together in order to allow the animation system to achieve maximum throughput.

Inter-object dependencies can be visualized as a forest of dependency trees. The game objects with no parents (no dependencies on any other object) represent the roots of the forest. An object that depends directly on one of these root objects resides in the first tier of children in one of the trees in the forest. An object that depends on a first-tier child becomes a second-tier child and so on. This is illustrated in Figure 16.14.

One solution to the problem of conflicting update order requirements is to collect objects into independent groups, which we'll call *buckets* here for lack of a better name. The first bucket consists of all root objects in the forest. The second bucket is comprised of all first-tier children. The third bucket contains all second-tier children and so on. For each bucket, we run a complete update of the game objects and the engine systems, complete with all update phases. Then we repeat the entire process for each bucket until there are no more buckets.

In theory, the depths of the trees in our dependency forest are unbounded. However, in practice, they are usually quite shallow. For example, we might have characters holding weapons, and those characters might or might not be riding on a moving platform or a vehicle. To implement this, we only need three tiers in our dependency forest, and hence only three buckets: one for



platforms/vehicles, one for characters and one for the weapons in the characters' hands. Many game engines explicitly limit the depth of their dependency forest so that they can use a fixed number of buckets (presuming they use a bucketed approach at all—there are of course many other ways to architect a game loop).

Here's what a bucketed, phased, batched update loop might look like:

```
enum Bucket
{
    kBucketVehiclesPlatforms,
    kBucketCharacters,
    kBucketAttachedObjects,
    kBucketCount
};

void UpdateBucket(Bucket bucket)
{
    // ...

    for (each gameObject in bucket)
    {
        gameObject.PreAnimUpdate(dt);
    }

    g_animationEngine.CalculateIntermediatePoses
        (bucket, dt);

    for (each gameObject in bucket)
    {
        gameObject.PostAnimUpdate(dt);
    }

    g_ragdollSystem.ApplySkeletonsToRagDolls(bucket);
    g_physicsEngine.Simulate(bucket, dt); // ragdolls etc.
    g_collisionEngine.DetectAndResolveCollisions
        (bucket, dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons(bucket);
    g_animationEngine.FinalizePoseAndMatrixPalette
        (bucket);

    for (each gameObject in bucket)
    {
        gameObject.FinalUpdate(dt);
    }
}
```

```

        // ...
    }

    void RunGameLoop()
    {
        while (true)
        {
            // ...

            UpdateBucket (kBucketVehiclesAndPlatforms);
            UpdateBucket (kBucketCharacters);
            UpdateBucket (kBucketAttachedObjects);

            // ...

            g_renderingEngine.RenderSceneAndSwapBuffers();
        }
    }

```

In practice, things might be a bit more complex than this. For example, some engine subsystems like the physics engine might not support the concept of buckets, perhaps because they are third-party SDKs or because they cannot be practically updated in a bucketed manner. However, this bucketed update is essentially what we used at Naughty Dog to implement all of the games in the *Uncharted* series as well as *The Last of Us*. So it's a method that has proven to be practical and reasonably efficient.

### 16.6.3.3 Object State Inconsistencies and One-Frame-Off Lag

Let's revisit game object updating, but this time thinking in terms of each object's local notion of time. We said in Section 16.6 that the state of game object  $i$  at time  $t$  can be denoted by a state vector  $\mathbf{S}_i(t)$ . When we update a game object, we are converting its previous state vector  $\mathbf{S}_i(t_1)$  into a new current state vector  $\mathbf{S}_i(t_2)$  (where  $t_2 = t_1 + \Delta t$ ).

In theory, the states of all game objects are updated from time  $t_1$  to time  $t_2$  instantaneously and in parallel, as depicted in Figure 16.15. However, presuming that our game update loop is single-threaded, we actually update the objects one by one—we loop over each game object and call some kind of update function on each one in turn. If we were to stop the program halfway through this update loop, half of our game objects' states would have been updated to  $\mathbf{S}_i(t_2)$ , while the remaining half would still be in their previous states  $\mathbf{S}_i(t_1)$ . This implies that if we were to ask two of our game objects what the current time is during the update loop, they may or may not agree! What's more, depending on where exactly we interrupt the update loop, the objects

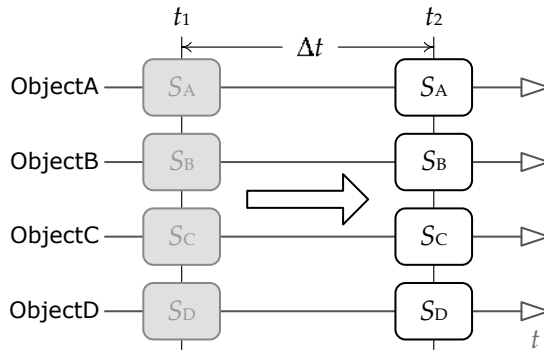


Figure 16.15. In theory, the states of all game objects are updated instantaneously and in parallel during each iteration of the game loop.

may all be in a partially updated state. For example, animation pose blending may have been run, but physics and collision resolution may not yet have been applied. This leads us to the following rule:

The states of all game objects are consistent *before* and *after* the update loop, but they may be inconsistent *during* it.

This is illustrated in Figure 16.16.

The inconsistency of game object states during the update loop is a major source of confusion and bugs, even among professionals within the game in-

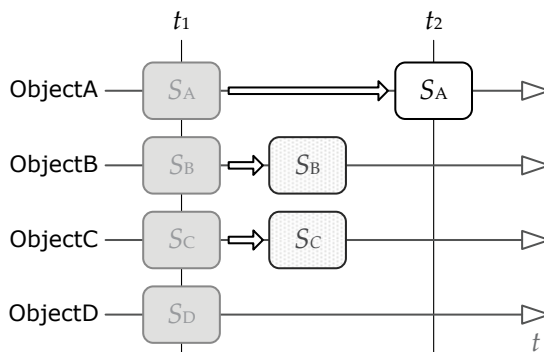


Figure 16.16. In practice, the states of the game objects are updated one by one. This means that at some arbitrary moment during the update loop, some objects will think the current time is  $t_2$  while others think it is still  $t_1$ . Some objects may be only partially updated, so their states will be internally inconsistent. In effect, the state of such an object lies at a point between  $t_1$  and  $t_2$ .

dustry. The problem rears its head most often when game objects query one another for state information during the update loop (which implies that there is a dependency between them). For example, if object B looks at the velocity of object A in order to determine its own velocity at time  $t$ , then the programmer must be clear about whether he or she wants to read the *previous* state of object A,  $S_A(t_1)$ , or the *new* state,  $S_A(t_2)$ . If the new state is needed but object A has not yet been updated, then we have an update order problem that can lead to a class of bugs known as *one-frame-off lags*. In this type of bug, the state of one object lags one frame behind the states of its peers, which manifests itself on-screen as a lack of synchronization between game objects.

#### 16.6.3.4 Object State Caching

As described above, one solution to this problem is to group the game objects into buckets (Section 16.6.3.2). One problem with a simple bucketed update approach is that it imposes somewhat arbitrary limitations on the way in which game objects are permitted to query one another for state information. If a game object A wants the *updated* state vector  $S_B(t_2)$  of another object B, then object B must reside in a *previously updated* bucket. Likewise, if object A wants the *previous* state vector  $S_B(t_1)$  of object B, then object B must reside in a *yet-to-be-updated* bucket. Object A should never ask for the state vector of an object within its own bucket, because, as we stated in the rule above, those state vectors may be only partially updated. Or at best, there's some uncertainty as to whether you're accessing the other object's state at time  $t_1$  or time  $t_2$ .

One way to improve consistency is to arrange for each game object to *cache* its previous state vector  $S_i(t_1)$  while it is calculating its new state vector  $S_i(t_2)$  rather than *overwriting* it in-place during its update. This has two immediate benefits. First, it allows any object to safely query the previous state vector of any other object without regard to update order. Second, it guarantees that a totally consistent state vector ( $S_i(t_1)$ ) will always be available, even during the update of the new state vector. To my knowledge there is no standard terminology for this technique, so I'll call it *state caching* for lack of a better name.

Another benefit of state caching is that we can linearly interpolate between the previous and next states in order to approximate the state of an object at any moment between these two points in time. The Havok physics engine maintains the previous and current state of every rigid body in the simulation for just this purpose.

The downside of state caching is that it consumes twice the memory of the update-in-place approach. It also only solves half the problem, because while

the previous states at time  $t_1$  are fully consistent, the new states at time  $t_2$  still suffer from potential inconsistency. Nonetheless, the technique can be useful when applied judiciously.

This technique is influenced strongly by the design principles of pure *functional programming* (see Section 16.9.2). In a pure functional programming language, all operations are performed by functions with a clear input and output, and no side-effects. All data is considered constant and immutable—rather than mutating an input datum in place, a brand new datum is always produced.

#### 16.6.3.5 Time-Stamping

One easy and low-cost way to improve the consistency of game object states is to time-stamp them. It is then a trivial matter to determine whether a game object's state vector corresponds to its configuration at a previous time or the current time. Any code that queries the state of another game object during the update loop can assert or explicitly check the time stamp to ensure that the proper state information is being obtained.

Time-stamping does not address the inconsistency of states during the update of a bucket. However, we can set a global or static variable to reflect which bucket is currently being updated. Presumably every game object “knows” in which bucket it resides. So we can check the bucket of a queried game object against the currently updating bucket and assert that they are not equal in order to guard against inconsistent state queries.

## 16.7 Applying Concurrency to Game Object Updates

In Chapter 4 we explored *hardware parallelism* and how to take advantage of the explicitly parallel computing hardware that has become the norm in recent gaming hardware. This is done via *concurrent programming* techniques. In Section 8.6, we introduced a number of approaches that allow a game engine to take advantage of the parallel processing resources. In this section, we'll take a look at ways in which concurrency and parallelism can be applied to the problem of updating the states of our game objects.

### 16.7.1 Concurrent Engine Subsystems

Clearly, the most performance-critical parts of our engine—such as rendering, animation, audio and physics—are the ones that will benefit most from parallel processing. So, whether or not our game object model is being updated in a

single thread or across multiple cores, it needs to be able to *interface* with low-level engine systems that are almost certainly multithreaded.

If our engine supports a general-purpose *job system* (see Section 8.6.4), we can use that job system to make our engine subsystems execute concurrently. In this scenario, each subsystem's frame update might be kicked off as a single job each frame. However, it's probably a better idea for each subsystem to kick off multiple jobs to perform its work each frame. For example, an animation system might kick off one job for each object in the game world that requires animation blending. Later in the frame, when the animation system performs its world matrix and skinning matrix computations, it could potentially employ a scatter/gather approach to divide this work across the available cores.

When making our low-level engine systems update concurrently, we need to ensure that their *interfaces* are *thread-safe*. We want to be sure that no external code can get into a data race situation, either in contention with other external clients, or in contention with the inner workings of the subsystem itself. This typically involves using locks on all external calls into each subsystem.

If our job system employs *user-level threads* (coroutines or fibers), then we will need to use spin locks to make our subsystems thread-safe. However, if our subsystems are updated by OS threads, then mutexes are also a viable option for this purpose.

If we can be certain that a particular engine subsystem only ever runs during one particular phase of the game loop, we may be able to employ *lock-not-needed assertions* rather than actually having to lock certain parts of the subsystem. See Section 4.9.7.5 for more information on these useful assertions.

Of course, another option is to attempt to use *lock-free* data structures to implement our engine subsystems critical (shared) data. Lock-free data structures are tricky to implement, and some data structures still have no known lock-free implementation. As such, if you opt for a lock-free approach, it's probably wise to restrict your efforts only to engine subsystems with the most stringent performance requirements.

## 16.7.2 Asynchronous Program Design

When interfacing with concurrent engine subsystems (for example from within a game object update), we must begin thinking *asynchronously*. When a time-consuming operation is to be performed, we should therefore avoid calling a *blocking function*—a function that does its work directly in the context of the calling thread, thereby blocking that thread or job until the work has been completed. Instead, whenever possible, large or expensive jobs should be requested by calling a *non-blocking function*—a function that sends the request

to be executed by another thread, core or processor and then immediately returns control to the calling function. The calling thread or job can proceed with other unrelated work, perhaps including updating other game objects, while we wait for the results of our request. Later in the same frame, or in next frame, we can pick up the results of the request and make use of them.

For example, a game might request that a ray be cast into the world in order to determine whether the player has line-of-sight to an enemy character. In a synchronous design, the ray cast would be done immediately in response to the request, and when the ray casting function returned, the results would be available, as shown below.

```
SomeGameObject::Update()
{
    // ...

    // Cast a ray to see if the player has line of sight
    // to the enemy.
    RayCastResult r = castRay(playerPos, enemyPos);

    // Now process the results...
    if (r.hitSomething() && isEnemy(r.getHitObject()))
    {
        // Player can see the enemy.
        // ...
    }

    // ...
}
```

In an asynchronous design, a ray cast request would be made by calling a function that simply sets up and enqueues a ray cast job, and then returns immediately. The calling thread or job can continue doing other unrelated work while the job is being processed by another CPU or core. Later, once the ray cast job has been completed, the calling thread or job can pick up the results of the ray cast query and process them:

```
SomeGameObject::Update()
{
    // ...

    // Cast a ray to see if the player has line of sight
    // to the enemy.
    RayCastResult r;
    requestRayCast(playerPos, enemyPos, &r);
```

```

// Do other unrelated work while we wait for the
// other CPU to perform the ray cast for us.

// ...

// OK, we can't do any more useful work. Wait for the
// results of our ray cast job. If the job is
// complete, this function will return immediately.
// Otherwise, the main thread will idle until the
// results are ready...
waitForRayCastResults(&r);

// Process results...
if (r.hitSomething() && isEnemy(r.getHitObject()))
{
    // Player can see the enemy.
    // ...
}

// ...
}

```

In many instances, asynchronous code can kick off a request on one frame, and pick up the results on the next. In this case, you may see code that looks like this:

```

RayCastResult r;
bool rayJobPending = false;

SomeGameObject::Update()
{
    // ...

    // Wait for the results of last frame's ray cast job.
    if (rayJobPending)
    {
        waitForRayCastResults(&r);

        // Process results...
        if (r.hitSomething() && isEnemy(r.getHitObject()))
        {
            // Player can see the enemy.
            // ...
        }
    }
}

```



```

    // Cast a new ray for next frame.
    rayJobPending = true;
    requestRayCast(playerPos, enemyPos, &r);

    // Do other work...
    // ...
}

```

### 16.7.2.1 When to Make Asynchronous Requests

One particularly tricky aspect of converting synchronous, unbatched code to use an asynchronous, batched approach is determining *when* during the game loop (a) to kick off the request and (b) to wait for and utilize the results. In doing this, it is often helpful to ask ourselves the following questions:

- *How early can we kick off this request?* The earlier we make the request, the more likely it is to be done when we actually need the results—and this maximizes CPU utilization by helping to ensure that the main thread is never idle waiting for an asynchronous request to complete. So for any given request, we should determine the earliest point during the frame at which we have enough information to kick it off, and kick it there.
- *How long can we wait before we need the results of this request?* Perhaps we can wait until later in the update loop to do the second half of an operation. Perhaps we can tolerate a one-frame lag and use last frame's results to update the object's state this frame. (Some subsystems like AI can tolerate even longer lag times because they update only every few seconds.) In many circumstances, code that uses the results of a request can in fact be deferred until later in the frame, given a little thought, some code refactoring, and possibly some additional caching of intermediate data.

### 16.7.3 Job Dependencies and Degree of Parallelism

In order to make the best use of a parallel computing platform, we'd like to keep all cores busy at all times. Presuming we're using a job system to parallelize our engine, each iteration of our game loop will be comprised of hundreds or even thousands of jobs running concurrently. However, *dependencies* between these jobs can lead to less-than-ideal utilization of the available cores. If job B takes as its input some data that is produced by job A, then job B cannot begin until job A is done. This creates a *dependency* between job B and job A.

The *degree of parallelism* (DOP) of a system, also known as its *degree of concurrency* (DOC), measures the theoretical maximum number of jobs that can

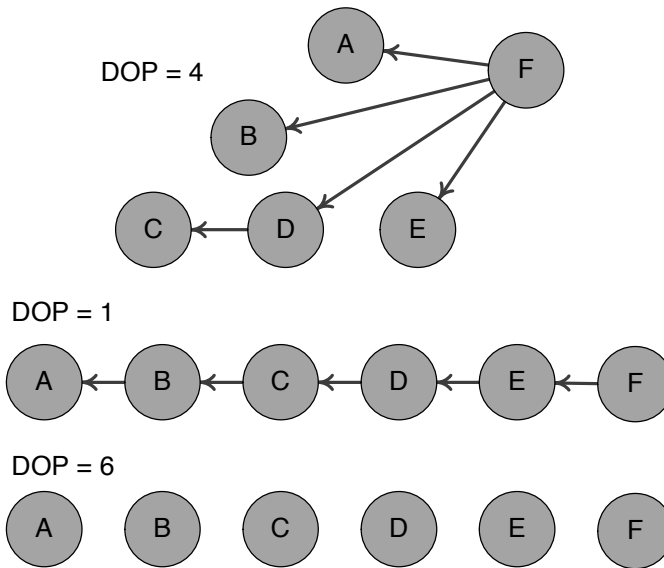


Figure 16.17. Three job dependency trees. The nodes of the tree are jobs, and the arrows represent dependencies between them. The number of leaves in such a tree indicates the system's degree of parallelism (DOP).

be running in parallel at any given moment in time. The degree of parallelism of a group of jobs can be determined by drawing a *dependency graph*. In such a graph, the jobs form the nodes of the tree, and the parent-child relationships represent dependencies. The number of leaves of the tree tells us the degree of parallelism of that collection of jobs. Figure 16.17 illustrates a few examples of job dependency trees and their corresponding degrees of parallelism.

In order to achieve full utilization of CPU resources in an explicitly parallel computer, we would like the DOP of our system to match or exceed the number of available cores. If the DOP of the software exactly equals the number of cores, we get maximum throughput. When the DOP of our system is higher than the number of cores, throughput is reduced because some jobs must run serially, but no core is idle. But when our system's DOP is less than the number of cores, some cores won't have any work to do.

Whenever a job is forced to wait for the jobs on which it depends to complete their tasks, a *synchronization point* (or "sync point" for short) is introduced into the system. Each sync point represents an opportunity for valuable CPU resources to be wasted while a job waits for its dependent jobs to complete their work. This is illustrated in Figure 16.18.

To maximize hardware utilization, we can attempt to increase the DOP of

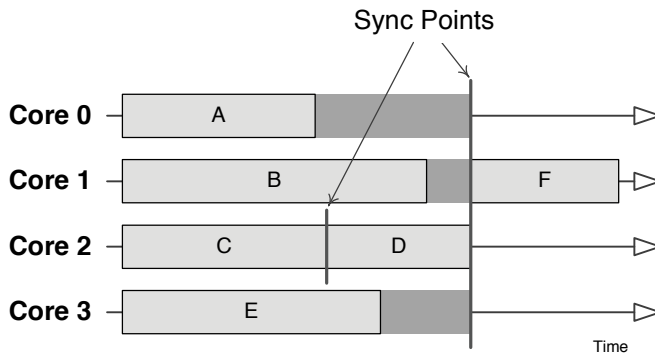


Figure 16.18. A synchronization point is introduced whenever one job is dependent upon the completion of one or more other jobs. Here, job D depends on job C, and job F depends on jobs A, B, D and E.

our system by reducing dependencies between jobs. We could also try to find some other unrelated work to do during idle periods. We can also reduce or eliminate the impact of a sync point by *deferring* it. For example, let's say that job D cannot commence its work until jobs A, B and C are done. If we try to schedule job D before A, B and C are all done, then it will obviously have to sit idle for some time. But if we defer job D so that it runs well after A, B and C are done, then we can be confident that D will never have to wait. This idea is illustrated in Figure 16.19. In his talk entitled "Diving Down the Concurrency Rabbit Hole" ([http://cellperformance.beyond3d.com/articles/public/concurrency\\_rabit\\_hole.pdf](http://cellperformance.beyond3d.com/articles/public/concurrency_rabit_hole.pdf)), Mike Acton says, "The secret of optimized concurrent design is *delay*." This is what he's talking about: Deferring sync points as a means of reducing or eliminating idle time in a concurrent system.

#### 16.7.4 Parallelizing the Game Object Model Itself

Game object models are notoriously difficult to parallelize for a few reasons. Game objects tend to be highly interdependent, and are typically dependent on numerous engine subsystems as well. Inter-object dependencies arise because game objects routinely communicate with one another and query one another's states during their updates. These interactions tend to occur multiple times during the update loop, and the pattern of communication can be unpredictable and highly sensitive to the inputs of the human player and the events that are occurring in the game world. This makes it difficult to process game object updates *concurrently* (using multiple threads or multiple CPU cores).

That being said, it certainly is possible to update a game object model con-

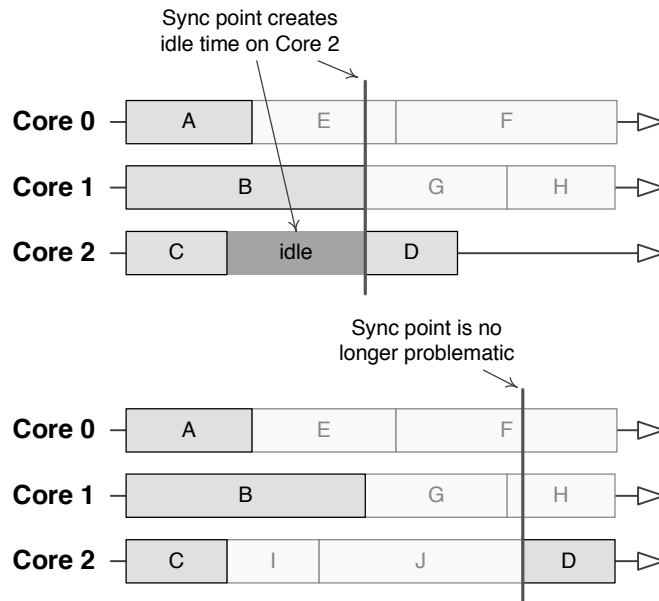


Figure 16.19. Job D depends on jobs A, B and C. Top: If we attempt to schedule job D immediately after job C on Core 2, the core will sit idle waiting for job B to complete. Bottom: If we delay job D's invocation until well after jobs A, B and C have completed, we free up Core 2 to run other jobs, thereby avoiding the idle time that had been caused by the sync point.

currently. As one example, Naughty Dog implemented a concurrent game object model when we ported our engine from PS3 to PS4 for *The Last of Us: Remastered*. In this section, we'll explore some of the problems one encounters when updating game objects concurrently, and we'll have a look at some of the solutions that Naughty Dog uses in our engine. Of course, these techniques are just one possible way to tackle the problem—other engines may use different approaches. And who knows? Perhaps *you* will invent a novel way of solving some of the vexing problems of concurrent game object model updates!

#### 16.7.4.1 Game Object Updates as Jobs

If our game engine has a job system, it will very likely be used to parallelize the various low-level subsystems in our engine, such as animation, audio, collision/physics, rendering, file I/O, and so on. So why not parallelize our game object updates by making them jobs, too? This is the approach used in the Naughty Dog engine. It works, but getting it to work correctly, and efficiently, is a non-trivial undertaking.

We discussed in Section 16.6.3.2 that, due to interdependencies between game objects, we need to control the order in which they update. One way to achieve this is to divide all of the objects in the game into  $N$  buckets, such that the objects in bucket  $B$  depend only on objects in buckets 0 through  $B - 1$ . If we take this approach, we could update all the game objects in each bucket by kicking them off as jobs, and let the job system schedule them across the available cores (and interspersed with whatever other jobs happen to be running at the time). This is roughly the technique employed in the Naughty Dog engine.

```
void UpdateBucket(int iBucket)
{
    job::Declaration aDecl[kMaxObjectsPerBucket];
    const int count = GetNumObjectsInBucket(iBucket);
    for (int jObject = 0; jObject < count; ++jObject)
    {
        job::Declaration& decl = aDecl[jObject];
        decl.m_pEntryPoint = UpdateGameObjectJob;
        decl.m_param = reinterpret_cast<uintptr_t>(
            GetGameObjectInBucket(iBucket, jObject));
    }
    job::KickJobsAndWait(aDecl, count);
}
```

Another way to deal with inter-object dependencies is to make them explicit. In this case, we would presumably have some way of declaring that game object A depends on game objects B, C and D, for example. These dependencies would form a simple *directed graph*. To update the game objects, we'd start by kicking off update jobs for all of the game objects with no dependencies on any other game object (i.e., those nodes of the dependency graph with no outgoing edges). As each job completes, we would walk to each dependent game object, and wait until all of its dependent objects' update jobs are complete. This process would repeat until the entire graph of game objects has been updated.

We can get into trouble with a graph of game object dependencies if the graph contains any *cycles*. A dependency cycle involving two or more game objects indicates that those objects cannot simply be updated in dependency order. To handle cycles, we either need to eliminate them by changing the way the objects interact (turning our graph into a *directed acyclic graph* or DAG), or we need to isolate these "clumps" of cyclically-dependent game objects and update each clump serially on a single core.

#### 16.7.4.2 Asynchronous Game Object Updates

We said in Section 16.7.1 that game object updates are usually done asynchronously. For example, rather than calling a *blocking function* to cast a ray, we would kick off an asynchronous *request* for a ray cast, and the collision subsystem would process this request at some future time during the frame. Later in the frame, or next frame, we would pick up the results of the ray cast, and act on them.

This approach continues to work well when the game objects themselves are also updating concurrently (across multiple cores). However, if our job system is based on *user-level threads* (coroutines or fibers), then *blocking calls* become a viable option, too. This works because a coroutine has the unique property of being able to *yield* (to another coroutine) and then continue where it left off at some later time (when another coroutine yields back to it). In a fiber-based job system (like the one used in the Naughty Dog engine), jobs aren't coroutines *per se*, but they do have this same property: A fiber-based job is able to “go to sleep” midway through its execution, and later to be “woken up” to continue execution where it left off.

Here's an example ray cast implemented using a blocking call, in a coroutine- or fiber-based job system:

```
SomeGameObject::Update()
{
    // ...

    // Cast a ray to see if the player has line of sight
    // to the enemy.
    RayCastResult r = castRayAndWait(playerPos, enemyPos);

    // zzz...

    // Wake up when ray cast result is ready!

    // Now process the results...
    if (r.hitSomething() && isEnemy(r.getHitObject()))
    {
        // Player can see the enemy.
        // ...
    }

    // ...
}
```

Notice that this implementation looks nearly identical to the example of what *not to do* that we presented in Section 16.7.2! Thanks to user-level threads, the

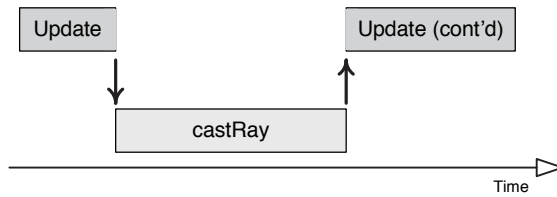


Figure 16.20. If our job system is implemented with user-level threads, a job can be interrupted part-way through its execution in order to perform an asynchronous operation, and resumed once that operation has been completed.

execution of our job can make use of blocking function calls after all! Figure 16.20 illustrates what is happening: The job has effectively been cleaved into two parts—the portion that runs before the blocking ray cast call, and the portion that runs after.

This mechanism is what allows us to implement job system functions like `WaitForCounter()` `KickJobsAndWait()`. These functions *block* their jobs, putting them to sleep and permitting other jobs to execute while they wait for the counter(s) in question to hit zero.

#### 16.7.4.3 Locking During Game Object Updates

Bucketed updates go a long way to solving the problems that arise due to inter-object dependencies. They ensure that the game objects are updated in the correct global order (e.g., the train car updates before the objects sitting on it). And they help with inter-object state queries. An object in bucket  $B$  can safely query the state of objects in buckets  $B - \Delta$ , and those in buckets  $B + \Delta$  (where  $\Delta > 0$ ), because in both cases we know that those objects won't be updating concurrently with the objects in bucket  $B$ . There is still a one-frame-off issue, however: If on frame  $N$ , an object in bucket  $B$  queries an object in bucket  $B - \Delta$ , it will see the state of that object on frame  $N$ ; however, if it queries an object in bucket  $B + \Delta$ , it will see the state of that object as it was last frame (on frame  $N - 1$ ) because it won't have updated yet.

So, with a bucketed updating system, we can safely access game objects in *other* buckets without needing any kind of lock. However, what about when game objects in the *same* bucket need to interact or query one another? Here, we are once again prone to concurrent race conditions, so we can't just do nothing and cross our fingers.

We might be tempted to introduce a single, global lock (mutex or spin lock) into the game object system. Every game object within one particular bucket could acquire this lock, perform its update (possibly interacting with other

game objects in the process), and then release the lock when it's done. This would certainly guarantee that inter-object communication would be free from data races. However, it would also have the highly undesirable effect of *serializing* the updates of all game objects within the bucket, reducing our “concurrent” game object update in effect to a single-threaded update! This occurs because the lock would prevent any two game objects from ever updating concurrently, even when those two objects don't interact in any way.

There are all sorts of other ways to tackle this problem. One approach we tried early on at Naughty Dog was to introduce a global locking system, but to only acquire the lock whenever a game object handle was *dereferenced* within a game object's update function. Game objects in our engine are referenced by handle rather than by raw pointers, to support memory defragmentation. So to get a raw pointer to a game object, one must first dereference its handle. This is a perfect opportunity to detect that one game object intends to interact with another. By only taking locks when interactions are actually likely to happen, we were able to recover some degree of concurrency. However, this locking system was complex, difficult to work with, and still led to inefficient use of the CPU cores during bucket updates.

#### 16.7.4.4 Object Snapshots

Upon analyzing the inter-dependencies between the game objects in a real game engine, we can make an observation: A large majority of the interactions between game objects during their updates are state queries. In other words, game object A reaches out and queries the current state of game objects B, C, and so on. When game object A does this, it really only needs to have access to a *read-only copy* of the states of these other game objects. It needn't interact with the objects themselves (which might or might not be concurrently updating at the moment of such interactions).

Given this state of affairs, it makes sense to arrange for each game object to provide a *snapshot* of its relevant state information—a read-only copy which can be queried, without locks or fear of data races, by any other game object in the system. Snapshots are really just an example of the *state caching* technique we described in Section 16.6.3.4. We used this approach at Naughty Dog for *The Last of Us: Remastered*, *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy*.

Here's how snapshots work in the Naughty Dog engine: At the start of each bucket update, we ask each game object to update its snapshot. These updates never query the state of other game objects, so they can be run concurrently without locks. Once all updates are complete, we kick off concurrent jobs to update the game objects' internal states. These too run concurrently.



Whenever a game object in bucket  $B$  needs to query the state of another object, it now has three options:

1. When querying an object in buckets  $B - \Delta$ , it can query it directly or query the object's snapshot.
2. When querying an object in bucket  $B$ , it queries the snapshot.
3. When querying an object in buckets  $B + \Delta$ , it again can query either the object itself, or its snapshot.

#### 16.7.4.5 Handling Inter-Object Mutation

Snapshots allow us to avoid locks when game objects *read* one another's game state. However, they do nothing to address the data races that can occur when one game object needs to *mutate* the state of another object in the same bucket. To handle these situations, Naughty Dog applied a combination of the following rules of thumb and techniques:

- Minimize inter-object mutation wherever possible.
- Inter-object mutation *between* buckets is safe, but
- inter-object mutation *within* a bucket must be handled carefully...
  - with locks,
  - or by requesting mutations by placing them onto a request queue, rather than applying the mutations immediately. The request queue is itself protected by a lock, and the handling of the requests in the queue is deferred until after the bucket update has completed.

Another option for inter-object mutation within a single bucket is to spawn a job in the *next* bucket whose job it is to synchronize these mutation operations. By deferring the action to a later bucket, we can be certain that the objects in question have completed their updated and will not be updating concurrently with our work. We did this at Naughty Dog to handle synchronized melee moves involving the player and an NPC.

#### 16.7.4.6 Future Improvements

The bucketed updating system we've described in this section isn't perfect by any means. Bucketed updates aren't as efficient as they could be, because each transition between buckets introduces a *sync point* into the game loop. At these sync points, some CPU cores may sit idle while we wait for all game objects in the bucket to complete their updates.

Snapshotting is also an imperfect solution to within-bucket dependency problems, because it only handles read-only queries; locks are still required for inter-object mutations. The snapshots themselves can be expensive to update as well (although one easy-to-apply optimization here is to only generate snapshots for objects on an as-needed basis).

There are lots of other ways to tackle these problems. The best way to discover how to update game objects concurrently is to experiment. Hopefully we've presented some useful ideas in this section that will pave the way forward as you experiment for yourself!

## 16.8 Events and Message-Passing

Games are inherently event-driven. An *event* is anything of interest that happens during gameplay. An explosion going off, the player being sighted by an enemy, a health pack getting picked up—these are all events. Games generally need a way to (a) notify interested game objects when an event occurs and (b) arrange for those objects to respond to interesting events in various ways—we call this *handling* the event. Different types of game objects will respond in different ways to an event. The way in which a particular type of game object responds to an event is a crucial aspect of its behavior, just as important as how the object's state changes over time in the absence of any external inputs. For example, the behavior of the ball in *Pong* is governed in part by its velocity, in part by how it reacts to the event of striking a wall or paddle and bouncing off, and in part by what happens when the ball is missed by one of the players.

### 16.8.1 The Problem with Statically Typed Function Binding

One simple way to notify a game object that an event has occurred is to simply call a method (member function) on the object. For example, when an explosion goes off, we could query the game world for all objects within the explosion's damage radius and then call a virtual function named something like `OnExplosion()` on each one. This is illustrated by the following pseudocode:

```
void Explosion::Update()
{
    // ...

    if (ExplosionJustWentOff())
    {
        GameObjectCollection damagedObjects;
```

```
g_world.QueryObjectsInSphere(GetDamageSphere(),
                             damagedObjects);

for (each object in damagedObjects)
{
    object.OnExplosion(*this);
}

// ...
}
```

The call to `OnExplosion()` is an example of *statically typed late function binding*. Function binding is the process of determining which function implementation to invoke at a particular call location—the implementation is, in effect, bound to the call. Virtual functions, such as our `OnExplosion()` event-handling function, are said to be *late-bound*. This means that the compiler doesn't actually know *which* of the many possible implementations of the function is going to be invoked at compile time—only at runtime, when the type of the target object is known, will the appropriate implementation be invoked. We say that a virtual function call is *statically typed* because the compiler *does* know which implementation to invoke given a particular object type. It knows, for example, that `Tank::OnExplosion()` should be called when the target object is a `Tank` and that `Crate::OnExplosion()` should be called when the object is a `Crate`.

The problem with statically typed function binding is that it introduces a degree of inflexibility into our implementation. For one thing, the virtual `OnExplosion()` function requires all game objects to inherit from a common base class. Moreover, it requires that base class to *declare* the virtual function `OnExplosion()`, even if not all game objects can respond to explosions. In fact, using statically typed virtual functions as event handlers would require our base `GameObject` class to declare virtual functions for *all possible events* in the game! This would make adding new events to the system difficult. It precludes events from being created in a data-driven manner—for example, within the world editing tool. It also provides no mechanism for certain types of objects, or certain individual object instances, to register interest in certain events but not others. Every object in the game, in effect, “knows” about every possible event, even if its response to the event is to do nothing (i.e., to implement an empty, do-nothing event handler function).

What we really need for our event handlers, then, is *dynamically typed late function binding*. Some programming languages support this feature natively (e.g., C#'s delegates). In other languages, the engineers must implement it

manually. There are many ways to approach this problem, but most boil down to taking a data-driven approach. In other words, we encapsulate the notion of a function call in an object and pass that object around at runtime in order to implement a dynamically typed late-bound function call.

### 16.8.2 Encapsulating an Event in an Object

An event is really comprised of two components: its *type* (explosion, friend injured, player spotted, health pack picked up, etc.) and its *arguments*. The arguments provide specifics about the event. (How much damage did the explosion do? Which friend was injured? Where was the player spotted? How much health was in the health pack?) We can encapsulate these two components in an object, as shown by the following rather over-simplified code snippet:

```
struct Event
{
    const U32 MAX_ARGS = 8;

    EventType  m_type;
    U32        m_numArgs;
    EventArg   m_aArgs[MAX_ARGS];
};
```

Some game engines call these things *messages* or *commands* instead of *events*. These names emphasize the idea that informing objects about an event is essentially equivalent to sending a message or command to those objects.

Practically speaking, event objects are usually not quite this simple. We might implement different types of events by deriving them from a root event class, for example. The arguments might be implemented as a linked list or a dynamically allocated array capable of containing arbitrary numbers of arguments, and the arguments might be of various data types.

Encapsulating an event (or message) in an object has many benefits:

- *Single event handling function.* Because the event object encodes its type internally, any number of different event types can be represented by an instance of a single class (or the root class of an inheritance hierarchy). This means that we only need *one* virtual function to handle *all* types of events (e.g., `virtual void OnEvent(Event& event);`).
- *Persistence.* Unlike a function call, whose arguments go out of scope after the function returns, an event object stores both its type and its arguments as data. An event object therefore has persistence. It can be stored in a queue for handling at a later time, copied and broadcast to multiple receivers and so on.

- *Blind event forwarding.* An object can forward an event that it receives to another object without having to “know” anything about the event. For example, if a vehicle receives a Dismount event, it can forward it to all of its passengers, thereby allowing them to dismount the vehicle, even though the vehicle itself knows nothing about dismounting.

This idea of encapsulating an event/message/command in an object is commonplace in many fields of computer science. It is found not only in game engines but in other systems like graphical user interfaces, distributed communication systems and many others. The well-known “Gang of Four” design patterns book [19] calls this the Command design pattern.

### 16.8.3 Event Types

There are many ways to distinguish between different types of events. One simple approach in C or C++ is to define a global enum that maps each event type to a unique integer.

```
enum EventType
{
    EVENT_TYPE_LEVEL_STARTED,
    EVENT_TYPE_PLAYER_SPAWNED,
    EVENT_TYPE_ENEMY_SPOTTED,
    EVENT_TYPE_EXPLOSION,
    EVENT_TYPE_BULLET_HIT,
    // ...
}
```

This approach enjoys the benefits of simplicity and efficiency (since integers are usually extremely fast to read, write and compare). However, it also suffers from two problems. First, knowledge of *all* event types in the entire game is centralized, which can be seen as a form of broken encapsulation (for better or for worse—opinions on this vary). Second, the event types are hard-coded, which means new event types cannot easily be defined in a data-driven manner. Third, enumerators are just indices, so they are order-dependent. If someone accidentally adds a new event type in the middle of the list, the indices of all subsequent event ids change, which can cause problems if event ids are stored in data files. As such, an enumeration-based event typing system works well for small demos and prototypes but does not scale very well at all to real games.

Another way to encode event types is via strings. This approach is totally free-form, and it allows a new event type to be added to the system by merely thinking up a name for it. But it suffers from many problems, including a

strong potential for event name conflicts, the possibility of events not working because of a simple typo, increased memory requirements for the strings themselves, and the relatively high cost of comparing strings relative to that of comparing integers. Hashed string ids can be used instead of raw strings to eliminate the performance problems and increased memory requirements, but they do nothing to address event name conflicts or typos. Nonetheless, the extreme flexibility and data-driven nature of a string- or string-id-based event system is considered worth the risks by many game teams, including Naughty Dog.

Tools can be implemented to help avoid some of the risks involved in using strings to identify events. For example, a central database of all event type names could be maintained. A user interface could be provided to permit new event types to be added to the database. Naming conflicts could be automatically detected when a new event is added, and the user could be disallowed from adding duplicate event types. When selecting a preexisting event, the tool could provide a sorted list in a drop-down combo box rather than requiring the user to remember the name and type it manually. The event database could also store metadata about each type of event, including documentation about its purpose and proper usage and information about the number and types of arguments it supports. This approach can work really well, but we should not forget to account for the costs of setting up and maintaining such a system, as they are not insignificant.

#### 16.8.4 Event Arguments

The arguments of an event usually act like the argument list of a function, providing information about the event that might be useful to the receiver. Event arguments can be implemented in all sorts of ways.

We might derive a new type of `Event` class for each unique type of event. The arguments can then be hard-coded as data members of the class. For example:

```
class ExplosionEvent : public Event
{
    Point    m_center;
    float    m_damage;
    float    m_radius;
};
```

Another approach is to store the event's arguments as a collection of *variants*. A variant is a data object that is capable of holding more than one type of data. It usually stores information about the data type that is currently being stored, as well as the data itself. In an event system, we might want our argu-

ments to be integers, floating-point values, Booleans or hashed string ids. So in C or C++, we could define a variant class that looks something like this:

```
struct Variant
{
    enum Type
    {
        TYPE_INTEGER,
        TYPE_FLOAT,
        TYPE_BOOL,
        TYPE_STRING_ID,
        TYPE_COUNT // number of unique types
    };

    Type      m_type;

    union
    {
        I32      m_asInteger;
        F32      m_asFloat;
        bool     m_asBool;
        U32      m_asStringId;
    };
};
```

The collection of variants within an `Event` might be implemented as an array with a small, fixed maximum size (say 4, 8 or 16 elements). This imposes an arbitrary limit on the number of arguments that can be passed with an event, but it also side-steps the problems of dynamically allocating memory for each event’s argument payload, which can be a big benefit, especially in memory-constrained console games.

The collection of variants might be implemented as a dynamically sized data structure, like a dynamically sized array (like `std::vector`) or a linked list (like `std::list`). This provides a great deal of additional flexibility over a fixed size design, but it incurs the cost of dynamic memory allocation. A pool allocator could be used to great effect here, presuming that each `Variant` is the same size.

#### 16.8.4.1 Event Arguments as Key-Value Pairs

A fundamental problem with an *indexed* collection of event arguments is order dependency. Both the sender and the receiver of an event must “know” that the arguments are listed in a specific order. This can lead to confusion and bugs. For example, a required argument might be accidentally omitted or an extra one added.

This problem can be avoided by implementing event arguments as key-value pairs. Each argument is uniquely identified by its key, so the arguments can appear in any order, and optional arguments can be omitted altogether. The argument collection might be implemented as a closed or open hash table, with the keys used to hash into the table, or it might be an array, linked list or binary search tree of key-value pairs. These ideas are illustrated in Table 16.1. The possibilities are numerous, and the specific choice of implementation is largely unimportant as long as the game's particular requirements have been effectively and efficiently met.

### 16.8.5 Event Handlers

When an event, message or command is received by a game object, it needs to respond to the event in some way. This is known as *handling* the event, and it is usually implemented by a function or a snippet of script code called an *event handler*. (We'll have more to learn about game scripting later on.)

Often an event handler is a single native virtual function or script function that is capable of handling all types of events (e.g., virtual void `OnEvent(Event& event)`). In this case, the function usually contains some kind of switch statement or cascaded if/else-if clause to handle the various types of events that might be received. A typical event handler function might look something like this:

```
virtual void SomeObject::OnEvent(Event& event)
{
    switch (event.GetType())
    {
        case SID("EVENT_ATTACK") :
            RespondToAttack(event.GetAttackInfo());
            break;

        case SID("EVENT_HEALTH_PACK") :
```

| Key       | Type     | Value       |
|-----------|----------|-------------|
| "event"   | stringid | "explosion" |
| "radius"  | float    | 10.3        |
| "damage"  | int      | 25          |
| "grenade" | bool     | true        |

Table 16.1. The arguments of an event object can be implemented as a collection of key-value pairs. The keys prevent order-dependency problems because each event argument is uniquely identified by its key.



```

        AddHealth(event.GetHealthPack().GetHealth());
        break;

    // ...

    default:
        // Unrecognized event.
        break;
    }
}

```

Alternatively, we might implement a suite of handler functions, one for each type of event (e.g., `OnThis()`, `OnThat()`, ...). However, as we discussed above, a proliferation of event handler functions can be problematic.

A Windows GUI toolkit called Microsoft Foundation Classes (MFC) was well-known for its *message maps*—a system that permitted any Windows message to be bound at runtime to an arbitrary non-virtual or virtual function. This avoided the need to declare handlers for all possible Windows messages in a single root class, while at the same time avoiding the big switch statement that is commonplace in non-MFC Windows message-handling functions. But such a system is probably not worth the hassle—a switch statement works really well and is simple and clear.

### 16.8.6 Unpacking an Event's Arguments

The example above glosses over one important detail—namely, how to extract data from the event's argument list in a type-safe manner. For example, `event.GetHealthPack()` presumably returns a `HealthPack` game object, which in turn we presume provides a member function called `GetHealth()`. This implies that the root `Event` class “knows” about health packs (as well as, by extension, every other type of event argument in the game). This is probably an impractical design. In a real engine, there might be derived `Event` classes that provide convenient data-access APIs such as `GetHealthPack()`. Or the event handler might have to unpack the data manually and cast them to the appropriate types. This latter approach raises type safety concerns, although practically speaking it usually isn't a huge problem because the type of the event is always known when the arguments are unpacked.

### 16.8.7 Chains of Responsibility

Game objects are almost always dependent upon one another in various ways. For example, game objects usually reside in a transformation hierarchy, which allows an object to rest on another object or be held in the hand of a character. Game objects might also be made up of multiple interacting components,

leading to a star topology or a loosely connected “cloud” of component objects. A sports game might maintain a list of all the characters on each team. In general, we can envision the interrelationships between game objects as one or more *relationship graphs* (remembering that a list and a tree are just special cases of a graph). A few examples of relationship graphs are shown in Figure 16.21.

It often makes sense to be able to pass events from one object to the next within these relationship graphs. For example, when a vehicle receives an event, it may be convenient to pass the event to all of the passengers riding on the vehicle, and those passengers may wish to forward the event to the objects in their inventories. When a multicomponent game object receives an event, it may be necessary to pass the event to all of the components so that they all get a crack at handling it. Or when an event is received by a character in a sports game, we might want to pass it on to all of his or her teammates as well.

The technique of forwarding events within a graph of objects is a common design pattern in object-oriented, event-driven programming, sometimes referred to as *Chain of Responsibility* [19]. Usually, the order in which the event is passed around the system is predetermined by the engineers. The event is passed to the first object in the chain, and the event handler returns a Boolean or an enumerated code indicating whether or not it recognized and handled the event. If the event is consumed by a receiver, the process of event forwarding stops; otherwise, the event is forwarded on to the next receiver in the chain. An event handler that supports Chain of Responsibility style event forwarding might look something like this:

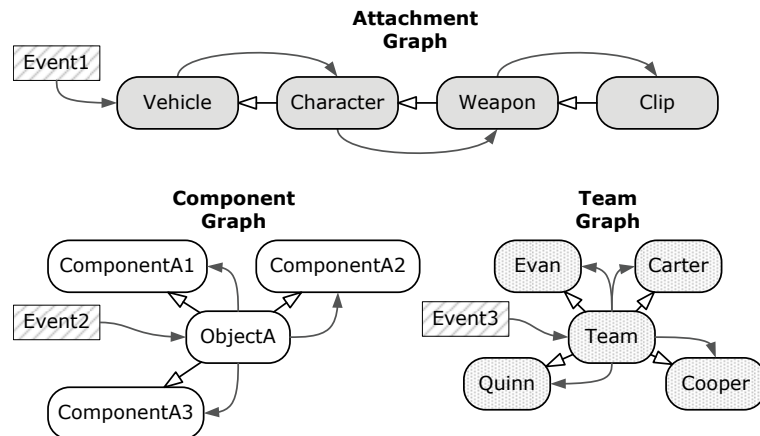


Figure 16.21. Game objects are interrelated in various ways, and we can draw graphs depicting these relationships. Any such graph might serve as a distribution channel for events.

```

virtual bool SomeObject::OnEvent(Event& event)
{
    // Call the base class' handler first.
    if (BaseClass::OnEvent(event))
    {
        return true;
    }

    // Now try to handle the event myself.
    switch (event.GetType())
    {
    case SID("EVENT_ATTACK") :
        RespondToAttack(event.GetAttackInfo());
        return false; // OK to forward this event to others.

    case SID("EVENT_HEALTH_PACK") :
        AddHealth(event.GetHealthPack().GetHealth());
        return true; // I consumed the event; don't forward.

    // ...

    default:
        return false; // I didn't recognize this event.
    }
}

```

When a derived class overrides an event handler, it can be appropriate to call the base class's implementation as well if the class is augmenting but not replacing the base class's response. In other situations, the derived class might be entirely replacing the response of the base class, in which case the base class's handler should not be called. This is another kind of responsibility chain.

Event forwarding has other applications as well. For example, we might want to multicast an event to all objects within a radius of influence (for an explosion, for example). To implement this, we can leverage our game world's object query mechanism to find all objects within the relevant sphere and then forward the event to all of the returned objects.

### 16.8.8 Registering Interest in Events

It's reasonably safe to say that most objects in a game do not need to respond to every possible event. Most types of game objects have a relatively small set of events in which they are "interested." This can lead to inefficiencies when multicasting or broadcasting events, because we need to iterate over a group

of objects and call each one's event handler, even if the object is not interested in that particular kind of event.

One way to overcome this inefficiency is to permit game objects to *register interest* in particular kinds of events. For example, we could maintain one linked list of interested game objects for each distinct type of event, or each game object could maintain a bit array, in which the setting of each bit corresponds to whether or not the object is interested in a particular type of event. By doing this, we can avoid calling the event handlers of any objects that do not care about the event.

Even better, we might be able to restrict our original game object query to include only those objects that are interested in the event we wish to multicast. For example, when an explosion goes off, we can ask the collision system for all objects that are within the damage radius *and* that can respond to Explosion events. This can save time overall, because we avoid iterating over objects that we know aren't interested in the event we're multicasting. Whether or not such an approach will produce a net gain depends on how the query mechanism is implemented and the relative costs of filtering the objects during the query versus filtering them during the multicast iteration.

### 16.8.9 To Queue or Not to Queue

Most game engines provide a mechanism for handling events immediately when they are sent. In addition to this, some engines also permit events to be queued for handling at an arbitrary future time. Event queuing has some attractive benefits, but it also increases the complexity of the event system significantly and poses some unique problems. We'll investigate the pros and cons of event queuing in the following sections and learn how such systems are implemented in the process.

#### 16.8.9.1 Some Benefits of Event Queuing

The following sections outline some of the benefits of event queuing.

##### *Control over When Events Are Handled*

We have seen that we must be careful to update engine subsystems and game objects in a specific order to ensure correct behavior and maximize runtime performance. In the same sense, certain kinds of events may be highly sensitive to exactly when within the game loop they are handled. If all events are handled immediately upon being sent, the event handler functions end up being called in unpredictable and difficult-to-control ways throughout the course of the game loop. By deferring events via an event queue, the engi-

neers can take steps to ensure that events are only handled when it is safe and appropriate to do so.

### *Ability to Post Events into the Future*

When an event is sent, the sender can usually specify a delivery time—for example, we might want the event to be handled later in the same frame, next frame or some number of seconds after it was sent. This feature amounts to an ability to post events into the future, and it has all sorts of interesting uses. We can implement a simple alarm clock by posting an event into the future. A periodic task, such as blinking a light every two seconds, can be executed by posting an event whose handler performs the periodic task and then posts a new event of the same type one time period into the future.

To implement the ability to post events into the future, each event is stamped with a desired delivery time prior to being queued. An event is only handled when the current game clock matches or exceeds its delivery time. An easy way to make this work is to sort the events in the queue in order of increasing delivery time. Each frame, the first event on the queue can be inspected and its delivery time checked. If the delivery time is in the future, we abort immediately because we know that all subsequent events are also in the future. But if we see an event whose delivery time is now or in the past, we extract it from the queue and handle it. This continues until an event is found whose delivery time is in the future. The following pseudocode illustrates this process:

```
// This function is called at least once per frame. Its
// job is to dispatch all events whose delivery time is
// now or in the past.

void EventQueue::DispatchEvents(F32 currentTime)
{
    // Look at, but don't remove, the next event on the
    // queue.
    Event* pEvent = PeekNextEvent();

    while (pEvent
        && pEvent->GetDeliveryTime() <= currentTime)
    {
        // Remove the event from the queue.
        RemoveNextEvent();

        // Dispatch it to its receiver's event handler.
        pEvent->Dispatch();

        // Peek at the next event on the queue (again
```

```

        // without removing it).
        pEvent = PeekNextEvent();
    }
}

```

### *Event Prioritization*

Even if our events are sorted by delivery time in the event queue, the order of delivery is still ambiguous when two or more events have exactly the same delivery time. This can happen more often than you might think, because it is quite common for events' delivery times to be quantized to an integral number of frames. For example, if two senders request that events be dispatched "this frame," "next frame" or "in seven frames from now," then those events will have identical delivery times.

One way to resolve these ambiguities is to assign *priorities* to events. Whenever two events have the same time stamp, the one with higher priority should always be serviced first. This is easily accomplished by first sorting the event queue by increasing delivery times and then sorting each group of events with identical delivery times in order of decreasing priority.

We could allow up to four billion unique priority levels by encoding our priorities in a raw, unsigned 32-bit integer, or we could limit ourselves to only two or three unique priority levels (e.g., low, medium and high). In every game engine, there exists some minimum number of priority levels that will resolve all real ambiguities in the system. It's usually best to aim as close to this minimum as possible. With a very large number of priority levels, it can become a small nightmare to figure out which event will be handled first in any given situation. However, the needs of every game's event system are different, and your mileage may vary.

#### 16.8.9.2 Some Problems with Event Queuing

##### *Increased Event System Complexity*

In order to implement a queued event system, we need more code, additional data structures and more complex algorithms than would be necessary to implement an immediate event system. Increased complexity usually translates into longer development times and a higher cost to maintain and evolve the system during development of the game.

##### *Deep-Copying Events and Their Arguments*

With an immediate event handling approach, the data in an event's arguments need only persist for the duration of the event handling function (and any

functions it may call). This means that the event and its argument data can reside literally anywhere in memory, including on the call stack. For example, we could write a function that looks something like this:

```
void SendExplosionEventToObject(GameObject& receiver)
{
    // Allocate event args on the call stack.
    Point centerPoint(-2.0f, 31.5f, 10.0f);
    F32    damage = 5.0f;
    F32    radius = 2.0f;

    // Allocate the event on the call stack.
    Event event("Explosion");
    event.SetArgFloat("Damage", damage);
    event.SetArgPoint("Center", &centerPoint);
    event.SetArgFloat("Radius", radius);

    // Send the event, which causes the receiver's event
    // handler to be called immediately, as shown below.
    event.Send(receiver);
    //{
    //    receiver.OnEvent(event);
    //}
}
```

When an event is queued, its arguments must persist beyond the scope of the sending function. This implies that we must copy the entire event object prior to storing the event in the queue. We must perform a *deep-copy*, meaning that we copy not only the event object itself but its entire argument payload as well, including any data to which it may be pointing. Deep-copying the event ensures that there are no dangling references to data that exist only in the sending function's scope, and it permits the event to be stored indefinitely. The example event-sending function shown above still looks basically the same when using a queued event system, but as you can see in the italicized code below, the implementation of the `Event::Queue()` function is a bit more complex than its `Send()` counterpart:

```
void SendExplosionEventToObject(GameObject& receiver)
{
    // We can still allocate event args on the call
    // stack.
    Point centerPoint(-2.0f, 31.5f, 10.0f);
    F32    damage = 5.0f;
    F32    radius = 2.0f;
```

```

// Still OK to allocate the event on the call stack
Event event("Explosion
event.SetArgFloat("Damage", damage);
event.SetArgPoint("Center", &centerPoint);
event.SetArgFloat("Radius", radius);

// This stores the event in the receiver's queue for
// handling at a future time. Note how the event
// must be deep-copied prior to being enqueued, since
// the original event resides on the call stack and
// will go out of scope when this function returns.
event.Queue(receiver);
//{
//    Event* pEventCopy = DeepCopy(event);
//    receiver.EnqueueEvent(pEventCopy);
//}
}

```

### *Dynamic Memory Allocation for Queued Events*

Deep-copying of event objects implies a need for dynamic memory allocation, and as we've already noted many times, dynamic allocation is undesirable in a game engine due to its potential cost and its tendency to fragment memory. Nonetheless, if we want to queue events, we'll need to dynamically allocate memory for them.

As with all dynamic allocation in a game engine, it's best if we can select a fast and fragmentation-free allocator. We might be able to use a *pool allocator*, but this will only work if all of our event objects are the same size and if their argument lists are comprised of data elements that are themselves all the same size. This may well be the case—for example, the arguments might each be a *Variant*, as described above. If our event objects and/or their arguments can vary in size, a *small memory allocator* might be applicable. (Recall that a small memory allocator maintains multiple pools, one for each of a few pre-determined small allocation sizes.) When designing a queued event system, always be careful to take dynamic allocation requirements into account.

Other designs are possible, of course. For example, at Naughty Dog we allocate queued events as relocatable memory blocks. See Section 6.2.2.2 for more information on relocatable memory.

### *Debugging Difficulties*

With queued events, the event handler is not called directly by the sender of that event. So, unlike in immediate event handling, the call stack does not tell us where the event came from. We cannot walk up the call stack in the debugger to inspect the state of the sender or the circumstances under which the event was sent. This can make debugging deferred events a bit tricky, and



things get even more difficult when events are forwarded from one object to another.

Some engines store debugging information that forms a paper trail of the event's travels throughout the system, but no matter how you slice it, event debugging is usually much easier in the absence of queuing.

Event queuing also leads to interesting and hard-to-track-down *race condition* bugs. We may need to pepper multiple event dispatches throughout our game loop, to ensure that events are delivered without incurring unwanted one-frame delays yet still ensuring that game objects are updated in the proper order during the frame. For example, during the animation update, we might detect that a particular animation has run to completion. This might cause an event to be sent whose handler wants to play a new animation. Clearly, we want to avoid a one-frame delay between the end of the first animation and the start of the next. To make this work, we need to update animation clocks first (so that the end of the animation can be detected and the event sent); then we should dispatch events (so that the event handler has a chance to request a new animation), and finally we can start animation blending (so that the first frame of the new animation can be processed and displayed). This is illustrated in the code snippet below:

```
while (true) // main game loop
{
    // ...

    // Update animation clocks. This may detect the end
    // of a clip, and cause EndOfAnimation events to
    // be sent.
    g_animationEngine.UpdateLocalClocks(dt);

    // Next, dispatch events. This allows an
    // EndOfAnimation event handler to start up a new
    // animation this frame if desired.
    g_eventSystem.DispatchEvents();

    // Finally, start blending all currently playing
    // animations (including any new clips started
    // earlier this frame).
    g_animationEngine.StartAnimationBlending();

    // ...
}
```

### 16.8.10 Some Problems with Immediate Event Sending

Not queuing events also has its share of issues. For example, immediate event handling can lead to extremely deep call stacks. Object A might send object B an event, and in its event handler, B might send another event, which might send another event, and another and so on. In a game engine that supports immediate event handling, it's not uncommon to see a call stack that looks something like this:

```
...
ShoulderAngel::OnEvent()
Event::Send()
Characer::OnEvent()
Event::Send()
Car::OnEvent()
Event::Send()
HandleSoundEffect()
AnimationEngine::PlayAnimation()
Event::Send()
Character::OnEvent()
Event::Send()
Character::OnEvent()
Event::Send()
Character::OnEvent()
Event::Send()
Car::OnEvent()
Event::Send()
Car::OnEvent()
Event::Send()
Car::Update()
GameWorld::UpdateObjectsInBucket()
Engine::GameLoop()
main()
```

A deep call stack like this can exhaust available stack space in extreme cases (especially if we have an infinite loop of event sending), but the real crux of the problem here is that every event handler function must be written to be fully *re-entrant*. This means that the event handler can be called recursively without any ill side-effects. As a contrived example, imagine a function that increments the value of a global variable. If the global is supposed to be incremented only once per frame, then this function is *not* re-entrant, because multiple recursive calls to the function will increment the variable multiple times.

### 16.8.11 Data-Driven Event/Message-Passing Systems

Event systems give the game programmer a great deal of flexibility over and above what can be accomplished with the statically typed function calling mechanisms provided by languages like C and C++. However, we can do better. In our discussions thus far, the logic for sending and receiving events is still hard-coded and therefore under the exclusive control of the engineers. If we could make our event system data-driven, we could extend its power into the hands of our game designers.

There are many ways to make an event system data-driven. Starting with the extreme of an entirely hard-coded (non-data-driven) event system, we could imagine providing some simple data-driven configurability. For example, designers might be allowed to configure how individual objects, or entire classes of object, respond to certain events. In the world editor, we can imagine selecting an object and then bringing up a scrolling list of all possible events that it might receive. For each one, the designer could use drop-down combo boxes and check boxes to control if, and how, the object responds, by selecting from a set of hard-coded, predefined choices. For example, given the event “PlayerSpotted,” AI-controlled characters might be configured to do one of the following actions: run away, attack or ignore the event altogether. The event systems of some real commercial game engines are implemented in essentially this way.

At the other end of the gamut, our engine might provide the game designers with a simple scripting language (a topic we’ll explore in detail in Section 16.9). In this case, the designer can literally write code that defines how a particular kind of game object will respond to a particular kind of event. In a scripted model, the designers are really just programmers (working with a somewhat less powerful but also easier-to-use and hopefully less error-prone language than the engineers), so anything is possible. Designers might define new types of events, send events and receive and handle events in arbitrary ways. This is what we do at Naughty Dog.

The problem with a simple, configurable event system is that it can severely limit what the game designers are capable of doing on their own, without the help of a programmer. On the other hand, a fully scripted solution has its own share of problems: Many game designers are not professional software engineers by training, so some designers find learning and using a scripting language a daunting task. Designers are also probably more prone to introducing bugs into the game than their engineer counterparts, unless they have practiced scripting or programming for some time. This can lead to some nasty surprises during alpha.

As a result, some game engines aim for a middle ground. They employ sophisticated graphical user interfaces to provide a great deal of flexibility without going so far as to provide users with a full-fledged, free-form scripting language. One approach is to provide a flowchart-style graphical programming language. The idea behind such a system is to provide the user with a limited and controlled set of atomic operations from which to choose but with plenty of freedom to wire them up in arbitrary ways. For example, in response to an event like “PlayerSpotted,” the designer could wire up a flowchart that causes a character to retreat to the nearest cover point, play an animation, wait 5 seconds, and then attack. A GUI can also provide error-checking and validation to help ensure that bugs aren’t inadvertently introduced. Unreal’s Blueprints is an example of such a system—see the following section for more details.

#### 16.8.11.1 Data Pathway Communication Systems

One of the problems with converting a function-call-like event system into a data-driven system is that different types of events tend to be incompatible. For example, let’s imagine a game in which the player has an electromagnetic pulse gun. This pulse causes lights and electronic devices to turn off, scares small animals and produces a shock wave that causes any nearby plants to sway. Each of these game object types may already have an event response that performs the desired behavior. A small animal might respond to the “Scare” event by scurrying away. An electronic device might respond to the “TurnOff” event by turning itself off. And plants might have an event handler for a “Wind” event that causes them to sway. The problem is that our EMP gun is not compatible with any of these objects’ event handlers. As a result, we end up having to implement a new event type, perhaps called “EMP,” and then write custom event handlers for every type of game object in order to respond to it.

One solution to this problem is to take the event type out of the equation and to think solely in terms of *sending streams of data* from one game object to another. In such a system, every game object has one or more *input ports* to which a data stream can be connected, and one or more *output ports* through which data can be sent to other objects. Provided we have some way of wiring these ports together, such as a graphical user interface in which ports can be connected to each other via rubber-band lines, then we can construct arbitrarily complex behaviors. Continuing our example, the EMP gun would have an output port, perhaps named “Fire,” that sends a Boolean signal. Most of the time, the port produces the value 0 (false), but when the gun is fired, it sends a brief (one-frame) pulse of the value 1 (true). The other game objects in

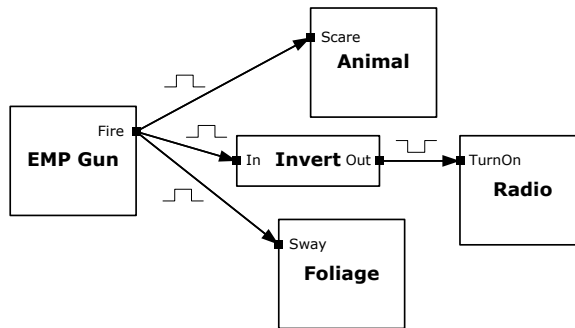


Figure 16.22. The EMP gun produces a 1 at its “Fire” output when fired. This can be connected to any input port that expects a Boolean value, in order to trigger the behavior associated with that input.

the world have binary input ports that trigger various responses. The animals might have a “Scare” input, the electronic devices a “TurnOn” input and the foliage objects a “Sway” input. If we connect the EMP gun’s “Fire” output port to the input ports of these game objects, we can cause the gun to trigger the desired behaviors. (Note that we’d have to pipe the gun’s “Fire” output through a node that *inverts* its input, prior to connecting it to the “TurnOn” input of the electronic devices. This is because we want them to turn *off* when the gun is firing.) The wiring diagram for this example is shown in Figure 16.22.

Programmers decide what kinds of port(s) each type of game object will have. Designers using the GUI can then wire these ports together in arbitrary ways in order to construct arbitrary behaviors in the game. The programmers also provide various other kinds of nodes for use within the graph, such as a node that inverts its input, a node that produces a sine wave or a node that outputs the current game time in seconds.

Various types of data might be sent along a data pathway. Some ports might produce or expect Boolean data, while others might be coded to produce or expect data in the form of a unit float. Still others might operate on 3D vectors, colors, integers and so on. It’s important in such a system to ensure that connections are only made between ports with compatible data types, or we must provide some mechanism for automatically converting data types when two differently typed ports are connected together. For example, connecting a unit-float output to a Boolean input might automatically cause any value less than 0.5 to be converted to false, and any value greater than or equal to 0.5 to be converted to true. This is the essence of GUI-based event systems like Unreal Engine 4’s Blueprints. A screenshot of Blueprints is shown in Figure 16.23.

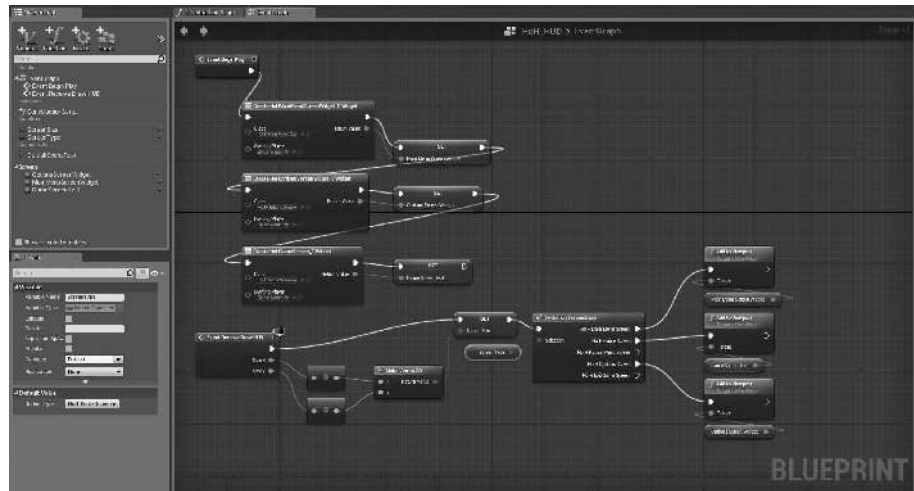


Figure 16.23. Unreal Engine 4's Blueprints.

### 16.8.11.2 Some Pros and Cons of GUI-Based Programming

The benefits of a graphical user interface over a straightforward, text-file-based scripting language are probably pretty obvious: ease of use, a gradual learning curve with the potential for in-tool help and tool tips to guide the user, and plenty of error-checking. The downsides of a flowchart style GUI include the high cost to develop, debug, and maintain such a system, the additional complexity, which can lead to annoying or sometimes schedule-killing bugs, and the fact that designers are sometimes limited in what they can do with the tool. A text-file-based programming language has some distinct advantages over a GUI-based programming system, including its relative simplicity (meaning that it is much less prone to bugs), the ability to easily search and replace within the source code, and the freedom of each user to choose the text editor with which they are most comfortable.

## 16.9 Scripting

A *scripting language* can be defined as a programming language whose primary purpose is to permit users to control and customize the behavior of a software application. For example, the Visual Basic language can be used to customize the behavior of Microsoft Excel; both MEL language and Python language can be used to customize the behavior of Maya. In the context of game engines, a

*scripting language* is a high-level, relatively easy-to-use programming language that provides its users with convenient access to most of the commonly used features of the engine. As such, a scripting language can be used by programmers and non-programmers alike to develop a new game or to customize—or “mod”—an existing game.

### 16.9.1 Runtime versus Data Definition

We should be careful to make an important distinction here. Game scripting languages generally come in two flavors:

- *Data-definition languages.* The primary purpose of a data-definition language is to permit users to create and populate data structures that are later consumed by the engine. Such languages are often declarative (see below) and are either executed or parsed offline or at runtime when the data is loaded into memory.
- *Runtime scripting language.* Runtime scripting languages are intended to be executed within the context of the engine at runtime. These languages are usually used to extend or customize the hard-coded functionality of the engine’s game object model and/or other engine systems.

In this section, we’ll focus primarily on using a runtime scripting language for the purpose of implementing gameplay features by extending and customizing the game’s object model.

### 16.9.2 Programming Language Characteristics

In our discussion of scripting languages, it will be helpful for us all to be on the same page with regard to programming language terminology. There are all sorts of programming languages out there, but they can be classified approximately according to a relatively small number of criteria. Let’s take a brief look at these criteria:

- *Interpreted versus compiled languages.* The source code of a *compiled* language is translated by a program called a compiler into machine code, which can be executed directly by the CPU. In contrast, the source code of an *interpreted* language is either parsed directly at runtime or is pre-compiled into platform-independent *byte code*, which is then executed by a *virtual machine* (VM) at runtime. A virtual machine acts like an emulation of an imaginary CPU, and byte code acts like a list of machine

language instructions that are consumed by this virtual CPU. The benefit of a virtual machine is that it can be quite easily ported to almost any hardware platform and embedded within a host application like a game engine. The biggest cost we pay for this flexibility is execution speed—a virtual machine usually executes its byte code instructions much more slowly than the native CPU executes its machine language instructions.

- *Imperative languages.* In an imperative language, a program is described by a sequence of instructions, each of which performs an operation and/or changes the state of data in memory. C and C++ are imperative languages.
- *Declarative languages.* A declarative language describes *what* is to be done but does not specify exactly *how* the result should be obtained. That decision is left up to the people implementing the language. Prolog is an example of a declarative language. Mark-up languages like HTML and TeX can also be classified as declarative languages.
- *Functional languages.* Functional languages, which are technically a subset of declarative languages, aim to avoid state altogether. In a functional language, programs are defined by a collection of functions. Each function produces its results with no side-effects (i.e., it causes no observable changes to the system, other than to produce its output data). A program is constructed by passing input data from one function to the next until the final desired result has been generated. These languages tend to be well-suited to implementing data processing pipelines. They also offer distinct advantages when implementing multithreaded applications, because with no mutable state, a functional language requires no mutex locking. OCaml, Haskell and F# are examples of functional languages.
- *Procedural versus object-oriented languages.* In a procedural language, the primary atom of program construction is the *procedure* (or *function*). These procedures and functions perform operations, calculate results and/or change the state of various data structures in memory. In contrast, an object-oriented language's primary unit of program construction is the *class*, a data structure that is tightly coupled with a set of procedures/functions that "know" how to manage and manipulate the data within that data structure.
- *Reflective languages.* In a reflective language, information about the data types, data member layouts, functions and hierarchical class relationships in the system is available for inspection at runtime. In a non-reflective language, the majority of this meta-information is known only



at compile time; only a very limited amount of it is exposed to the run-time code. C# is an example of a reflective language, while C and C++ are examples of non-reflective languages.

#### 16.9.2.1 Typical Characteristics of Game Scripting Languages

The characteristics that set a game *scripting* language apart from its *native* programming language brethren include:

- *Interpreted.* Most game scripting languages are interpreted by a virtual machine, not compiled. This choice is made in the interest of flexibility, portability and rapid iteration (see below). When code is represented as platform-independent byte code, it can easily be treated like data by the engine. It can be loaded into memory just like any other asset rather than requiring help from the operating system (as is necessary with a DLL on a PC platform or a PRX on the PlayStation 3, for example). Because the code is executed by a virtual machine rather than directly by the CPU, the game engine is afforded a great deal of flexibility regarding how and when script code will be run.
- *Lightweight.* Most game scripting languages have been designed for use in an embedded system. As such, their virtual machines tend to be simple, and their memory footprints tend to be quite small.
- *Support for rapid iteration.* Whenever native code is changed, the program must be recompiled and relinked, and the game must be shut down and rerun in order to see the effects of the changes (unless your development environment supports some form of edit-and-continue). On the other hand, when script code is changed, the effects of the changes can usually be seen very rapidly. Some game engines permit script code to be reloaded on the fly, without shutting down the game at all. Others require the game to be shut down and rerun. But either way, the turnaround time between making a change and seeing its effects in-game is usually much faster than when making changes to the native language source code.
- *Convenience and ease of use.* Scripting languages are often customized to suit the needs of a particular game. Features can be provided that make common tasks simple, intuitive and less error-prone. For example, a game scripting language might provide functions or custom syntax for finding game objects by name, sending and handling events, pausing or manipulating the passage of time, waiting for a specified amount of time to pass, implementing finite state machines, exposing tweakable param-

eters to the world editor for use by the game designers, or even handling network replication for multiplayer games.

### 16.9.3 Some Common Game Scripting Languages

When implementing a runtime game scripting system, we have one fundamental choice to make: Do we select a third-party commercial or open source language and customize it to suit our needs, or do we design and implement a custom language from scratch?

Creating a custom language from scratch is usually not worth the hassle and the cost of maintenance throughout the project. It can also be difficult or impossible to hire game designers and programmers who are already familiar with a custom, in-house language, so there's usually a training cost as well. However, this is clearly the most flexible and customizable approach, and that flexibility can be worth the investment.

For many studios, it is more convenient to select a reasonably well-known and mature scripting language and extend it with features specific to your game engine. There are a great many third-party scripting languages from which to choose, and many are mature and robust, having been used in a great many projects both within and outside the game industry.

In the following sections, we'll explore a number of custom game scripting languages and a number of game-agnostic languages that are commonly adapted for use in game engines.

#### 16.9.3.1 QuakeC

Id Software's John Carmack implemented a custom scripting language for *Quake*, known as *QuakeC* (QC). This language was essentially a simplified variant of the C programming language with direct hooks into the *Quake* engine. It had no support for pointers or defining arbitrary structs, but it could manipulate *entities* (*Quake*'s name for game objects) in a convenient manner, and it could be used to send and receive/handle game events. QuakeC is an interpreted, imperative, procedural programming language.

The power that QuakeC put into the hands of gamers is one of the factors that gave birth to what is now known as the *mod community*. Scripting languages and other forms of data-driven customization allow gamers to turn many commercial games into all sorts of new gaming experiences—from slight modifications on the original theme to entirely new games.

#### 16.9.3.2 UnrealScript

Probably the best-known example of an entirely custom scripting language is Unreal Engine's *UnrealScript*. This language is based on a C++-like syntactic-

cal style, and it supports most of the concepts that C and C++ programmers have become accustomed to, including classes, local variables, looping, arrays and structs for data organization, strings, hashed string ids (called `FName` in Unreal) and object references (but not free-form pointers). UnrealScript is an interpreted, imperative, object-oriented language.

Epic no longer supports the UnrealScript language. Instead, developers customize game behavior either via the Blueprints graphical “scripting” system, or by writing C++ code.

### 16.9.3.3 Lua

Lua is a well-known and popular scripting language that is easy to integrate into an application such as a game engine. The Lua website (<http://www.lua.org/about.html>) calls the language the “leading scripting language in games.”

According to the Lua website, Lua’s key benefits are:

- *Robust and mature.* Lua has been used on numerous commercial products, including Adobe’s *Photoshop Lightroom*, and many games, including *World of Warcraft*.
- *Good documentation.* Lua’s reference manual [25] is complete and understandable and is available online and in book formats. A number of books have been written about Lua, including [26] and [50].
- *Excellent runtime performance.* Lua executes its byte code more quickly and efficiently than many other scripting languages.
- *Portable.* Out of the box, Lua runs on all flavors of Windows and UNIX, mobile devices and embedded microprocessors. Lua is written in a portable manner, making it easy to adapt to new hardware platforms.
- *Designed for embedded systems.* Lua’s memory footprint is very small (approximately 350 KiB for the interpreter and all libraries).
- *Simple, powerful and extensible.* The core Lua language is very small and simple, but it is designed to support meta-mechanisms that extend its core functionality in virtually limitless ways. For example, Lua itself is not an object-oriented language, but OOP support can and has been added via a meta-mechanism.
- *Free.* Lua is open source and is distributed under the very liberal MIT license.

Lua is a dynamically typed language, meaning that variables don’t have types—only values do. (Every value carries its type information along with it.) Lua’s primary data structure is the *table*, also known as an associative array. A

table is essentially a list of key-value pairs with an optimized ability to index into the array by key.

Lua provides a convenient interface to the C language—the Lua virtual machine can call and manipulate functions written in C as easily as it can those written in Lua itself.

Lua treats blocks of code, called *chunks*, as first-class objects that can be manipulated by the Lua program itself. Code can be executed in source code format or in precompiled byte code format. This allows the virtual machine to execute a string that contains Lua code, just as if the code were compiled into the original program. Lua also supports some powerful advanced programming constructs, including *coroutines*. This is a simple form of cooperative multitasking, in which each thread must yield the CPU to other threads explicitly (rather than being time-sliced as in a preemptive multithreading system).

Lua does have some pitfalls. For example, its flexible function binding mechanism makes it possible (and quite easy) to redefine an important global function like `sin()` to perform a totally different task (which is usually not something one intends to do). But all in all, Lua has proven itself to be an excellent choice for use as a game scripting language.

#### 16.9.3.4 Python

Python is a procedural, object-oriented, dynamically typed scripting language designed with ease of use, integration with other programming languages, and flexibility in mind. Like Lua, Python is a common choice for use as a game scripting language. According to the official Python website (<http://www.python.org>), some of Python's best features include:

- *Clear and readable syntax.* Python code is easy to read, in part because the syntax enforces a specific indentation style. (It actually parses the whitespace used for indentation in order to determine the scope of each line of code.)
- *Reflective language.* Python includes powerful runtime introspection facilities. Classes in Python are first-class objects, meaning they can be manipulated and queried at runtime, just like any other object.
- *Object-oriented.* One advantage of Python over Lua is that OOP is built into the core language. This makes integrating Python with a game's object model a little easier.
- *Modular.* Python supports hierarchical packages, encouraging clean system design and good encapsulation.

- *Exception-based error handling.* Exceptions make error-handling code in Python simpler, more elegant and more localized than similar code in a non-exception-based language.
- *Extensive standard libraries and third-party modules.* Python libraries exist for virtually every task imaginable. (Really!)
- *Embeddable.* Python can be easily embedded into an application, such as a game engine.
- *Extensive documentation.* There's plenty of documentation and tutorials on Python, both online and in book form. A good place to start is the Python website, <http://www.python.org>.

Python syntax is reminiscent of C in many respects (for example, its use of the = operator for assignment and == for equality testing). However, in Python, *code indentation* serves as the only means of defining *scope* (as opposed to C's opening and closing braces). Python's primary data structures are the *list*—a linearly indexed sequence of atomic values or other nested lists—and the *dictionary*—a table of key-value pairs. Each of these two data structures can hold instances of the other, allowing arbitrarily complex data structures to be constructed easily. In addition, *classes*—unified collections of data elements and functions—are built right into the language.

Python supports *duck typing*, which is a style of dynamic typing in which the functional interface of an object determines its type (rather than being defined by a static inheritance hierarchy). In other words, any class that supports a particular interface (i.e., a collection of functions with specific signatures) can be used interchangeably with any other class that supports that same interface. This is a powerful paradigm: In effect, Python supports polymorphism without requiring the use of inheritance. Duck typing is similar in some respects to C++ template metaprogramming, although it is arguably more flexible because the bindings between caller and callee are formed dynamically, at runtime. Duck typing gets its name from the well-known phrase (attributed to James Whitcomb Riley), "If it walks like a duck and quacks like a duck, I would call it a duck." See [http://en.wikipedia.org/wiki/Duck\\_typing](http://en.wikipedia.org/wiki/Duck_typing) for more information on duck typing.

In summary, Python is easy to use and learn, embeds easily into a game engine, integrates well with a game's object model, and can be an excellent and powerful choice as a game scripting language.

#### 16.9.3.5 Pawn/Small/Small-C

*Pawn* is a lightweight, dynamically typed, C-like scripting language created by Marc Peter. The language was formerly known as *Small*, which itself was

an evolution of an earlier subset of the C language called *Small-C*, written by Ron Cain and James Hendrix. It is an interpreted language—the source code is compiled into byte code (also known as P-code), which is interpreted by a virtual machine at runtime.

Pawn was designed to have a small memory footprint and to execute its byte code very quickly. Unlike C, Pawn's variables are dynamically typed. Pawn also supports finite state machines, including state-local variables. This unique feature makes it a good fit for many game applications. Good online documentation is available for Pawn (<http://www.compuphase.com/pawn/pawn.htm>). Pawn is open source and can be used free of charge under the Zlib/libpng license (<http://www.opensource.org/licenses/zlib-license.php>).

Pawn's C-like syntax makes it easy to learn for any C/C++ programmer and easy to integrate with a game engine written in C. Its finite state machine support can be very useful for game programming. It has been used successfully on a number of game projects, including *Freaky Flyers* by Midway. Pawn has shown itself to be a viable game scripting language.

#### 16.9.4 Architectures for Scripting

Script code can play all sorts of roles within a game engine. There's a gamut of possible architectures, from tiny snippets of script code that perform simple functions on behalf of an object or engine system to high-level scripts that manage the operation of the game. Here are just a few of the possible architectures:

- *Scripted callbacks.* In this approach, the engine's functionality is largely hard-coded in the native programming language, but certain key bits of functionality are designed to be customizable. This is often implemented via a *hook function* or *callback*—a user-supplied function that is called by the engine for the purpose of allowing customization. Hook functions can be written in the native language, of course, but they can also be written in a scripting language. For example, when updating game objects during the game loop, the engine might call an optional callback function that can be written in script. This gives users the opportunity to customize the way in which the game object updates itself over time.
- *Scripted event handler.* An event handler is really just a special type of hook function whose purpose is to allow a game object to respond to some relevant occurrence within the game world (e.g., responding to an explosion going off) or within the engine itself (e.g., responding to

an out-of-memory condition). Many game engines allow users to write event handler hooks in script as well as in the native language.

- *Extending game object types, or defining new ones, with script.* Some scripting languages allow game object types that have been implemented in the native language to be extended via script. In fact, callbacks and event handlers are examples of this on a small scale, but the idea can be extended even to the point of allowing entirely new types of game objects to be defined in script. This might be done via *inheritance* (i.e., deriving a class written in script from a class written in the native language) or via *composition* (i.e., attaching an instance of a scripted class to a native game object).
- *Scripted components or properties.* In a component- or property-based game object model, it only makes sense to permit new components or property objects to be constructed partially or entirely in script. This approach was used by Gas Powered Games for *Dungeon Siege*. The game object model was property-based, and it was possible to implement properties in either C++ or Gas Powered Games' custom scripting language, *Skrit* (<http://ds.heavengames.com/library/dstk/skrit/skrit>). By the end of the project, they had approximately 148 scripted property types and 21 native C++ property types.
- *Script-driven engine.* Script might be used to drive an entire engine system. For example, the game object model could conceivably be written entirely in script, calling into the native engine code only when it requires the services of lower-level engine components.
- *Script-driven game.* Some game engines actually flip the relationship between the native language and the scripting language on its head. In these engines, the script code runs the whole show, and the native engine code acts merely as a library that is called to access certain high-speed features of the engine. The Panda3D engine (<http://www.panda3d.org>) is an example of this kind of architecture. Panda3D games can be written entirely in the Python language, and the native engine (implemented in C++) acts like a library that is called by script code. (Panda3D games can also be written entirely in C++.)

### 16.9.5 Features of a Runtime Game Scripting Language

The primary purpose of many game scripting languages is to implement gameplay features, and this is often accomplished by augmenting and customizing a game's object model. In this section, we'll explore some of the most common requirements and features of such a scripting system.

### 16.9.5.1 Interface with the Native Programming Language

In order for a scripting language to be useful, it must not operate in a vacuum. It's imperative for the game engine to be able to execute script code, and it's usually equally important for script code to be capable of initiating operations within the engine as well.

A runtime scripting language's virtual machine (VM) is generally embedded within the game engine. The engine initializes the virtual machine, runs script code whenever required, and manages those scripts' execution. The unit of execution varies depending on the specifics of the language and the game's implementation.

- In a functional scripting language, the *function* is often the primary unit of execution. In order for the engine to call a script function, it must look up the byte code corresponding to the name of the desired function and spawn a virtual machine to execute it (or instruct an existing VM to do so).
- In an object-oriented scripting language, *classes* are typically the primary unit of execution. In such a system, objects can be spawned and destroyed, and methods (member functions) can be invoked on individual class instances.

It's usually beneficial to allow two-way communication between script and native code. Therefore, most scripting languages allowing native code to be invoked from script as well. The details are language- and implementation-specific, but the basic approach is usually to allow certain script functions to be implemented in the native language rather than in the scripting language. To call an engine function, script code simply makes an ordinary function call. The virtual machine detects that the function has a native implementation, looks up the corresponding native function's address (perhaps by name or via some other kind of unique function identifier), and calls it. For example, some or all of the member functions of a Python class or module can be implemented using C functions. Python maintains a data structure, known as a *method table*, that maps the name of each Python function (represented as a string) to the address of the C function that implements it.

#### *Case Study: Naughty Dog's DC Language*

As an example, let's have a brief look at how Naughty Dog's runtime scripting language, a language called DC, was integrated into the engine.

DC is a variant of the Scheme language (which is itself a variant of Lisp language). Chunks of executable code in DC are known as *script lambdas*, which



are the approximate equivalent of functions or code blocks in the Lisp family of languages. A DC programmer writes script lambdas and identifies them by giving them globally unique names. The DC compiler converts these script lambdas into chunks of byte code, which are loaded into memory when the game runs and can be looked up by name using a simple functional interface in C++.

Once the engine has a pointer to a chunk of script lambda byte code, it can execute the code by calling a “virtual machine execution” function in the engine and passing the byte code pointer to it. The function itself is surprisingly simple. It spins in a loop, reading byte code instructions one-by-one, and executing each instruction. When all instructions have been executed, the function returns.

The virtual machine contains a bank of registers, which can hold any kind of data the script may want to deal with. This is implemented using a *variant* data type—a union of all the data types (see Section 16.8.4 for a discussion of variants). Some instructions cause data to be loaded into a register; others cause the data held in a register to be looked up and used. There are instructions for performing all of the mathematical operations available in the language, as well as instructions for performing conditional checks—implementations of DC’s (`if ...`), (`when ...`) and (`cond ...`) instructions and so on.

The virtual machine also supports a *function call stack*. Script lambdas in DC can call other script lambdas (i.e., functions) that have been defined by a script programmer via DC’s (`defun ...`) syntax. Just like any procedural programming language, a stack is needed to keep track of the states of the registers and the return address when one function calls another. In the DC virtual machine, the call stack is literally a stack of register banks—each new function gets its own private bank of registers. This prevents us from having to save off the state of the registers, call the function, and then restore the registers when the called function returns. When the virtual machine encounters a byte code instruction that tells it to call another script lambda, the byte code for that script lambda is looked up by name, a new stack frame is pushed, and execution continues at the first instruction of that script lambda. When the virtual machine encounters a return instruction, the stack frame is popped from the stack, along with the return “address” (which is really just the index of the byte code instruction in the calling script lambda after the one that called the function in the first place).

The following pseudocode should give you a feel for what the core instruction-processing loop of the DC virtual machine looks like:

```

void DcExecuteScript(DCByteCode* pCode)
{
    DCStackFrame* pCurStackFrame
        = DcPushStackFrame(pCode);

    // Keep going until we run out of stack frames (i.e.,
    // the top-level script lambda "function" returns).
    while (pCurStackFrame != nullptr)
    {
        // Get the next instruction. We will never run
        // out, because the return instruction is always
        // last, and it will pop the current stack frame
        // below.
        DCInstruction& instr
            = pCurStackFrame->GetNextInstruction();

        // Perform the operation of the instruction.
        switch (instr.GetOperation())
        {
            case DC_LOAD_REGISTER_IMMEDIATE:
            {
                // Grab the immediate value to be loaded
                // from the instruction.
                Variant& data = instr.GetImmediateValue();

                // Also determine into which register to
                // put it.
                U32 iReg = instr.GetDestRegisterIndex();

                // Grab the register from the stack frame.
                Variant& reg
                    = pCurStackFrame->GetRegister(iReg);

                // Store the immediate data into the
                // register.
                reg = data;
            }
            break;

            // Other load and store register operations...

            case DC_ADD_REGISTERS:
            {
                // Determine the two registers to add. The
                // result will be stored in register A.
                U32 iRegA = instr.GetDestRegisterIndex();
                U32 iRegB = instr.GetSrcRegisterIndex();
            }
        }
    }
}

```

```

        // Grab the 2 register variants from the
        // stack.
        Variant& dataA
            = pCurStackFrame->GetRegister(iRegA);

        Variant& dataB
            = pCurStackFrame->GetRegister(iRegB);

        // Add the registers and store in
        // register A.
        dataA = dataA + dataB;
    }
    break;

// Other math operations...

case DC_CALL_SCRIPT_LAMBDA:
    {
        // Determine in which register the name of
        // the script lambda to call is stored.
        // (Presumably it was loaded by a previous
        // load instr.)
        U32 iReg = instr.GetSrcRegisterIndex();

        // Grab the appropriate register, which
        // contains the name of the lambda to call.
        Variant& lambda
            = pCurStackFrame->GetRegister(iReg);

        // Look up the byte code of the lambda by
        // name.
        DCByteCode* pCalledCode
            = DcLookUpByteCode(lambda.AsStringId());

        // Now "call" the lambda by pushing a new
        // stack frame.
        if (pCalledCode)
        {
            pCurStackFrame
                = DcPushStackFrame(pCalledCode);
        }
    }
    break;

case DC_RETURN:
    {

```

```

        // Just pop the stack frame. If we're in
        // the top lambda on the stack, this
        // function will return nullptr, and the
        // loop will terminate.
        pCurStackFrame = DcPopStackFrame();
    }
    break;

    // Other instructions...

    // ...

    } // end switch
} // end while
}

```

In the above example, we assume that the global functions `DcPushStackFrame()` and `DcPopStackFrame()` manage the stack of register banks for us in some suitable way and that the global function `DcLookUpByteCode()` is capable of looking up any script lambda by name. We won't show implementations of those functions here, because the purpose of this example is simply to show how the inner loop of a script virtual machine might work, not to provide a complete functional implementation.

DC script lambdas can also call native functions—i.e., global functions written in C++ that serve as hooks into the engine itself. When the virtual machine comes across an instruction that calls a native function, the address of the C++ function is looked up by name using a global table that has been hard-coded by the engine programmers. If a suitable C++ function is found, the arguments to the function are taken from registers in the current stack frame, and the function is called. This implies that the C++ function's arguments are always of type `Variant`. If the C++ function returns a value, it too must be a `Variant`, and its value will be stored into a register in the current stack frame for possible use by subsequent instructions.

The global function table might look something like this:

```

typedef Variant DcNativeFunction(U32 argCount,
                                   Variant* aArgs);

struct DcNativeFunctionEntry
{
    StringId          m_name;
    DcNativeFunction* m_pFunc;
};

```

```
DcNativeFunctionEntry g_aNativeFunctionLookupTable[] =
{
    { SID("get-object-pos"), DcGetObjectPos },
    { SID("animate-object"), DcAnimateObject },
    // etc.
};
```

A native DC function implementation might look something like the following. Notice how the Variant arguments are passed to the function as an array. The function must verify that the number of arguments passed to it equals the number of arguments it expects. It must also verify that the types of the argument(s) are as expected and be prepared to handle errors that the DC script programmer may have made when calling the function. At Naughty Dog, we wrote an argument iterator which allows us to extract and verify the arguments one by one in a convenient manner.

```
Variant DcGetObjectPos(U32 argCount, Variant* aArgs)
{
    // Argument iterator expecting at most 2 args.
    DcArgIterator args(argCount, aArgs, 2);

    // Set up a default return value.
    Variant result;
    result.SetAsVector(Vector(0.0f, 0.0f, 0.0f));

    // Use iterator to extract the args. It flags missing
    // or invalid arguments as errors automatically.
    StringId objectName = args.NextStringId();
    Point* pDefaultPos = args.NextPoint(kDcOptional);

    GameObject* pObject
        = GameObject::LookUpByName(objectName);
    if (pObject)
    {
        result.SetAsVector(pObject->GetPosition());
    }
    else
    {
        if (pDefaultPos)
        {
            result.SetAsVector(*pDefaultPos);
        }
        else
        {
            DcErrorMessage("get-object-pos: "
                           "Object '%s' not found.\n",
                           objectName.ToDebugString());
        }
    }
}
```

```
    }  
  
    return result;  
}
```

Note that the function `StringId::ToDebugString()` performs a reverse look-up to convert a string id back to its original string. This requires the game engine to maintain some kind of database mapping each string id to its original string. During development such a database can make life much easier, but because it consumes a lot of memory, the database should be omitted from the final shipped product. (The function name `ToDebugString()` reminds us that the reverse conversion from string id back to string should only be performed for debugging purposes—the game itself must *never* rely on this functionality!)

#### 16.9.5.2 Game Object References

Script functions often need to interact with game objects, which themselves may be implemented partially or entirely in the engine’s native language. The native language’s mechanisms for referencing objects (e.g., pointers or references in C++) won’t necessarily be valid in the scripting language. (It may not support pointers at all, for example.) Therefore, we need to come up with some reliable way for script code to reference game objects.

There are a number of ways to accomplish this. One approach is to refer to objects in script via opaque numeric *handles*. The script code can obtain object handles in various ways. It might be passed a handle by the engine, or it might perform some kind of query, such as asking for the handles of all game objects within a radius of the player or looking up the handle that corresponds to a particular object name. The script can then perform operations on the game object by calling native functions and passing the object’s handle as an argument. On the native language side, the handle is converted back into a pointer to the native object, and then the object can be manipulated as appropriate.

Numeric handles have the benefit of simplicity and should be easy to support in any scripting language that supports integer data. However, they can be unintuitive and difficult to work with. Another alternative is to use the names of the objects, represented as strings, as our handles. This has some interesting benefits over the numeric handle technique. For one thing, strings are human-readable and intuitive to work with. There is a direct correspondence to the names of the objects in the game’s world editor. In addition, we can choose to reserve certain special object names and give them “magic”

meanings. For example, in Naughty Dog’s scripting language, the reserved name “self” always refers to the object to which the currently running script is attached. This allows game designers to write a script, attach it to an object in the game, and then use the script to play an animation on the object by simply writing (`animate 'self name-of-animation`).

Using strings as object handles has its pitfalls, of course. Strings typically occupy more memory than integer ids. And because strings vary in length, dynamic memory allocation is required in order to copy them. String comparisons are slow. Script programmers are apt to make mistakes when typing the names of game objects, which can lead to bugs. In addition, script code can be broken if someone changes the name of an object in the game world editor but forgets to update the name of the object in script.

Hashed string ids overcome most of these problems by converting any strings (regardless of length) into an integer. In theory, hashed string ids enjoy the best of both worlds—they can be read by users just like strings, but they have the runtime performance characteristics of an integer. However, for this to work, your scripting language needs to support hashed string ids in some way. Ideally, we’d like the script compiler to convert our strings into hashed ids for us. That way, the runtime code doesn’t have to deal with the strings at all, only the hashed ids (except possibly for debugging purposes—it’s nice to be able to see the string corresponding to a hashed id in the debugger). However, this isn’t always possible in all scripting languages. Another approach is to allow the user to use strings in script and convert them into hashed ids at runtime, whenever a native function is called.

Naughty Dog’s DC scripting language leverages the concept of “symbols,” which are native to the Scheme programming language, to encode its string ids. Writing `'foo`—or more verbosely, (`quote foo`)—in DC/Scheme corresponds to the string id `SID("foo")` in C++.

### 16.9.5.3 Receiving and Handling Events in Script

Events are a ubiquitous communication mechanism in most game engines. By permitting event handler functions to be written in script, we open up a powerful avenue for customizing the hard-coded behavior of our game.

Events are usually sent to individual objects and handled within the context of that object. Hence, scripted event handlers need to be associated with an object in some way. Some engines use the game object type system for this purpose—scripted event handlers can be registered on a per-object-type basis. This allows different types of game objects to respond in different ways to the same event but ensures that all instances of each type respond in a consistent and uniform way. The event handler functions themselves can be simple script functions, or they can be members of a class if the scripting language is

object-oriented. In either case, the event handler is typically passed a handle to the particular object to which the event was sent, much as C++ member functions are passed the `this` pointer.

In other engines, scripted event handlers are associated with individual object instances rather than with object types. In this approach, different instances of the same type might respond differently to the same event.

There are all sorts of other possibilities, of course. For example, in Naughty Dog's engine (used to create the *Uncharted* and *The Last of Us* series), scripts are objects in their own right. They can be associated with individual game objects, they can be attached to regions (convex volumes that are used to trigger game events), or they can exist as stand-alone objects in the game world. Each script can have multiple states (that is, scripts are finite state machines in the Naughty Dog engine). In turn, each state can have one or more event handler code blocks. When a game object receives an event, it has the option of handling the event in native C++. It also checks for an attached script object, and if one is found, the event is sent to that script's current state. If the state has an event handler for the event, it is called. Otherwise, the script simply ignores the event.

#### 16.9.5.4 Sending Events

Allowing scripts to handle game events that are generated by the engine is certainly a powerful feature. Even more powerful is the ability to generate and send events from script code either back to the engine or to other scripts.

Ideally, we'd like to be able not only to send predefined types of events from script but to define entirely new event types in script. Implementing this is trivial if event types are strings or string ids. To define a new event type, the script programmer simply comes up with a new event type name and types it into his or her script code. This can be a highly flexible way for scripts to communicate with one another. Script A can define a new event type and send it to Script B. If Script B defines an event handler for this type of event, we've implemented a simple way for Script A to "talk" to Script B. In some game engines, event- or message-passing is the only supported means of inter-object communication in script. This can be an elegant yet powerful and flexible solution.

#### 16.9.5.5 Object-Oriented Scripting Languages

Some scripting languages are inherently object-oriented. Others do not support objects directly but provide mechanisms that can be used to implement classes and objects. In many engines, gameplay is implemented via an object-



oriented game object model of some kind. So it makes sense to permit some form of object-oriented programming in script as well.

### *Defining Classes in Scripts*

A class is really just a bunch of data with some associated functions. So any scripting language that permits new data structures to be defined, and provides some way to store and manipulate functions, can be used to implement classes. For example, in Lua, a class can be built out of a table that stores data members and member functions.

### *Inheritance in Script*

Object-oriented languages do not necessarily support inheritance. However, if this feature is available, it can be extremely useful, just as it is in native programming languages like C++.

In the context of game scripting languages, there are two kinds of inheritance: deriving scripted classes from other scripted classes and deriving scripted classes from native classes. If your scripting language is object-oriented, chances are the former is supported out of the box. However, the latter is tough to implement even if the scripting language supports inheritance. The problem is bridging the gap between two languages and two low-level object models. We won't get into the details of how this might be implemented here, as the implementation is bound to be specific to the pair of languages being integrated. UnrealScript is the only scripting language I've encountered that allows scripted classes to derive from native classes in a seamless way.

### *Composition/Aggregation in Script*

We don't need to rely on inheritance to extend a hierarchy of classes—we can also use composition or aggregation to similar effect. In script, then, all we really need is a way to define classes and associate instances of those classes with objects that have been defined in the native programming language. For example, a game object could hold a pointer or reference to an optional component written entirely in script. We can delegate certain key functionality to the script component, if it exists. The script component might have an `Update()` function that is called whenever the game object is updated, and the scripted component might also be permitted to register some of its member functions/methods as event handlers. When an event is sent to the game object, it calls the appropriate event handler on the scripted component, thus giving the script programmer an opportunity to modify or extend the behavior

of the natively implemented game object.

#### 16.9.5.6 Scripted Finite State Machines

Many problems in game programming can be solved naturally using finite state machines (FSMs). For this reason, some engines build the concept of FSMs right into the core game object model. In such engines, every game object can have one or more states, and it is the states—not the game object itself—that contain the update function, event handler functions and so on. Simple game objects can be created by defining a single state, but more complex game objects have the freedom to define multiple states, each with a different update and event-handling behavior.

If your engine supports a state-driven game object model, it makes a lot of sense to provide finite state machine support in the scripting language as well. And of course, even if the core game object model doesn't support finite state machines natively, one can still provide state-driven behavior by using a state machine on the script side. An FSM can be implemented in any programming language by using class instances to represent states, but some scripting languages provide tools especially for this purpose. An object-oriented scripting language might provide custom syntax that allows a class to contain multiple states, or it might provide tools that help the script programmer easily aggregate state objects together within a central hub object and then delegate the update and event-handling functions to it in a straightforward way. But even if your scripting language provides no such features, you can always adopt a methodology for implementing FSMs and follow those conventions in every script you write.

#### 16.9.5.7 Multithreaded Scripts

It's often useful to be able to execute multiple scripts in parallel. This is especially true on today's highly parallelized hardware architectures. If multiple scripts can run at the same time, we are in effect providing *parallel threads of execution* in script code, much like the threads provided by most multitasking operating systems. Of course, the scripts may not actually run in parallel—if they are all running on a single CPU, the CPU must take turns executing each one. However, from the point of view of the script programmer, the paradigm is one of parallel programming.

Most scripting systems that provide parallelism do so via *cooperative multitasking*. This means that a script will execute until it explicitly yields to another script. This is in contrast with a *preemptive multitasking* approach, in which the execution of any script could be interrupted at any time to permit another

script to execute.

One simple approach to cooperative multitasking in script is to permit scripts to explicitly go to sleep, waiting for something relevant to happen. A script might wait for a specified number of seconds to elapse, or it might wait until a particular event is received. It might wait until another thread of execution has reached a predefined synchronization point. Whatever the reason, whenever a script goes to sleep, it puts itself on a list of sleeping script threads and tells the virtual machine that it can start executing another eligible script. The system keeps track of the conditions that will wake up each sleeping script—when one of these conditions becomes true, the script or scripts waiting on the condition are woken up and allowed to continue executing.

To see how this works in practice, let's look at an example of a multi-threaded script. This script manages the animations of two characters and a door. The two characters are instructed to walk up to the door—each one might take a different, and unpredictable, amount of time to reach it. We'll put the script's threads to sleep while they wait for the characters to reach the door. Once they both arrive at the door, one of the two characters opens the door, which it does by playing an "open door" animation. Note that we don't want to hard-code the duration of the animation into the script itself. That way, if the animators change the animation, we won't have to go back and modify our script. So we'll put the threads to sleep again while they wait for the animation to complete. A script that accomplishes this is shown below, using a simple C-like pseudocode syntax.

```
procedure DoorCinematic()
{
    thread Guy1()
    {
        // Ask guy1 to walk to the door.
        CharacterWalkToPoint(guy1, doorPosition);

        // Go to sleep until he gets there.
        WaitUntil(CHARACTER_ARRIVAL);

        // OK, we're there. Tell the other threads
        // via a signal.
        RaiseSignal("Guy1Arrived");

        // Wait for the other guy to arrive as well.
        WaitUntil(SIGNAL, "Guy2Arrived");

        // Now tell guy1 to play the "open door"
        // animation.
        CharacterAnimate(guy1, "OpenDoor");
    }
}
```

```

        WaitFor(ANIMATION_DONE);

        // OK, the door is open. Tell the other threads.
        RaiseSignal("DoorOpen");

        // Now walk thru the door.
        CharacterWalkToPoint(guy1, beyondDoorPosition);
    }

    thread Guy2()
    {
        // Ask guy2 to walk to the door.
        CharacterWalkToPoint(guy2, doorPosition);

        // Go to sleep until he gets there.
        WaitFor(CARACTER_ARRIVAL);

        // OK, we're there. Tell the other threads
        // via a signal.
        RaiseSignal("Guy2Arrived");

        // Wait for the other guy to arrive as well.
        WaitFor(SIGNAL, "Guy1Arrived");

        // Now wait until guy1 opens the door for me.
        WaitFor(SIGNAL, "DoorOpen");

        // OK, the door is open. Now walk thru the door.
        CharacterWalkToPoint(guy2, beyondDoorPosition);
    }
}

```

In the above, we assume that our hypothetical scripting language provides a simple syntax for defining threads of execution within a single function. We define two threads, one for Guy1 and one for Guy2.

The thread for Guy1 tells the character to walk to the door and then goes to sleep waiting for his arrival. We're hand-waving a bit here, but let's imagine that the scripting language magically allows a thread to go to sleep, waiting until a character in the game arrives at a target point to which he was requested to walk. In reality, this might be implemented by arranging for the character to send an event back to the script and then waking the thread up when the event arrives.

Once Guy1 arrives at the door, his thread does two things that warrant further explanation. First, it raises a signal called "Guy1Arrived." Second, it goes to sleep waiting for another signal called "Guy2Arrived." If we look at the thread for Guy2, we see a similar pattern, only reversed. This pattern of

raising a signal and then waiting for another signal is to synchronize the two threads.

In our hypothetical scripting language, a *signal* is just a Boolean flag with a name. The flag starts out false, but when a thread calls `RaiseSignal(name)`, the named flag's value changes to true. Other threads can go to sleep, waiting for a particular named signal to become true. When it does, the sleeping thread(s) wake up and continue executing. In this example, the two threads are using the “Guy1Arrived” and “Guy2Arrived” signals to synchronize with one another. Each thread raises its signal and then waits for the other thread's signal. It does not matter which signal is raised first—only when both signals have been raised will the two threads wake up. And when they do, they will be in perfect synchronization. Two possible scenarios are illustrated in Figure 16.24, one in which Guy1 arrives first, the other in which Guy2 arrives first. As you can see, the order in which the signals are raised is irrelevant, and the threads always end up in sync after both signals have been raised.

## 16.10 High-Level Game Flow

A game object model provides the foundations upon which a rich and entertaining collection of game object types can be implemented with which to populate our game worlds. However, by itself, a game object model only permits us to define the kinds of objects that exist in our game world and how they behave individually. It says nothing of the player's objectives, what happens if he or she completes them, and what fate should befall the player if he or she fails.

For this, we need some kind of system to control high-level game flow. This is often implemented as a finite state machine. Each state usually represents a

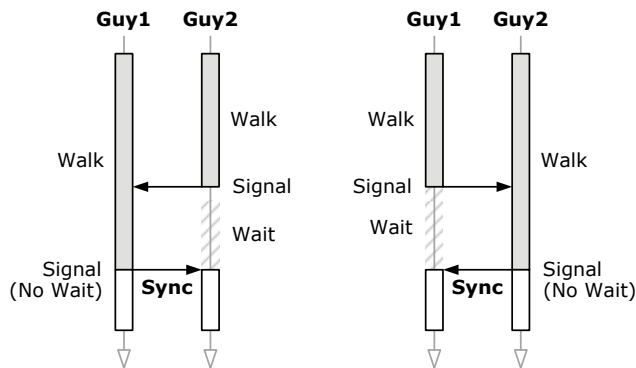


Figure 16.24. Two examples showing how a simple pattern of raising one signal and then waiting on another can be used to synchronize a pair of script threads.

single player objective or encounter and is associated with a particular locale within the virtual game world. As the player completes each task, the state machine advances to the next state, and the player is presented with a new set of goals. The state machine also defines what should happen in the event of the player's failure to accomplish the necessary tasks or objectives. Often, failure sends the player back to the beginning of the current state, so he or she can try again. Sometimes after enough failures, the player has run out of "lives" and will be sent back to the main menu, where he or she can choose to play a new game. The flow of the entire game, from the menus to the first "level" to the last, can be controlled through this high-level state machine.

The task system used in Naughty Dog's *Jak and Daxter*, *Uncharted* and *The Last of Us* franchises is an example of such a state-machine-based system. It allows for linear sequences of states (called *tasks* at Naughty Dog). It also permits parallel tasks, where one task branches out into two or more parallel tasks, which eventually merge back into the main task sequence. This parallel task feature sets the Naughty Dog task graph apart from a regular state machine, since state machines typically can only be in one state at a time.